Workload Orchestration

Use **VCF Automation** or **VCF Operations orchestrator** to configure workload orchestration.

In This Chapter

- Managing VMware Cloud Foundation Operations Orchestrator
- Administering VCF Operations Orchestrator
- Developing Workflows with VCF Operations Orchestrator
- Using the VCF Operations orchestrator Plug-ins

Getting to Know VCF

To apply this documentation, you must be acquainted with the VCF Product Overview, VCF Design, and VCF Release Notes.

Managing VMware Cloud Foundation Operations Orchestrator

VMware Cloud Foundation Operations orchestrator (VCF Operations orchestrator) is a development- and process-automation platform that provides a library of extensible workflows. VCF Operations orchestrator automates management and operational tasks of both Broadcom and third-party applications such as service desks, change management systems, and IT asset management systems.

VCF Operations Orchestrator Overview

VCF Operations orchestrator contains a workflow library and a workflow engine to allow you to create and run workflows that automate orchestration processes. You run workflows on the objects of different technologies that VCF Operations orchestrator accesses through a series of plug-ins.

VCF Operations orchestrator provides a standard set of plug-ins to allow you to orchestrate tasks in the different environments that the plug-ins expose.

VCF Operations orchestrator also presents an open architecture for plugging in external third-party applications to the orchestration platform. You can run workflows on the objects of the plugged-in technologies that you define yourself. VCF Operations orchestrator connects to an authentication provider to manage user accounts and to a preconfigured PostgreSQL database to store information from the workflows that it runs.

Key Features of the VCF Operations Orchestrator Platform

VCF Operations orchestrator is composed of three distinct layers: an orchestration platform that provides the common features required for an orchestration tool, a plug-in architecture to integrate control of subsystems, and a library of workflows. VCF Operations orchestrator is an open platform that can be extended with new plug-ins and content, and can be integrated into larger architectures through a REST API.

VCF Operations orchestrator includes several key features that help with running and managing workflows.

Persistence	A production-grade PostgreSQL database is used to store relevant information, such as processes, workflow states, and the VCF Operations orchestrator configuration.
Central management	VCF Operations orchestrator provides a central tool to manage your processes. The application server-based platform, with full version history, can store scripts and process-related primitives in the same storage location. This way, you can avoid scripts without versioning and proper change control on your servers.

Check-pointing	Every step of a workflow is saved in the database, which prevents data-loss if you must restart the server. This feature is especially useful for long-running processes.
Versioning	All VCF Operations orchestrator platform objects have an associated version history. Version history is useful for basic change management when distributing processes to project stages or locations.
Git integration	You can integrate a Git repository to further improve version and source control of your VCF Operations orchestrator content. With Git, you can manage workflow development across multiple VCF Operations orchestrator instances.
Scripting engine	The Mozilla Rhino JavaScript engine provides a way to create building blocks for the VCF Operations orchestrator client platform. The scripting engine is enhanced with basic version control, variable type checking, name space management, and exception handling. The engine can be used in the following building blocks: • Actions • Workflows • Policies In addition to JavaScript, you can also use Python, Node.js, and PowerShell/PowerCLI runtimes as a way of creating workflows and actions.
Workflow engine	The workflow engine is used by VCF Operations orchestrator for core processes such as: • Processing the workflow schema • Performs the workflow and action runs • Manages user interactions • Creates checkpoints for VCF Operations orchestrator objects The capabilities to manage VCF Operations orchestrator content are provided by the Orchestrator platform. Users, other workflows, schedules, or policies can start workflows.
Policy engine	You can use the policy engine to monitor and generate events to react to changing conditions in the VCF Operations orchestrator server or a plugged-in technology. Policies can aggregate events from the platform or the plug-ins, which helps you to handle changing conditions on any of the integrated technologies.
Development and resources	The VCF Operations orchestrator landing page provides quick access to resources to help you develop your own plug-ins, for use in VCF Operations orchestrator. You will also find information about using the VCF Operations orchestrator REST API to send requests to the VCF Operations orchestrator server.
Security	VCF Operations orchestrator provides the following advanced security functions: • Public Key Infrastructure (PKI) to sign and encrypt content imported and exported between servers. • Digital Rights Management (DRM) to control how exported content can be viewed, edited, and redistributed. • Transport Layer Security (TLS) to provide encrypted communications between the VCF Operations orchestrator deployment, server, and HTTPS access to the web front end. • Advanced access rights management to provide control over access to processes and the objects manipulated by these processes.

Encryption	VCF Operations orchestrator uses a FIPS-compliant Advanced Encryption Standard (AES) with a 256-bit cipher key for encryption of strings. The cipher key is randomly generated and is unique across appliances that are not part of a cluster. All nodes in a cluster share a cipher key.

Installing VCF Operations Orchestrator

To install VCF Operations orchestrator, you must deploy it from an OVF template. For more information, see Deploying VCF Operations orchestrator.

> **Note:**
>
> VCF Automation includes an embedded VCF Operations orchestrator instance which is predeployed. For more information on the VCF Operations orchestrator deployment in VCF Automation, see VCF Operations Orchestrator Design.

For more information on configuring a VCF Operations orchestrator cluster, see Configure a VCF Operations Orchestrator Cluster.

Upgrading VCF Operations Orchestrator

You can upgrade your VMware Aria Automation Orchestrator 8.18.1 deployment to VCF Operations orchestrator 9.1. For more information on upgrading VMware Aria Automation Orchestrator, see Upgrading VCF Operations Orchestrator Deployments.

VCF Operations Orchestrator Licensing

The VCF Operations orchestrator deployment license is taken automatically from the authentication provider as part of the configuration process of the deployment. You do not license VCF Operations orchestrator manually. Supported authentication providers are VCF Automation, vSphere, and VCF SSO. For more information on VCF licensing, see Licensing Overview.

Supported VCF Operations Orchestrator Integrations

VCF Operations orchestrator supports the integrations with the following VCF components. Depending on the integration, you access your VCF Operations orchestrator instance, the objects it exposes, and the VCF Operations orchestrator workflows differently.

Integration	Configuration	How you access VCF Operations orchestrator
VCF Automation	Configure VCF Operations Orchestrator for VCF Automation All Apps Organizations	**Orchestrate** tab in the organization UI
VCF Automation for VM Apps	Configure VCF Operations Orchestrator for VCF Automation VM Apps Organizations	**Orchestrator** tab in the organization UI
vSphere	Configure VCF Operations Orchestrator for vSphere or Configure VCF Operations Orchestrator for vSphere by Using VCF SSO	FQDN address of the VCF Operations orchestrator deployment
VCF Operations	Configure VCF Operations Orchestrator for VCF Operations	FQDN address of the VCF Operations orchestrator deployment

Administering VCF Operations Orchestrator

Use this section to configure, scale, secure, and maintain VCF Operations orchestrator. The tasks are organized by operational lifecycle so you must complete the initial setup before moving to scaling, access control, or server configuration topics.

Before You Begin

Complete the following prerequisites before starting any task in this section.

- VCF Operations orchestrator 9.1 is deployed. For deployment instructions, see *Deploying VMware Cloud Foundation Operations Orchestrator*.
- You have network access to the VCF Operations orchestrator server.
- You have administrator credentials for VCF Operations orchestrator.
- The VMware Cloud Foundation environment is operational and you can reach the VCF Automation instance that the orchestrator is paired with.

Initial VCF Operations Orchestrator Setup

Before you begin automating tasks and managing systems and applications with VCF Operations orchestrator, you must perform additional configuration tasks such as managing license and certificate information.

Activate or Deactivate SSH Access to VCF Operations Orchestrator

You can activate or deactivate SSH access to VCF Operations orchestrator.

- Download and deploy VCF Operations orchestrator.
- Verify that VCF Operations orchestrator is up and running.

1. Log in to the VCF Operations orchestrator appliance command line as **root**.
2. To activate SSH access, run the `/usr/bin/toggle-ssh enable` command.
3. To deactivate SSH access, run the `/usr/bin/toggle-ssh disable` command.

You can configure the SSH settings of the VCF Operations orchestrator appliance by editing the `/etc/ssh/sshd_config` file. By editing this file, you can remove any ciphers or MACs that you do not consider safe.

Managing Certificates in VCF Operations Orchestrator

Issued for a particular server and containing information about the server public key, the certificate allows you to sign all elements created in VCF Operations orchestrator and guarantee authenticity. When the client receives an element from your server, typically a package, the client verifies your identity and decides whether to trust your signature.

You can manage the VCF Operations orchestrator certificates by using the *`ssl_trust_manager`* tagged workflows.

Import a certificate to the VCF Operations orchestrator trust store

VCF Operations orchestrator uses a secure connection to communicate with vCenter, relational database management system (RDBMS), LDAP, Single Sign-On, and other servers. You can import the required TLS certificate from a URL or a PEM-encoded file. Each time you want to use a TLS connection to a server instance, you must import the corresponding certificate from the **Trusted Certificates** tab on the **Certificates** page and import the corresponding TLS certificate.

You can load the TLS certificate in VCF Operations orchestrator from a URL address or a PEM-encoded file.

Option	Description
Import from URL or proxy URL	The URL of the remote server: `https://`*`your_server_IP_address`* or *`your_server_IP_address:port`*
Import from file	Path to the PEM-encoded certificate file. **Note:** You can also import a trusted certificate by running the **Import a trusted certificate from a file** workflow in the VCF Operations orchestrator. The file imported through this workflow must be DER-encoded.

Package signing certificate

Packages exported from an VCF Operations orchestrator server are digitally signed. Import, export, or generate a new certificate to be used for signing packages. Package signing certificates are a form of digital identification that is used to guarantee encrypted communication and a signature for your VCF Operations orchestrator packages.

The VCF Operations orchestrator includes a package signing certificate that is generated automatically, based on the network settings of the appliance. If the network settings of the appliance change, you must generate a new package signing certificate manually. After generating a new package signing certificate, all future exported packages are signed with the new certificate.

Generate a Custom TLS Certificate for VCF Operations Orchestrator

You can use the VCF Operations orchestrator to generate a new TLS certificate for your environment or set an existing custom certificate.

Verify that SSH access for the VCF Operations orchestrator is enabled. See Activate or Deactivate SSH Access to VCF Operations Orchestrator.

The VCF Operations orchestrator includes a Trusted Layer Security (TLS) certificate that is generated automatically, based on the network settings of the appliance. If the network settings of the appliance change, you must generate a new certificate manually. You can create a certificate chain to guarantee encrypted communication and provide a signature for your packages. However, the recipient cannot be sure that the self-signed package is in fact a package issued by your server and not a third party claiming to be you. To prove the identity of your server, use a certificate signed by a Certificate Authority (CA).

VCF Operations orchestrator generates a server certificate that is unique to your environment. The private key is stored in the `vmo_keystore` table of the VCF Operations orchestrator database.

> **Note:**
> To configure your VCF Operations orchestrator to use an existing custom TLS certificate, see Set a Custom TLS Certificate for VCF Operations Orchestrator.

1. Log in to the VCF Operations orchestrator command line over SSH as **root**.
2. Run the `vracli certificate ingress --generate auto --set stdin` command.
3. To apply the custom certificate to your VCF Operations orchestrator, run the deployment script.
   a) Navigate to the `/opt/scripts/` directory.
   ```
   cd /opt/scripts/
   ```
   b) Run the `./deploy.sh` script.

   > **Important:**
   > Do not interrupt the deployment script. You receive the following message when the script finishes running:

   ```
   Prelude has been deployed successfully.
   To access, go to your_orchestrator_address
   ```

To confirm that the new certificate chain is applied, run the `vracli certificate ingress --list` command.

Set a Custom TLS Certificate for VCF Operations Orchestrator

Set a custom TLS Certificate for your VCF Operations orchestrator.

- Verify that SSH access for the VCF Operations orchestrator is enabled. See Activate or Deactivate SSH Access to VCF Operations Orchestrator.
- Verify that the PEM file containing the TLS certificate contains the following components in the set order:
  a. The private key for the certificate.
  b. The primary certificate.
  c. If applicable, the Certificate Authority (CA) intermediate certificate or certificates.
  d. The root CA certificate.

  For example, the TLS certificate can have the following structure:

```
 -----BEGIN RSA PRIVATE KEY-----
<Private Key>
-----END RSA PRIVATE KEY-----
-----BEGIN CERTIFICATE-----
<Primary TLS certificate>
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
<Intermediate certificate>
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
<Root CA certificate>
-----END CERTIFICATE-----
```

The VCF Operations orchestrator includes a Trusted Layer Security (TLS) certificate that is generated automatically, based on the network settings of the appliance.

You can configure your VCF Operations orchestrator to use an existing custom TLS certificate. You can set the certificate by importing the relevant PEM file from your local machine into the VCF Operations orchestrator. You can also set your custom TLS certificate by copying the certificate chain directly into the VCF Operations orchestrator. Both procedures require you to run the `./deploy.sh` script before the new TLS certificate can be used in your VCF Operations orchestrator deployment.

For information on generating a new custom TLS certificate, see Generate a Custom TLS Certificate for VCF Operations Orchestrator.

1. Set the certificate by importing the PEM file into the VCF Operations orchestrator.
   a) Import the certificate PEM from your local machine by running a secure copy (SCP) command from an SSH shell. For Linux, you can use a terminal SCP command:
      ```
      scp ~/PEM_local_filepath/your_cert_file.PEM root@orchestrator_FQDN_or_IP:/
      PEM_orchestrator_filepath/your_cert_file.PEM
      ```
      For Windows, you can use a PuTTY client PSCP command:
      ```
      pscp C:\PEM_local_filepath\your_cert_file.PEM
      root@<orchestrator_FQDN_or_IP>:/PEM_orchestrator_filepath/your_cert_file.PEM
      ```
   b) Log in to the VCF Operations orchestrator command line over SSH as **root**.
   c) Run the `vracli certificate ingress --set`*`your_cert_file.PEM`* command.
2. Optional: Set the certificate by copying the certificate chain directly into the appliance.
   a) Log in to the VCF Operations orchestrator command line over SSH as **root**.
   b) Run the `vracli certificate ingress --set stdin` command.
   c) Copy and paste the certificate chain, and press Ctrl+D.

You have set custom TLS certificate for your VCF Operations orchestrator.

To confirm that the new certificate chain is applied, run the `vracli certificate ingress --list` command.

Scaling and High Availability for VCF Operations Orchestrator

Configure load balancing and scale server resources to support production workloads and high-availability deployments.

Configuring VCF Operations Orchestrator Load Balancing

Load balancers distribute work among servers in high-availability deployments. The system administrator backs up the load balancers regularly at the same time as other components.

SSL Pass-Through

SSL pass-through is used with the load balancing configurations. SSL pass-through is used for these reasons:

- Ease of deployment: not having to deploy VCF Operations orchestrator certificates to the load balancer simplifies deployment and reduces complexity.
- No operational overhead: at the time of certificate renewal, no configuration changes are required to the load balancer.
- Ease of communication: the individual host names of the load-balanced components are the subject alternate name field of the certificates, which allows for effective communication between the client and cluster nodes.

Load Balancer Notifications

It is recommended to activate notifications for scenarios where a VCF Operations orchestrator node in a server pool goes down.

VMware NSX Data Center supports enabling notifications when an alert is raised in VCF Operations and VCF Operations for networks. For more information, see the VCF Operations and VCF Operations for networks documentation.

One-Arm and Multi-Arm Topologies

One-arm and multi-arm deployments route load balancer traffic differently.

- In one-arm deployments, the load balancer is not physically in line of the traffic, which means the load balancer ingress and egress traffic goes through the same network interface. Traffic from the client through the load balancer is network address translated (NAT) with the load balancer as its source address. The nodes send their return traffic to the load balancer before being passed back to the client. Without this reverse packet flow, return traffic would try to reach the client directly, causing connections to fail.
- In a multi-arm configuration, the traffic is routed through the load balancer. The end devices typically have the load balancer as their default gateway. The most common deployment is a one-arm configuration. The same principles apply to multi-arm deployments. For this document, the VCF Operations orchestrator components are deployed in a one-arm configuration. Multi-arm deployments are also supported, and their configuration are similar to the one-arm configuration.

Load Balancing Prerequisites

Before configuring your load balancer, verify that you cover the following prerequisites.

- Avi Load Balancer. Before you can start a high-availability implementation of VCF Operations orchestrator by using Avi Load Balancer, verify that your load balancer topology is configured. For more information on configuring Avi Load Balancer, see the Avi Load Balancer documentation.
- F5 BIG-IP LTM. Before you can start a high-availability implementation of VCF Operations orchestrator by using the F5 BIG-IP LTM load balancer, verify the load balancer is installed and licensed and the DNS server configuration is complete. For more information on configuring F5 BIG-IP LTM, see the F5 documentation.
- NetScaler. Before you can start a high-availability implementation of VCF Operations orchestrator by using the NetScaler load balancer, verify that NetScaler is installed. For more information on configuring NetScaler, see the NetScaler documentation.
- Certificates. Request Certificate Authority (CA) signed certificate containing the load balancer fully qualified domain name (FQDN) and the hostnames of the cluster nodes in the `SubjectAltNames` section. This configuration enables the load balancer to serve traffic without SSL errors.

Configuring Health Monitors

You can use the health status monitor to verify the status of your cluster deployment.

To check the health status of your load balancer, navigate to `https://<load_balancer_FQDN>/vco/api/healthstatus`.

Load Balancer	Health Monitor Configuration
Avi Load Balancer Monitor	When configuring the health monitor for your Avi Load Balancer, verify that you use the following values: • Type - HTTP • Interval - 5 • Timeout - 4 • Successful checks - 3 • Failed checks - 3 • Monitoring port - 8008 • Client request header - GET /health HTTP/1.0 • Response code - 2XX
F-5 BIG IP Load Balancer Monitor	When configuring the health monitor for your F-5 BIG IP load balancer, verify that you use the following values: • Type - HTTP • Interval - 3 • Timeout - 10 • Send string - GET /health HTTP/1.0\r\n\r\n • Receive string - HTTP/1\.(0|1) (200) • Alias service port - 8008
NetScaler Load Balancer Monitor	When configuring the health monitor for your NetScaler load balancer, verify that you use the following values: • Type - HTTP • Interval - 5 • Timeout - 4 • Retries - 3 • Success retries - 1 • HTTP Request/Send String - GET /health • Response code - 200 • Receiving string - none • Destination port - 8008 • Secure - no

Load Balancer Configuration Checklist

You must configure your load balancer before finishing the initial deployment of VCF Operations orchestrator.

During the deployment process of VCF Operations orchestrator, a load balancer typically will route half of the traffic to the secondary nodes, which are not yet be configured, causing the deployment to fail. To avoid these failures and to complete the initial deployment, you must perform these steps.

1. Configure the F5, NSX, or NetScaler load balancer.
2. Turn off the health monitors or change them temporarily to default ICMP, and ensure traffic is still forwarding to your primary nodes.
3. Deactivate all secondary nodes from the load balancer pools.
4. Install and configure all system components as detailed in the VCF Operations orchestrator documentation.
5. When all components are installed, enable all non-primary nodes on the load balancer.
6. Configure the load balancer with all monitors (health checks) enabled.
   After you complete this procedure, update the monitor that you created.
7. Verify that all nodes are in the expected state with the health monitor activated in the load balancer after installation.
   The pool, service groups, and virtual server of the virtual appliance nodes must be available and running. All virtual appliance nodes should be available, running, and activated.

Scale the Heap Memory Size of the VCF Operations orchestrator Server

You can scale the heap memory size of the VCF Operations orchestrator server by creating a custom profile and modifying the resource metrics file.

- Scaling the heap memory of the VCF Operations orchestrator is only applicable for standalone VCF Operations orchestrator instances and is not supported for embedded VCF Operations orchestrator instances in VCF Automation.
- Enable SSH access to the VCF Operations orchestrator. See Activate or Deactivate SSH Access to VCF Operations Orchestrator.
- Increase the RAM of the virtual machine on which VCF Operations orchestrator is deployed to the next suitable increment. Ensure that enough memory remains available for the other services on the appliance by scaling the VCF Operations orchestrator resources before you increase the heap.

You can adjust the heap memory size of the VCF Operations orchestrator server, so your orchestration environment can manage changing workloads. For example, you can increase the heap memory of your VCF Operations orchestrator deployment if you are planning to manage multiple vCenter instances.

1. Log in the VCF Operations orchestrator command line over SSH as **root**.
2. To create the custom profile directory and the required directory tree that is used when the profile is active, run the following script:

```
            vracli cluster exec -- bash -c 'base64 -d <<<
IyBDcmVhdGUgY3VzdG9tIHByb2ZpbGUgZGlyZWN0b3J5Cm1rZGlyIC1wIC9ldGMvdm13YXJlLXByZWx1ZGUvcHJvZmlsZXMvY3VzdG9tLXByb2ZpbGU
| bash'

            vracli cluster exec -- bash -c 'base64 -d <<<
IyBDcmVhdGUgY3VzdG9tIHByb2ZpbGUgZGlyZWN0b3J5Cm1rZGlyIC1wIC9ldGMvdm13YXJlLXByZWx1ZGUvcHJvZmlsZXMvY3VzdG9tLXByb2ZpbGU
| bash'
```

3. Edit the resource metrics file in your custom profile with the desired memory values.

```
vi /etc/vmware-prelude/profiles/custom-profile/helm/prelude_vco/90-resources.yaml
```

4. The `90-resources.yaml` file should contain the following default properties:

```
            polyglotRunnerMemoryRequest: 1000M
polyglotRunnerMemoryLimit: 6000M
polyglotRunnerMemoryLimitVco: 5700M
polyglotRunnerEphemeralStorageLimit: 20Gi
polyglotRunnerEphemeralStorageRequest: 4Gi

ephemeralStorageLimit: "2Gi"
ephemeralStorageRequest: "512Mi"

serverMemoryLimit: 4.4G
serverMemoryRequest: 3.4G
serverJvmMaxRamPercentage: 40
```

```
serverJvmMetaspaceMax: 500m
```

Property Type	Description

Polyglot properties	Memory properties associated with the Polyglot scripting feature. The value of these properties is set in megabytes (M). When editing these values, remember that on average a container needs `64M` of memory. With the default memory limit of `6000M`, you can run approximately 100 Polyglot scripts in parallel.
	If you want to increase the number of Polyglot scripts that can run in parallel, you need to increase the values of the `polyglotRunnerMemoryLimit` and `polyglotRunnerMemoryLimitVco` properties.
	First, edit the memory limit of the `polyglotRunnerMemoryLimit` property and then change the value of `polyglotRunnerMemoryLimitVco` to be `300M` less than the value you set in the `polyglotRunnerMemoryLimit` property.
	The following is an example polyglot memory limit configuration:
	`polyglotRunnerMemoryRequest: 1000M`
	`polyglotRunnerMemoryLimit: 7000M`
	`polyglotRunnerMemoryLimitVco: 6700M`

Server memory properties	The memory properties of the VCF Operations orchestrator server. The value of the `serverMemoryLimit` and `serverMemoryRequest` properties is set in gigabytes (G). The `serverJvmMaxRamPercentage` property sets the JVM heap as a percentage of the server memory.
	The following is an example server memory configuration:
	`serverMemoryLimit: 14G`
	`serverMemoryRequest: 12G`
	`serverJvmHeapMax: 8G`
	`serverJvmMetaspaceMax: 1G`

5. Save the changes to the resource metrics file and run the `deploy.sh` script.

   ```
   /opt/scripts/deploy.sh
   ```

You have changed the heap memory size of your VCF Operations orchestrator server.

User Roles in VCF Operations Orchestrator

As an administrator, you can set permissions that control access to features and content in VCF Operations orchestrator. Access rights are separated into user roles and group permissions. How you assign VCF Operations orchestrator user roles and what roles are available depends on the type of VCF Automation organization you manage.

VCF Operations orchestrator user roles in VCF Automation for VM Apps

User Role	Access Rights
Orchestrator Administrator	Administrators can access all VCF Operations orchestrator features and content, including the content created by specific groups. Responsible for setting user roles, creating and deleting groups, and adding users to groups. Administrators are not limited by group permissions. Tenant administrators from VCF Automation environments used to authenticate VCF Operations orchestrator have **Administrator** rights by default.
Orchestrator Viewer	Viewers have read-only access to all content in the VCF Operations orchestrator, but cannot create, edit, run, or export content. Viewers can also see all groups and group content. Viewers are not limited by group permissions. The Viewer role overwrites the Workflow Designer role when set to the same user account.
Orchestrator Workflow Designer	Access rights that users with the Workflow Developer role depend on their group permissions. See VCF Operations orchestrator Group Permissions.

Note:

Users without an Orchestrator role but with an Organization role can access workflow runs started on their behalf and user interaction approvals that they are assigned to answer.

VCF Operations orchestrator Rights Bundle in VCF Automation for All Apps

The VCF Automation provider administrator must publish the Orchestrator rights bundle for the specific All Apps organization to make VCF Operations orchestrator functionality available to users in this organization. After the rights bundle are published, the organization administrator receives the Extensibility Management rights and Orchestrator Administration right, while the Organization Auditor receives Extensibility view rights and Orchestrator view right with the Organization Administrator or Organization Auditor roles in VCF Automation organizations.

To grant other VCF Automation users access to VCF Operations orchestrator features, such as the option to create and edit workflows, you must assign the relevant VCF Automation rights to these users.

The Orchestrator rights bundle includes the following rights.

Functionality	View rights	Manage rights
Extensibility	View events, subscriptions, actions, action runs, workflows, and workflow runs	Create, update, delete, and deactivate extensibility subscriptions. Create, update, and delete extensibility actions. Cancel and delete extensibility action runs
Custom Actions	View resource actions	Create, update, and delete resource actions
Custom Resources	View custom resources	Create, update, and delete custom resources
Workflows	View Orchestrator workflows published in the catalog	Publish Orchestrator workflows to the catalog
Orchestrator	Orchestrator Viewer	• Orchestrator Administrator • Orchestrator Designer

Note:

The information in the **Orchestrator** row is valid for working inside the VCF Operations orchestrator instance itself.

VCF Operations orchestrator Group Permissions

All users can view and manage all the content that comes with out-of-the-box plug-ins or additionally installed plug-ins. Group permissions control the custom content and what users can view and use, such as workflows, actions, policies, configuration elements, and resource elements

Access rights of users with Administrator and Viewer/Auditor roles are not restricted by group permissions. Access rights of users without an assigned role and users with a workflow designer role depend on the group assigned to them. You can extend the access rights of these users by modifying their group permissions. In this way, you can organize users into common projects. For example, you can create a group that includes users working on developing a custom VCF Operations orchestrator plug-in and allow them to modify only content that is specific to their group.

Role in VM Apps organization	Role in All Apps organization	Group Permissions		
		No assigned group	**Run**	**Run and edit**
Orchestrator Workflow Designer	• Organization User • Custom roles	• View system content. • View and run own content. • Create, run, edit, and delete own content.	• View system content • View and run own runs. • Create, run, edit, and delete own content. • Add own content to the group. • Run group content, but cannot edit it.	• View system content. • View and run own runs. • Create, run, edit, and delete own content. • Add own content to the group. • Run and edit group content. Not available for VCF Operations orchestrator instances authenticated with vSphere.
User without an assigned role	User without an assigned role	• View own runs. • Respond to user interaction requests. These access rights are granted by default to users in VCF Automation and vSphere without an assigned VCF Operations orchestrator role and group.	• View and run own runs. • View and run group content.	• View and run own runs. • View and run group content. To be able to create, edit, and add content, users in this group must be assigned a Workflow Designer role. Not available for VCF Operations orchestrator instances authenticated with vSphere.

Assign VCF Operations Orchestrator Roles to Users in VCF Automation

As an administrator, you can add users to VCF Operations orchestrator and set what features they can view and use.

Role management controls the access of users to VCF Operations orchestrator features. Role management covers both the VCF Operations orchestrator user interface and the API functionality.

1. Open the Access Control page in VCF Automation.
   a) In VCF Automation for All Apps, select **Administer** > **Access Control**.
   b) In VCF Automation for VM Apps, select **Infrastructure** > **Access Control**.
2. Click **Import Users**.
3. Search for the user you want to add to VCF Operations orchestrator.
4. Assign a role to the user.
   - In VCF Automation for All Apps, you can select predefined and custom roles.
   - In VCF Automation for VM Apps, you can select predefined roles only.

5. Click **Save**.

Create VCF Operations Orchestrator Groups in VCF Automation

As an administrator, you can use groups to set what VCF Operations orchestrator content users can view and access.

You can use the VCF Operations orchestrator tab in VCF Automation to set group permissions to VCF Operations orchestrator workflows, actions, policies, configuration elements, resource elements, and packages.

> **Note:**
> Users from VCF Operations orchestrator instances authenticated with vSphere, can only have **Run** group permissions.

1. Navigate to **Orchestrate** > **Administration** > **Groups**.
2. Click **New Group**.
3. On the **Summary** tab, add a name and description for the group.
4. On the **Users** tab, click **Add**.
   a) Search for a user you want to add to the group.
   b) Assign group permissions to the user.
   c) Click **Add**.
5. On the **Items** tab, add VCF Operations orchestrator objects to the group.
6. Click **Create**.

Create a custom VCF Operations orchestrator role in VCF Automation

As a VCF Automation organization administrator, you can create custom roles that define what users can see and do in the organization portal. You can then assign users to those roles.

Create a custom VCF Operations orchestrator role in a VM Apps organization in VCF Automation

1. In VCF Automation for VM Apps, select **Infrastructure** > **Administration** > **Custom Roles** and click **New**.
2. Enter a unique name that you can identify when you assign users to the role.
3. Select the check boxes that correspond to the permissions you want the users to have over the resources.
4. Click **Create**.
5. In the list, click the custom role name and click **Assign**.
6. Add the users or groups that you want to have this role and click **Add**.

Create a custom VCF Operations orchestrator role in an All Apps organization in VCF Automation

1. In VCF Automation, select **Administer** > **Access Control** > **Roles** and click **New**.
2. Enter a unique name that you can identify when you assign users to the role.
3. Select the check boxes that correspond to the permissions you want the users to have over the resources.
4. Click **Save**.
5. For VCF Automation, orgs, once the role is created, the customer navigates to the Users and Groups tabs within Access Control and then can assign the custom role to existing users and/or groups or import new users and/or groups and assign the role.
6. Add the users or groups that you want to have this role and click **Add**.

Organize VCF Operations orchestrator Content with Folders

Manage your VCF Operations orchestrator object inventory using folders.

Create a Folder or Subfolder in VCF Operations orchestrator

Organize your VCF Operations orchestrator objects by using a hierarchical folder structure.

You can create folders and subfolders to organize VCF Operations orchestrator workflows, actions, policies, configuration elements, and resource elements.

1. In VCF Automation, open the **Orchestrate** tab.
2. From the left navigation pane, select an object page, such as **Workflows**.
3. From the top-right, select the tree view icon ().
4. To create a subfolder, select a parent folder from the tree view on the left.
5. Click **New Folder**.
6. Enter a name and description, and click **Save**.
7. Add objects or subfolders to the newly created folder.
8. (Optional) To edit the folder name, click **Edit**.

Move Objects or Folders in VCF Operations orchestrator

Reorganize your VCF Operations orchestrator content by moving the content into another folder.

You cannot move actions between action modules, or move any objects to a root folder. The root folder includes the main object folders and subfolders, but cannot be used to store objects.

1. In VCF Automation, open the **Orchestrate** tab.
2. From the left navigation pane, select an object page, such as **Workflows**.
3. From the top-right, select the tree view icon ().
4. Expand the tree view, and select the object or folder you want to move.
5. Drag the object or folder to its new parent folder.

   Note:

   You can also move objects into new folders directly from the object editor. On the **Summary** tab, click **Select Folder**, and select the new parent folder for the object. Another move option is to select objects from the table on the folder page. This option is useful for performing batch move operations that include multiple VCF Operations orchestrator objects.

Delete a Folder or Subfolder in VCF Operations orchestrator

Delete obsolete folders or subfolders from your VCF Operations orchestrator.

You cannot delete the corresponding root-level folder of each VCF Operations orchestrator object type.

1. In VCF Automation, open the Orchestrate tab.
2. From the left navigation pane, select an object page, such as Workflows.
3. From the top-right, select the tree view icon ().
4. Select the check box next to the folder you want to delete.

   Note:

   To delete a subfolder, select the parent folder from the tree view and then tick the check box.
5. Click **Delete**.
6. If the selected folder is empty, confirm that you want to delete the folder, and click **Delete**.
7. If the selected folder contains objects or subfolders, confirm that you want to delete the folder, and click **Delete**.

   Note:

   You receive the message `Could not delete item 'your_folder_name': Folder 'your_folder_name' is not empty`.

   To delete the folder and all its content, click **Force delete**. Confirm that you want to delete the folder, and click **Delete**.

Note:

You can also perform a batch delete by selecting multiple objects from the table included in the folder menu.

VCF Operations Orchestrator Server Configuration

You can configure system-level properties that control VCF Operations orchestrator server behavior, including scripting runtime restrictions, timeout values, file system access, and authentication options for standalone instances.

Setting Server File System Access for Workflows and Actions in VCF Operations Orchestrator

In VCF Operations orchestrator, the workflows and actions have limited access to specific file system directories. You can extend access to other parts of the server file system by modifying the `js-io-rights.conf` configuration file.

Rules in the js-io-rights.conf File Permitting Write Access to the VCF Operations Orchestrator System

The `js-io-rights.conf` file contains rules that permit write access to defined directories in the server file system.

Mandatory Content of the js-io-rights.conf File

Each line of the `js-io-rights.conf` file must contain the following information.

- A plus (`+`) or minus (`-`) sign to indicate whether rights are permitted or denied
- The read (`r`), write (`w`), and run (`x`) levels of rights
- The path on which to apply the rights.

Note:
The `js-io-rights.conf` file is located in the `/data/vco/usr/lib/vco/app-server/conf/` folder. All content with access to the VCF Operations orchestrator file system must be mapped under this root folder.

Default Content of the js-io-rights.conf File

The default content of the `js-io-rights.conf` configuration file in the automation is as follows:

```
-rwx /
+rwx /var/run/vco
+rx /etc/vco
-rwx /etc/vco/app-server/security/
+rx /var/log/vco/
```

The first two lines in the default `js-io-rights.conf` configuration file allow the following access rights:

`-rwx /`	All access to the file system is denied.
`+rwx /var/run/vco`	Read, write, and run access is permitted in the `/var/run/vco` directory.

Rules in the js-io-rights.conf File

VCF Operations orchestrator resolves access rights in the order they appear in the `js-io-rights.conf` file. Each line can override the previous lines.

Important:
You can permit access to all parts of the file system by setting `+rwx /` in the `js-io-rights.conf` file. However, doing so represents a high security risk.

Set Server File System Access for Workflows and Actions in VCF Operations Orchestrator

To change which parts of the server file system that workflows and the VCF Operations orchestrator API can access, modify the `js-io-rights.conf` configuration file. The `js-io-rights.conf` file is created when a workflow attempts to access the VCF Operations orchestrator server file system.

1. Log in to the VCF Operations orchestrator command line as **root**.
2. Navigate to the `/data/vco/usr/lib/vco/app-server/conf` directory.
3. Open the `js-io-rights.conf` configuration file in a text editor.
4. Add the necessary lines to the `js-io-rights.conf` file to allow or deny access to areas of the file system.
   For example, the following line denies the execution rights in the `/data/vco/var/run/vco/noexec` directory:
   ```
   -x /data/vco/var/run/vco/noexec
   ```
   `/data/vco/var/run/vco/noexec` retains execution rights, but `/data/vco/var/run/vco/noexec/bar` does not. Both directories remain readable and writable.

You modified the access rights to the file system for workflows and for the VCF Operations orchestrator API.

Set JavaScript Access to Java Classes

By default, VCF Operations orchestrator restricts JavaScript access to a limited set of Java classes. If you require JavaScript access to a wider range of Java classes, you must set an VCF Operations orchestrator configuration property.

Allowing the JavaScript engine full access to the Java virtual machine (JVM) presents potential security issues. Malformed or malicious scripts might have access to all the system components to which the user who runs the VCF Operations orchestrator server has access. Therefore, by default the VCF Operations orchestrator JavaScript engine can access only the classes in the `java.util.*` package.

If you require JavaScript access to classes outside of the `java.util.*` package, you can list in a configuration file the Java packages to which to allow JavaScript access. You then set the `com.vmware.scripting.rhino-class-shutter-file` configuration property to point to this file.

1. Create a text configuration file to store the list of Java packages to which to allow JavaScript access.
   For example, to allow JavaScript access to all the classes in the `java.net` package and to the `java.lang.Object` class, you add the following content to the file.
   ```
   java.net.*
   java.lang.Object
   ```
2. Enter a name for the configuration file.
3. Save the configuration file in a subdirectory of `/data/vco/usr/lib/vco`.

   Note:
   The configuration file cannot be saved under another directory.
4. Log in to VCF Operations orchestrator and navigate to **System Settings**.
5. Click **Configuration Properties**.
6. Click **Add Property**.
7. In the **Name** text box, enter `com.vmware.scripting.rhino-class-shutter-file`.
8. In the **Value** text box, enter `/usr/lib/vco/`*`your_configuration_file_subdirectory`*.
9. In the **Description** text box, enter a description for the system property.
10. Click **Add**.
11. To finish setting the configuration property, click **Save**.
12. Wait for the VCF Operations orchestrator server to restart.

The JavaScript engine has access to the Java classes that you specified.

Set Custom Timeout Property in VCF Operations Orchestrator

When vCenter is overloaded, it takes more time to return the response to the VCF Operations orchestrator server than the 20000 milliseconds set by default. To prevent this situation, you must modify the VCF Operations orchestrator configuration file to increase the default timeout period.

If the default timeout period expires before the completion of certain operations, the VCF Operations orchestrator server log contains errors.

```
Operation 'getPropertyContent' total time : '5742228' for 1823 calls, mean time :
'3149.0', min time : '0', max time : '32313' Timeout, unable to get property 'info'
com.vmware.vmo.plugin.vi4.model.TimeoutException
```

1. Log in to VCF Operations orchestrator and navigate to **System Settings**.
2. Click **Configuration Properties**.
3. Click **Add Property**.
4. In the **Name** text box, enter `com.vmware.vmo.plugin.vi4.waitUpdatesTimeout`.
5. In the **Value** text box, enter the new timeout period in milliseconds.
6. Optional: In the **Description** text box, enter a description for the system property.
7. Click **Add**.
8. To finish setting the configuration property, click **Save**.
9. Wait for the VCF Operations orchestrator server to restart.

The value you set overrides the default timeout setting of 20000 milliseconds.

Activate Basic Authentication for VCF Operations Orchestrator

You can activate basic authentication for your external VCF Operations orchestrator instances by setting a configuration property. Activating basic authentication is not supported for embedded VCF Operations orchestrator instances.

> **CAUTION:**
>
> Using basic authentication is not recommended. Instead of basic authentication, use an access token to perform calls to the VCF Operations orchestrator REST API. This approach is applicable for both external and embedded VCF Operations orchestrator instances. For more information on retrieving your access token, see Get Your Access Token for the VCF Automation API.

The basic authentication of your VCF Operations orchestrator deployment is deactivated by default. In certain use cases, you must activate this authentication by setting the `com.vmware.o11n.sso.basic-authentication.enabled` configuration property. For example, you must activate this system property if you are planning on using the VCF Operations orchestrator Multi-Node plug-in for deployments which are authenticated with VCF Automation.

1. Log in to VCF Operations orchestrator and navigate to **System Settings**.
2. Click **Configuration Properties**.
3. Click **Add Property**.
4. Under **Name**, enter `com.vmware.o11n.sso.basic-authentication.enabled`.
5. Under **Value**, enter `true`.
6. Optional: Enter a description for the new system property.
7. Click **Add**.
8. To finish setting the configuration property, click **Save**.
9. Wait for the VCF Operations orchestrator server to restart.

Operations and Maintenance Tasks for VCF Operations Orchestrator

As a VCF Operations orchestrator administrator, you perform routine operational tasks including backup and restore, version history management, and Git-based content lifecycle management.

Back Up and Restore VCF Operations Orchestrator

You can back up and restore your VCF Operations orchestrator deployment by using vSphere virtual machine (VM) snapshots.

The following procedure is based around backing up and restoring a clustered VCF Operations orchestrator deployment. For a standalone VCF Operations orchestrator deployment, you take a vSphere snapshot and revert your deployment from it without the additional cluster-specific steps outlined in this procedure.

> **Note:**
> For more information on using vSphere virtual machine snapshots, see Take a Snapshot of a Virtual Machine and Revert a Virtual Machine Snapshot.

1. Identify the primary node of your VCF Operations orchestrator cluster.
   a) Log in to the VCF Operations orchestrator appliance command line of one of your nodes over SSH as **root**.
   b) Find the node with the `primary` role by running the `kubectl -n prelude exec postgres-0` command.

      ```
      kubectl -n prelude exec postgres-0 --chpst -u postgres repmgr cluster show --terse --compact
      ```

   c) Find the FQDN address of the primary node by running the `kubectl -n prelude get pods` command.

      ```
      kubectl -n prelude get pods -o wide
      ```

2. Back up your VCF Operations orchestrator deployment.
   a) Log in to the vSphere Client.
   b) Take snapshots of your VCF Operations orchestrator nodes.

      When backing up your nodes, follow these requirements:

      - First, back up your secondary nodes and after that, back up the primary node.
      - Use a quiesced snapshot.
      - When taking snapshots of each node in a cluster, verify that you are taking each snapshot in 40 seconds of each other.

      First, back up your secondary nodes and after that, back up the primary node.

      > **Note:**
      >
      > Do not take snapshots of your VCF Operations orchestrator nodes with the **Snapshot the virtual machine's memory** option activated.

3. Restore your VCF Operations orchestrator deployment.
   a) Revert your VCF Operations orchestrator nodes from the snapshots you created in step 2.
   b) Power on the VCF Operations orchestrator nodes.

      When powering on the nodes, you must follow a specific order. First, power on your primary node and after that, power on your secondary nodes.

Git Branching for VCF Operations orchestrator Content Management

Use branching to organize how your VCF Operations orchestrator content is managed in your remote Git server.

By using Git, you can increase the flexibility for your VCF Operations orchestrator developers by providing a centralized repository. For example, you can use Git to manage the workflow development across multiple VCF Operations orchestrator environments.

You can push and pull VCF Operations orchestrator content to and from branches. You can use branching to manage the development of specific groups of VCF Operations orchestrator content, before they are merged back into your main branch.

In this use case, you are using a GitLab project to manage VCF Operations orchestrator content that uses the Python runtime. This use case represents an example of the Git functionality in VCF Operations orchestrator and does not represent the limits of the feature scope.

> **Note:**
> If you are more familiar with GitHub, you can use a GitHub repository for this use case.

Prerequisites

- Create a GitLab project for your VCF Operations orchestrator environment. See Create a project.
- Generate an access token for your GitLab project and copy it to your clipboard for use during the configuration process. See Creating a personal access token.

  > **Note:**
  >
  > For this use case, you are using a GitLab project. If you are more familiar with GitHub, you can use a GitHub repository. For information generating a GitHub token, see Creating a personal access token for the command line.
- Verify that your Git integration is set to push changes to the **Python-branch** Git branch.
- Create a Python-based VCF Operations orchestrator object.

Step 1. Prepare your GitLab environment

1. Log in to your GitLab account.
2. Navigate to your GitLab project.
3. On the left navigation pane, select **Repository** > **Branches**.
4. On the **Overview** tab, click **New Branch**.
5. Under **Branch name**, enter `Python-branch`.
6. Leave the **Create from** option as `master`.
7. Click **Create branch**.

You have created a branch for your Python-based VCF Operations orchestrator objects.

Step 2. Configure a connection to the Git repository

To use Git for management of your VCF Operations orchestrator object inventory, you must configure a connection to your Git repository by using the VCF Operations orchestrator.

Note:

You cannot add multiple Git repositories from different accounts over SSH because VCF Operations orchestrator creates one SSH key for each instance. To add multiple Git repositories, you can add them over HTTP as described in this documentation.

Be aware of the following limitations when using a Git repository with your VCF Operations orchestrator deployment:

- Only SHA-1 commit hashes are supported.
- SHA-1 commit hash conflict between the internal Git repository and the remote Git repository are possible and are not automatically resolved. In such scenarios, all local changes should be discarded.
- Performing manual changes in the Git repository causes merge conflicts when pulling the remote Git repository in the VCF Operations orchestrator. If the end of line sequence is changed both revisions will appear to be the same, but will always result in a merge conflict that must be resolved.

1. In VCF Automation, open the **Orchestrate** tab.
2. Navigate to **Administration** > **Git Repositories**.
3. Click **Add Repository**.
4. Enter the URL address of your Git repository.

   Note:

   For example, https://gitlab.com/*myusername*/*my-vro-repo*. You can also establish a connection with the SSH protocol.
5. Enter the user name for your Git profile.
6. Enter the access token of your Git repository.
7. To validate the connection to the Git repository, click **Validate**.
8. (Optional) Change the name used to identify the repository in the VCF Operations orchestrator.
9. (Optional) Add a short description for the connected Git repository.
10. To activate the connected Git repository, click **Make active repository**.

    Note:

    Only one Git repository can be active at a time. You can change the active Git repository from the **Git Repositories** page.
11. To finish the configuration process, click **Save**.
12. Navigate back to the **Git Repositories** menu and confirm that the status of the repository is **Active**.

Step 3. Push changes to the Git repository

You can push a local change set to a Git repository. Each change set can consist of one or more modified VCF Operations orchestrator objects.

Note:

You can push a local change set to a Git repository. Each change set can consist of one or more modified VCF Operations orchestrator objects.

1. In VCF Automation, open the **Orchestrate** tab.
2. Edit your Python action.
   a. Navigate to **Library** > **Actions**, and select your Python action.
   b. Make some minor changes to the action, such as changing the description.
   c. Save the action.
3. Push your changes to the Git repository.

   Note:

   You can also push local changes on a per-object level by clicking the **Version** option displayed at the bottom of the object editor.

   a. Navigate to **Administration** > **Git History**.

Note:

The **Git History** page displays the current differences between the local version branch and the selected Git repository branch. You can expand the entry for any modified VCF Operations orchestrator object to view the version differences. You can discard a local change set by selecting **Discard all**.

b. Click **Push**.
c. Enter a commit title.
d. (Optional) Enter a short description for the commit.
e. Select the changes to your Python action that you want to push to the Git repository.

4. To finish pushing the local change set to the Git repository, click **Push**.

After you finish development in your Git branch, merge it with the main branch. See How to create a merge request.

Developing Workflows with VCF Operations Orchestrator

Workflow development involves using the workflow editor in VCF Operations orchestrator, the VCF Operations orchestrator APIs, and the JavaScript, Python, Node.js, and PowerShell scripting languages.

What Does Workflow Development Involve

The process for developing a custom workflow involves a series of steps. Depending on the type of workflow that you are developing, you might follow a different sequence of step or skip a step. For example, you can create a workflow without custom scripting.

Generally, developing a workflow involves the following steps.

- Create a new workflow or create a duplicate of an existing workflow from the standard library.
- Provide general information about the workflow.
- Define the input and output parameters of the workflow.
- Lay out and link the workflow schema to define the logical flow of the workflow.
- Bind the input and output parameters of each schema element to workflow variables.
- Write the necessary scripts for scriptable task elements or custom decision elements.
- Create the workflow presentation from the **Input Form** tab.
- Validate the workflow.

Developing VCF Operations orchestrator Workflows Best Practices

You can follow best practices for developing VCF Operations orchestrator workflows by multiple users and in a clustered environment.

- Each developer has a dedicated test standalone VCF Operations orchestrator instance for creating and developing workflows. As VCF Automation can only have single VCF Operations orchestrator instance integrated to it, this must be a separate standalone instance.
- Workflows are saved as maven projects on a shared source code control system.
- To ensure optimal performance of the VCF Operations orchestrator production deployment, it is best to import workflows in a scheduled period.

Working with the Workflow Editor

You create and edit workflows in the Orchestrator workflow editor. The workflow editor is the Integrated Development Environment (IDE) of VCF Operations orchestrator for developing workflows. The workflow editor consists of tabs where you edit the components of your workflows.

- Summary. Provide general information about the workflow, such as the workflow name, description, tags, and version number. The Summary tab can also be used to configure the server restart behavior and workflow run failure behavior.
- Variables. Define the variables for your workflow. Variables were previously known as attributes.
- Input/Output. Define the input and output parameters of your workflow. The input parameters provide the data that the workflow processes during the workflow run. The output parameters are provided when the workflow run finishes.

- Schema. Build your workflow by dragging workflow schema elements from the workflow palette on the left side of the Schema tab. By clicking an element in the schema diagram, you can define and edit the element's behavior.
- Input Form. Define the layout of the user input dialog box that appears when users run a workflow. You arrange the parameters and variables into presentation steps and tabs to ease identification of parameters in the input parameters dialog box. You define the constraints on the input parameters that users can provide in the presentation by setting the parameter properties. You can also add external validation for your workflow by using actions.
- Version History. View and manage the version history of the workflow. Compare and restore versions and push and pull workflows to and from your integrated Git repository.
- Audit. View information about events related to the workflow such as, when it was saved, when a workflow run was performed, and when a workflow run was finished.

VCF Operations Orchestrator Workflow Architecture

Workflows consist of a schema, workflow variables, and input and output parameters. The workflow schema is the main component of a workflow as it defines all the workflow elements and the logical connections between them. The workflow variables and parameters are the properties that workflows use to transfer data. VCF Operations orchestrator saves a workflow token every time a workflow runs, recording the details of that specific workflow run.

Workflow Parameters

Workflows receive input parameters and generate output parameters when they run.

Input Parameters

Most workflows require a certain set of input parameters to run. An input parameter is an argument that the workflow processes when it runs. The user, an application, another workflow, or an action passes input parameters to a workflow for the workflow to process when it runs.

For example, if a workflow resets a virtual machine, the workflow requires the name of the virtual machine as an input parameter.

Output Parameters

A workflow's output parameters represent the result from the workflow run. Output parameters can change when a workflow or a workflow element runs. During a workflow run, the workflow can receive the output parameters of other workflows as input parameters.

For example, if a workflow creates a snapshot of a virtual machine, the output parameter for the workflow is the resulting snapshot.

Workflow Variables

Workflow elements process data that they receive as input parameters, and set the resulting data as workflow variables or output parameters.

Read-only workflow variables act as global constants for a workflow. Writable variables act as a workflow’s global variables.

You can use variables to transfer data between the elements of a workflow. You can obtain variables in the following ways:

- Define variables when you create a workflow.
- Set the output parameter of a workflow element as a workflow variable.
- Inherit variables from a configuration element.

Workflow Schema

A workflow schema is a graphical representation that shows the workflow as a flow diagram of interconnected workflow elements. The workflow schema is the most important element of a workflow as it determines its logic.

Workflow Presentation

When users run a workflow, they provide the values for the input parameters of the workflow in the workflow presentation. When you organize the workflow presentation, consider the type and number of input parameters of the workflow. You can configure the presentation of your workflows from the Input Form tab of the workflow editor.

Workflow Tokens

A workflow token represents a workflow that is running or has finished running.

A workflow is an abstract description of a process that defines a generic sequence of steps and a generic set of required input parameters. When you run a workflow with a set of input parameters, you receive an instance of this abstract workflow that behaves according to the specific input parameters you give it. This specific instance of a finished or a running workflow is called a workflow token.

Workflow Token Attributes

Workflow token attributes are the specific parameters with which a workflow token runs. The workflow token attributes are an aggregation of the workflow's global variables and the specific input and output parameters with which you run the workflow token.

Core Concepts for Using Python, Node.js, and PowerShell Scripts in VCF Operations Orchestrator

When creating your script for use in VCF Operations orchestrator, you must verify that your script has the correct structure and formatting.

Supported Runtimes

For developing VCF Operations orchestrator actions and workflows, you can use the following runtimes:

- Python 3.10
- Node.js 20
- PowerCLI 13 (PowerShell 7.4)

  Note:
  The PowerCLI runtime includes PowerShell and the following modules: VMware.PowerCLI, PowerNSX, PowervRA.
- PowerShell 7.4

  Note:
  The PowerCLI 12 and Node.js 18 runtimes are deprecated and will be removed in a future release.

You can add any custom source code to the new runtimes, but to accept context and inputs, and return a result from and to the VCF Operations orchestrator engine, you must follow the correct functional format.

Scripting Recommendations

For simpler scripting tasks, you can add **Scriptable task** elements to your workflow schema. You can use VCF Operations orchestrator actions for more complex scripting tasks.

Using actions provides two specific benefits:

- Actions can be created, updated, imported, and exported independently from workflows.
- Actions are standalone objects that can be run and debugged in their own environment which can lead to a smoother development process. See Running and Debugging Actions .

Script Function Requirements

The default name for your script function is `handler`. The function accepts two arguments, context and input. Context is a map object, containing system information. For example, `vroURL` can contain the URL of the VCF Operations orchestrator instance you want to call, while `executionId` contains the token ID of a workflow run.

An input is a map object containing all inputs that are provided to the actions. For example, if you define an input in your action called `myInput`, you can access it from the inputs argument, such as `inputs.myInput` or `inputs["myInput"]`, depending on your runtime. Anything that you return from the function, is the result of the action. Therefore, the return type of your action must correspond to the type of content that the script returns in VCF Operations orchestrator. If you return a primitive number, the action return type must be a number type. If you return a string, the action return type must be a string type. If you return a complex object, the return type must be mapped to either `Properties` or `Composite Type`. These same principles also apply to arrays.

Supported input and output parameter types for Python, Node.js, and PowerShell runtimes:

- `String`
- `Number`
- `Boolean`
- `Date`
- `Properties`
- `Composite Type`

Define the Entry Handler

By default, the value of the entry handler is `handler.handler`. This value means that the VCF Operations orchestrator engine looks for a top-level file in your ZIP package called `handler.py`, `handler.js`, or `handler.ps1`, that includes a function called `handler`. Any differences to the names of the function and handler file must be reflected in the value of the entry handler. For example, if your main handler is called `index.js` and your function is called `callMe`, you must set the value for the entry handler to `index.callMe`.

Scripting Environments in VCF Operations Orchestrator

You can add modules and libraries for use in your VCF Operations orchestrator scripts as dependencies directly from VCF Operations orchestrator.

Core Concepts

When creating scripts for VCF Operations orchestrator, you must ensure your script follows the correct structure and formatting requirements.

VCF Operations orchestrator supports the following runtimes for developing actions and workflows.

Runtime	Version	Notes
Python	3.10	
Node.js	22	
PowerCLI	13 (PowerShell 7.4)	Includes VMware PowerCLI, PowerNSX, and PowervRA modules
PowerShell	7.4	

Scripting Recommendations

You can define your script either in a **Scriptable Task** element added to your workflow schema or an action. Scriptable tasks are useful for simpler scripts, while actions should be used for more complex scripts. Aside from enabling more complex scripting, actions have the following advantages over scriptable tasks.

- **Independence:** actions can be created, updated, imported, and exported independently from workflows.
- **Reusability:** actions are standalone objects that can be used across multiple workflows.
- **Easier development:** Actions can be run and debugged in their own environment, leading to a smoother development process. See Running and Debugging Actions.

Script Function Requirements

The default script function has the following structure:

- **Function name:** the default function name is `Handler`.
- **Arguments:** each script has two required arguments: `context` and `inputs`.

The `context` argument is a map object that contains the system information about the runtime environment. The object contains the following parameters:

Parameter	Description
`vroURL`	The URL address of your VCF Operations orchestrator deployment.
`executionId`	The token ID of the workflow run.

The `inputs` argument is a map object containing all inputs provided to the action. For example, if you define an input called `myInput`, you can access it through the following arguments, depending your selected runtime:

- **Python:** `inputs["myInput"]`
- **Node.js:** `inputs.myInput` or `inputs["myInput"]`
- **PowerShell:** `$inputs.myInput` or `$inputs["myInput"]`

The value returned from your function becomes the result of action when you run it. The return type of the action in VCF Operations orchestrator must match the type returned by your script.

Script Return Type	VCF Operations orchestrator Return Type
Number	Number
String	String
Boolean	Boolean
Date	Date
Complex Object	Properties or Composite Type
Array	Array of corresponding type

Regarding supported parameter types, the following types are supported for both input and output parameters:

- String
- Number
- Boolean
- Date
- Properties
- Composite Type

Entry Handler for ZIP-Based Actions

When uploading actions as ZIP packages, you must define the entry handler to specify which file and function to run. This can either be the default entry handler or a custom entry handler.

The default entry handler value is `handler.handler`. This handler directs VCF Operations orchestrator to look up a top-level file, which can be either `handler.py`, `handler.js`, or `handler.ps1`, depending on the selected runtime. After looking up the file, the handler runs the `handler` function included in the file.

If the top-level file or the function it contains has a different name, you must create a custom entry handler with a value different from `handler.handler`. The format for the entry handler must be *<<filename>.<function_name>*. The following table includes example file and function names and the value for the respective custom entry handlers.

File Name	Function Name	Entry Handler Value
`handler.js`	`handler`	`handler.handler` (default)
`index.js`	`callMe`	`index.callMe`
`main.py`	`process`	`main.process`
`script.ps1`	`execute`	`script.execute`

Configuring Scripting Environments in VCF Operations Orchestrator

You can use the VCF Operations orchestrator client to create a script environment for use in your workflows.

Overview

Script environments allow you to add external modules and libraries (like npm packages) as dependencies for your scripts. Each environment can bundle multiple dependencies together for reuse across workflows and actions.

> **Note:**
>
> Scripting environments can be exported and imported as part of packages, similarly to workflows and actions.

Environment Repository Settings

Custom repositories in **Manage Repositories** are always available out of the box. However, the default public registries and the additional repository feature must be manually enabled.

To enable your repositories, navigate to **System Settings** > **Performance and Features** > **Environment Repositories**.

Repository Setting	Default State	Description
Default Public Repositories	Disabled	Makes the default public registry, `npmjs.org` for Node.js, `PyPI` for Python, `NuGet` for PowerShell/PowerCLI, available as a repository option when adding dependencies.
Suppress Default Environment Warning	Disabled	Only visible when **Default Public Repositories** is enabled. Suppresses the warning shown when a dependency uses a public repository.
Additional Repositories	Disabled	For Python, PowerShell, and PowerCLI environments, enables the **Additional Repository** field for dependencies, similar to `--extra-index-url` in pip. This allows a dependency to specify a secondary repository alongside its primary repository.

Adding a Repository

1. Navigate to **Assets** > **Environments** and click **Manage Repositories**.
2. Click **Add REPO**.
3. Configure the repository:
   a. Under **Name**, enter a name for your repository.
   b. Under **Runtime Environment**, select the runtime type.
   c. Under **Location**, enter the repisoory URL.

The following table includes the supported repository types for each runtime environment.

Runtime Environment	Repository Type
Node.js	npm
Python	Simple (PyPI-compatible)
PowerShell	NuGet v2
PowerCLI	NuGet v2

Support for Multiple Repositories

For Python, PowerShell, and PowerCLI environments, you can configure multiple repositories. Using multiple repositories is useful scenarios such as the following:

- You have custom packages in a private/hosted repository.
- Dependencies for those packages are in a different repository or public registry.

Node.js environments are limited to a single repository.

> **Note:**
>
> Support for multiple repositories must be activated from **System Settings**.

Selecting Repositories for Dependencies

When adding dependencies to your environment, you can specify which repository to use for each individual dependency. This allows you to mix packages from different sources within the same environment.

Creating a Script Environment

The following procedure demonstrates how you can create a Node.js environment with the `lodash` library.

1. Create the environment.
   a. Log in to the VCF Operations orchestrator.
   b. Navigate to **Assets** > **Environemnts**.
   c. Click **New Environment**.
2. Configure the general settings
   a. Under the **General** tab, enter a name for your environment. For example, `lodash-env`.
   b. Optional: Enter a description, version number, tags, and group permissions.
3. Add dependencies.
   a. Select the **Definitions** tab.
   b. Under **Runtime Environment**, verify that **Node.js** is selected.
   c. (Optional) Adjust the memory limit and timeout properties.
   d. Under **Dependencies**, click **Add.**
   e. Enter the dependency details:
      a. **Name**: `lodash`
      b. **Version**: `4.17.21`
      c. **Repository**: `Default Public Repository`

      > **Note:**
      >
      > The default public repository must be enabled from **System Settings**.

   f. Click **Add**.
4. (Optional) Add environment variables.
5. Create and verify the environment.
   a. To finalize the environment, click Create.
   b. After creating the environment, navigate to the **Download Logs** tab.
   c. Verify that your dependencies are available.

Using the Environment

1. Open a workflow scripting element or action.
2. Select your environment for the Runtime Environment drop-down,

Your dependencies, such as **lodash**, are now available in your script.

Runtime Limits for Scripts

Some Python, Node.js, or PowerShell scripts can exceed the default resource limits of VCF Operations orchestrator. As a result, you must adjust memory and timeout values for your action scripts.

Default Resource Limits

VCF Operations orchestrator applies the following default limits to action scripts:

Runtime	Memory	Timeout
Python	64 MB	180 seconds
Node.js	64 MB	180 seconds
PowerShell	150 MB	180 seconds
PowerCLI	150 MB	180 seconds

When to Adjust Limits

Your action scripts can fail if they exceed the default limits. Common scenarios include:

- **High memory usage:** scripts that use multiple third-party dependency modules.
- **Long-running operations:** scripts that perform time-intensive operations, such as API calls, data processing, and others.
- **Large data processing**: scripts that handle large datasets or complex objects.

How to Change Runtime Limits

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Actions** and select your action.
3. Select the **Script** tab.
4. Select your runtime environment from the drop-down,
5. Under the **Inputs** section, locate the **memory** and **timeout** fields.
6. Adjust the fields as needed. The value of the **memory** field must be in MB, while the value of the **timeout** field must be in seconds.
7. To apply the new runtime limits, click **Save.**
8. To test the runtime limits, click **Run.**
9. Verify if the action run finished successfully.

Alternatively, if your script requires high resources consistently, you can break up the script into multiple smaller scriptable tasks. This approach has the following advantages:

- Reduces the memory footprint for each task.
- Improves maintainability and debugging.
- Allows for better resource management across workflow runs.

Configuring a Photon OS Container for Python Packages

Depending on the operating system (OS) used for compiling your Python script, your workflows or actions can fail after importing the relevant ZIP archive to the VCF Operations orchestrator.

Install Docker. See Get Docker.

The OS of the runtime container used for Python in VCF Operations orchestrator is based on Photon 3.0. Python script packages compiled for another OS, such as Linux for example, are incompatible with the runtime container. This problem can cause the Python script to fail, when you attempt to use it as part of your VCF Operations orchestrator workflows or actions. In such a scenario, you receive the following error message in your logs:

```
-04:00errorCannot find module action
```

To resolve this problem, you must install the required Python package in a Photon OS container folder.

1. Navigate to the parent folder of your Python script.
2. Create a container with the base Photon image by mounting a container folder to your parent folder.

   Note:
   The following script is a singular Docker command that you must run in its entirety to create a suitable container.

   ```
   docker run -ti -v
   $(pwd)/<name_of_folder_that_containes_your_python_script>/:/
   <name_of_folder_that_containes_your_python_script>
   photon:3.0
   ```

3. Install Python in the container.

   ```
   tdnf install -y python3-3.7.5-5.ph3 python3-pip-3.7.5-5.ph3
   ```

4. Navigate to the container folder that includes your Python script.
5. Add your Python script and packages.

   Note:
   Install packages required for your Python script in the `lib` folder.

   ```
   pip3 install <package_name> -t lib/
   ```

6. Exit the container and navigate to the local folder you mounted to the container.
7. Compress all relevant files and folders into a ZIP archive.
8. Import the ZIP archive into VCF Operations orchestrator and validate the script by running it as part of an action.

Building VCF Operations Orchestrator Workflows

A workflow is a series of actions and decisions that you run sequentially. VCF Operations orchestrator provides a library of workflows that perform common management tasks. VCF Operations orchestrator also provides libraries of the individual actions that the workflows perform.

Workflows combine actions, decisions, and results that, when performed in a particular order, finish a specific task or a specific process in a virtual environment. Workflows perform tasks such as provisioning virtual machines, backing up, performing regular maintenance, sending emails, performing SSH operations, managing the physical infrastructure, and other general utility operations. Workflows accept inputs according to their function. You can create workflows that run according to defined schedules, or that run if certain anticipated events occur. Information can be provided by you, by other users, by another workflow or action, or by an external process such as a Web service call from an application. Workflows perform some validation and filtering of information before they run.

Workflows can call upon other workflows. For example, you can have workflow that calls up another workflow to create a new virtual machine.

You create workflows by using the VCF Operations orchestrator interface's integrated development environment (IDE), that provides access to the workflow library and the ability to run workflows on the workflow engine. The workflow engine can also take objects from external libraries that you plug in to VCF Operations orchestrator. This feature allows you to customize processes or implement functions that third-party applications provide.

Create a Workflow in VCF Operations Orchestrator

You can use VCF Operations orchestrator to create and edit workflows.

1. Select **Library** > **Workflows**.
2. Click **New Workflow**.
3. Enter the name of the new workflow and click **Create**.
4. Use the workflow editor to configure the variables, workflow inputs and outputs, schema structure, and presentation of the workflow.
5. To finish editing the workflow, click **Save**.

   Note:
   You can track changes to workflows in the **Version History** tab. For more information, see Managing VCF Operations orchestrator Version History.

You can use the VCF Operations orchestrator token replay feature to optimize the performance of your workflows. For more information, see Using Workflow Token Replay in VCF Operations orchestrator.

Create VCF Operations Orchestrator Workflow Parameters and Variables

After you create a workflow, you must define the global variables, input parameters, and output parameters of the workflow.

Workflow variables store data that workflows process internally. Unlike input and output parameters, variables values can be preset. Variables were previously known as attributes. Workflow input parameters are data provided by an outside source, such as a user or another workflow. Workflow output parameters are data that the workflow delivers when it finishes running.

When you create new variables and input parameters, you select a parameter type. If the parameter type is an object that exists in Orchestrator, such as a configuration element, for example, then you can search for this object in the Value text box.

Create Workflow Parameters

You can use input and output parameters to pass data into and out of the workflow.

You can create the parameters of a workflow in the workflow editor. The input parameters are the initial data that the workflow requires to run. Users provide the values for the input parameters when they run the workflow. The output parameters are the data the workflow returns when it finishes running.

1. Navigate to **Library** > **Workflows**, and select your workflow.
2. Select the**Inputs/Outputs** tab.
3. To add a parameter, click **New**.
4. Create a workflow parameter.
   1. Select if you want to create an input or output parameter.
   2. Enter a name.

      Note:

      The parameter name must be a single word that can only contain letters, numbers, and the dollar ("$") and underscore ("_") symbols.
   3. Enter a description.
   4. Set the parameter type.

Note:

To create a composite type parameter, that includes multiple different parameter types, select New Composite Type from the Type text box.

5. To create an array, click the Array check box.
6. To finish creating your parameter, click Create.

5. Set a default value for your input parameters.
   1. Select the Input Form tab.
   2. Select your input parameter and click the Values tab.
   3. Select the value source for your input parameter, by expanding the Default value text box. The available value sources are dependant on the parameter type set in the previous step.

Value Source	Decription
Constant	The default value source. Enter a constant default value for your input parameter.
Conditional value	Add one or more conditional expressions that define your input parameter value.
External source	Import an external action that configures the value of your input parameter. The filtering of available actions is done by parameter type.
Bind field	Bind the value of the select input parameter to another input parameter or variable.
Computed value	Configure the input parameter value by using operators. The type of available operators depends on the input parameter type. For example, for number type input parameters, you can select the arithmetic operators Add, Substract, Multiply. **Note:** Compute value is not available for SDK type input parameters, such as AD:UserGroup, SSH:Host, and so on.

6. Configure output parameter exception handling.
   1. Select the Schema tab, and add a workflow element to the schema diagram.
   2. Add your output parameters to the workflow element.
   3. Expand the Exception handling menu item.
   4. Select a variable that you want to use to handle output parameter exceptions.

You have created an input or output parameter for the workflow. To complete the configuration, click **Save**.

Create Workflow Variables

Workflow variables are the data that workflows process.

1. Navigate to **Library** > **Workflows**, and select your workflow.
2. Select the Variables tab.
3. To add a variable, click New.
4. Create a workflow variable.
   1. Enter a name.

      Note:

      The variable name must be a single word that can only contain letters, numbers, and the dollar ("$") and underscore ("_") symbols.
   2. Enter a description.
   3. Set the parameter type.

Note:

To create a composite type variable, that includes multiple different parameter types, select New Composite Type from the Type text box.

4. To create an array, click the Array check box.
5. Add a value for your workflow variable.
6. To finish creating your variable, click **Create**.

You created a variable for the workflow. To complete the configuration, click **Save**.

Edit Workflows and Actions from the Parent Workflow

Edit workflows and actions directly from the parent workflow in the VCF Operations orchestrator.

Create a workflow that calls up another workflow, action, or both.

Editing child workflows and actions directly from the parent workflow can help streamline workflow development.

1. Navigate to **Library** > **Workflows**, and select your workflow.
2. Select the **Schema** tab.
3. Depending on the object type, double-click the **Workflow element** or **Action element** in the workflow canvas.
4. Edit the object.
5. To finish editing the child workflow or action, click **Save**.
6. To return to the parent workflow, close the object editor.

Working with the VCF Operations Orchestrator Workflow Schema

A workflow schema is a graphical representation of a workflow that shows the workflow as a flow diagram of interconnected workflow elements. The workflow schema defines the logical flow of a workflow.

Building a Workflow in the Workflow Schema

Workflow schemas consist of a sequence of schema elements. Workflow schema elements are the building blocks of the workflow, and can represent decisions, scripted tasks, actions, exception handlers, or even other workflows.

You build workflows in the workflow editor by dragging schema elements from the workflow palette on the left of the workflow editor into the workflow schema diagram.

Edit a Workflow Schema

You build a workflow by creating a sequence of schema elements that define the logical flow of the workflow.

By default, all elements in the workflow schema are linked. Links between the elements are represented as arrows. When you add a new element to the workflow schema, you must drag it onto an arrow or an existing workflow element that is not linked to a next element. After you add workflow elements to the schema, you can delete existing links and create new links to define the logical flow of the workflow.

A workflow schema must have at least one End workflow element, but it can have several.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Click the **Schema** tab.
3. Drag a schema element from the **Generic** menu in the left pane, to the workflow schema.
4. Click the element you dragged to the workflow schema and enter a name.

Note:

You should provide each element in your workflow with a unique name that describes its function. For example, your workflow might include two **Scriptable task** elements, one used to create a virtual machine, and another

used to delete a virtual machine. In such a use case, you might want to name your elements, `Create VM` and `Delete VM`. You cannot rename the `End workflow` and `Throw exception` elements.

5. Right-click an element in the schema and select either **Arrange vertically** or **Arrange horizontally** to manage the schema orientation around the specific element.
6. Drag schema elements from the **Basic**, **Log**, or **Network** menus to the workflow schema.

   Note:

   You can edit the names of the elements in the **Basic**, **Log**, or **Network** menus. You cannot edit their scripting.
7. Repeat this procedure until you have added all the required schema elements to the workflow schema.
8. To complete the configuration, click **Save**.

Next, define the properties and, if applicable, the scripting of the elements you added to the workflow schema and link and bind them all together.

Promote Input and Output Parameters

You can promote the input and output parameters of a child element to the parent workflow.

You can promote a custom variable that you have defined in the workflow editor. You can promote predefined variables only by replacing an input parameter with a variable of matching type.

Note:

If you promote a predefined variable and assign a custom value to it, a duplicate variable is created to avoid overwriting the value of the original variable. The duplicate variable retains the name of the original variable and increments the numerical value at the end of the variable's name.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Add a `Workflow` element or an `Action` element to the workflow schema.
4. Use the selected element to add a workflow or an action to the workflow.
   The following notification appears at the top of the schema pane.No Inputs/Outputs were auto-binded. To create workflow-level variables/parameters for other inputs/outputs, please promote them.
5. Click **Promote**.
6. Select the mapping type for each input parameter.

Option	Description
Input	The argument is mapped to an input workflow parameter.
Skip	The argument is mapped to a NULL value.
Variable	The argument is mapped to a workflow variable.

7. Select the mapping type for each output parameter.

Option	Description
Output	The argument is mapped to an output workflow parameter.
Skip	The argument is mapped to a NULL value.
Variable	The argument is mapped to a workflow variable.

8. Click **Promote**.
9. To complete the configuration, click **Save**.

You promoted parameters to the parent workflow.

Schema Elements and Schema Element Properties

The workflow editor presents the workflow schema elements in menus on the Schema tab. You can use the schema elements available in the Schema tab to build a workflow.

Generic Schema Elements

Element	Description
Start	The starting point of the workflow. All workflows contain this element. Always present on the Schema tab. A workflow can have only one start element. Start elements have one output and no input, and cannot be removed from the workflow schema.
Scriptable task	General-purpose tasks you define. You write JavaScript functions in this element.
Decision	A boolean function. Decision elements take one input parameter and return either true or false. The type of decision that the element makes depends on the type of the input parameter. Decision elements let the workflow branch into different directions, depending on the input parameter the decision element receives. If the received input parameter corresponds to an expected value, the workflow continues along a certain route. If the input is not the expected value, the workflow continues on an alternative path.
Decision activity	A boolean function. A decision activity runs a workflow and binds its output parameters to a true or a false path.
User interaction	Lets users pass new input parameters to the workflow. You can design how the user interaction element presents the request for input parameters and place constraints on the parameters that users can provide. When a running workflow arrives at a user interaction element, it enters a passive state and prompts the user for input. You can set a timeout period within which the users must provide input. The workflow resumes according to the data the user passes to it, or returns an exception if the timeout period expires. While it is waiting for the user to respond, the workflow token is in the waiting state.
Waiting timer	Used by long-running workflows. When a running workflow arrives at a Waiting Timer element, it enters a passive state. You set an absolute date at which the workflow resumes running. While it is waiting for the date, the workflow token is in the waiting-signal state.
Waiting event	Used in long-running workflows. When a running workflow arrives at a Waiting Event element, it enters a passive state. You define a trigger event that the workflow awaits before it resumes running. While it is waiting for the event, the workflow token is in the waiting-signal state.
End workflow	The end point of a workflow. You can have multiple end elements in a schema, to represent the various possible outcomes of the workflow. End elements have one input with no output. When a workflow reaches an End Workflow element, the workflow token enters the completed state.
Thrown exception	Creates an exception and stops the workflow. Multiple occurrences of this element can be present in the workflow schema. Exception elements have one input parameter, which can only be of the String type, and have no output parameter. When a workflow reaches an Exception element, the workflow token enters the failed state.
Workflow note	Lets you annotate sections of the workflow. You can stretch notes to delineate sections of the workflow. You can change the background color of the notes to differentiate workflow zones. Workflow notes provide only visual information, to help you understand the schema.
Action element	Calls on an action from the Orchestrator libraries of actions. When a workflow reaches an action element, it calls and runs that action.

Element	Description
Workflow element	Starts another workflow synchronously. When a workflow reaches a Workflow element in its schema, it runs that workflow as part of its own process. The original workflow continues only after the called workflow completes its run.
Foreach element	Runs a workflow on every element from an array. For example, you can run the Rename Virtual Machine workflow on all virtual machines from a folder.
Asynchronous workflow	Starts a workflow asynchronously. When a workflow reaches an asynchronous workflow element, it starts that workflow and continues its own run. The original workflow does not wait for the called workflow to complete.
Schedule workflow	Creates a task to run the workflow at a set time, and then the workflow continues its run.
Nested workflow	Starts several workflows simultaneously. You can decide to nest local workflows and remote workflows that are in a different Orchestrator server. You can also run workflows with different credentials. The workflow waits for all the nested workflows to complete before continuing its run.
Default error handler	Handles workflow errors that are not caught by standard error handlers. You can use any available schema elements to handle errors.
Switch	Switches to alternative workflow paths, based on a workflow variable or parameter.

Predefined Task Schema Elements

In addition to generic schema elements, you can add other pre-defined task elements to your workflows. Pre-defined elements are non-editable scripted elements that perform standard tasks that workflows commonly use. They are grouped into three menus.

Menu	Elements
Basic	• Sleep • Change credential • Wait until date • Wait for custom event • Send custom event • Increase counter • Decrease counter
Log	• System log • System warning • System error • Server log • Server warning • Server error • System and Server log • System and Server warning • System and Server error
Network	• HTTP post • HTTP get

Schema Element Properties

Schema elements have properties that you define and edit in the Schema tab of the workflow palette. You access the properties of a schema element by clicking an element that you have dragged into the workflow schema. The properties

of the element appear in tabs on the right-hand side of the workflow editor. The following table provides more information about each tab.

Certain workflow schema elements, like the Decision element, do not include multiple tabs to manage their properties.

Tab	Description	Applies to these schema elements
General	Provide general information about the schema element. Details options: • Name • Next element • Business status This property is a brief description of what the element does. When a workflow is running, the workflow token shows the business status of each element as it runs. This feature is useful for tracking workflow status. • Description Action options: • Inputs • Outputs Exception handling options: • Output exception binding	• Scriptable task • Action element • Sleep • Change credential • Wait until date • Wait for custom event • Send custom event • Increase counter • Decrease counter • System log • System warning • System error • Server log • Server warning • Server error • System and Server log • System and Server warning • System and Server error • HTTP post • HTTP get
Scripting	Enter the JavaScript script used in the schema element.	• Scriptable task • Action element • Sleep • Change credential • Wait until date • Wait for custom event • Send custom event • Increase counter • Decrease counter • System log • System warning • System error • Server log • Server warning • Server error • System and Server log • System and Server warning • System and Server error • HTTP post • HTTP get • Asynchrous element • Schedule workflow • Switch

Tab	Description	Applies to these schema elements
Info	The **Info** tab has the same function as the **General** tab. Details options: • Name • Next element • Business status This property is a brief description of what the element does. When a workflow is running, the workflow token shows the business status of each element as it runs. This feature is useful for tracking workflow status. • Description	• Asynchrous element • Schedule workflow • Switch
Cases	Define which other element the **Switch** element switches to.	• Switch

Links and Bindings

Links between elements determine the logical flow of the workflow. Bindings populate elements with data from other elements by binding input and output parameters to workflow variables.

To understand links and bindings, you must understand the difference between the logical flow of a workflow and the data flow of a workflow.

Logical Flow of a Workflow

The logical flow of a workflow is the progression of the workflow from one element to the next in the schema as the workflow runs. You define the logical flow of the workflow by linking elements in the schema.

The standard path is the path that the workflow takes through the logical flow if all elements run as expected. The exception path is the path that the workflow takes through the logical flow if an element does not run as expected.

Different styles of arrows in the workflow schema denote the different paths that the workflow can take through its logical flow.

- A blue arrow denotes the standard path that the workflow takes from one element to the next.
- A green arrow denotes the path that the workflow takes if a Boolean decision element returns true.
- A red dashed arrow denotes the path that the workflow takes if a Boolean decision element returns false. Alternatively, a red dashed arrow denotes the exception path that the workflow takes if a workflow element does not run correctly.

The following figure shows an example workflow schema that demonstrates the different paths that workflows can take.

This example workflow can take the following paths through its logical flow.

- Standard path,truedecision result, no exceptions.
  - The decision element returnstrue.
  - The `SnapVMsInResourcePool` workflow runs successfully.
  - The `sendHtmlEmail` action runs successfully.
  - The workflow ends successfully in thecompletedstate.
- Standard path, `falsedecision` result, no exceptions.
  - The decision element returnsfalse.
  - The operation the scriptable task element defines runs successfully.
  - ThesendHtmlEmailaction runs successfully.
  - The workflow ends successfully in thecompletedstate.
- `truedecision` result, exception.

  - The decision element returnstrue.
  - TheSnapVMsInResourcePoolworkflow encounters an error.
  - The workflow returns an exception and stops in thefailedstate.
- `falsedecision` result, exception.
  - The decision element returnsfalse.
  - The operation the Scriptable task element defines encounters an error.
  - The workflow returns an exception and stops in thefailedstate.

Element Links

Links connect schema elements and define the logical flow of the workflow from one element to the next.

Elements can usually set only one outgoing link to another element in the workflow and one exception link to an element that defines its exception behavior. The outgoing link defines the standard path of the workflow. The exception link defines the exception path of the workflow. In most cases, a single schema element can receive incoming standard path links from multiple elements.

The following elements are exceptions to the preceding statements.

- The `Start` Workflow element cannot receive incoming links and has no exception link.
- `Exception` elements can receive multiple incoming exception links, and have no outgoing or exception links.
- `Decision` elements have two outgoing links that define the paths the workflow takes depending on the decision's true or false result. Decisions have no exception link.
- `End Workflow` elements cannot have outgoing links or exception links.

Create Standard Path Links

Standard path links determine the normal run of the workflow.

- Open a workflow for editing in the workflow editor.
- Verify that the **Schema** tab of the workflow editor contains elements.

When you link one element to another, you always link the elements in the order in which they run in the workflow. You always start from the element that runs first to create a link between two elements.

- Place the pointer on the element that you want to connect to another element. A blue and a red arrow appear on the element's right.
- Place the pointer on the blue arrow. The blue arrow enlarges.
- Left-click the blue arrow, hold down the left mouse button, and move the pointer to the target element. A blue arrow appears between the two elements and a green rectangle appears around the target element.
- Release the left mouse button. The blue arrow remains between the two elements.

A standard path now links the elements.

The elements are joined, but you have not defined the data flow. You must define the input and output bindings to bind incoming and outgoing data to workflow variables.

Data Flow of a Workflow

The data flow of a workflow is the manner in which the workflow element input and output parameters bind to workflow variables as each element of the workflow runs. You define the data flow of a workflow by using schema element bindings.

When an element in the workflow schema runs, it requires data in the form of input parameters. It takes the data for its input parameters by binding to a workflow variables that you set when you create the workflow, or by binding to an variable that a preceding element in the workflow set when it ran.

The element processes the data, possibly transforms it, and generates the results of its run in the form of output parameters. The element binds its resulting output parameters to new workflow variables that it creates. Other elements in

the schema can bind to these new workflow variables as their input parameters. The workflow can generate the variables as its output parameters at the end of its run.

Element bindings

You must bind all workflow input and output parameters. Bindings set data in the elements, and define the output and exception behavior of the elements. Links define the logical flow of the workflow, whereas bindings define the data flow.

To set data in an element, generate output parameters from the element after processing, and handle any errors that might occur when the element runs, you must set the element binding.

Binding	What to do
Input binding	Set a schema element's incoming data. For most schema elements, you can do this from the **Inputs/Outputs** menu of the **General** tab.
Output binding	Define the output parameters used when an element finishes its run. For most schema elements, you can do this from the **Inputs/Outputs** menu of the **General** tab.
Output Exception bindings	Link to exception handlers if the element encounters an exception when it runs.

Output exception bindings write values into the bound source parameter.

If the element changes the values of the input parameters that it receives when it runs, you must bind them to a workflow variable by using an output exception binding. Binding the element's output parameters to workflow variables lets other elements that follow it in the workflow schema to take those output parameters as their input parameters.

A common mistake when creating workflows is not to bind output parameter values to reflect the changes that the element makes to the workflow variables.

> **Important:**
>
> When you add an element that requires input and output parameters of a type that you have already defined in the workflow, VCF Operations orchestrator sets the bindings to these parameters. You must verify that the parameters that VCF Operations orchestrator binds are correct, in case the workflow defines different parameters of the same type to which the element can bind.

Define Element Bindings

After you link elements to create the logical flow of the workflow, you define element bindings to define how each element processes the data it receives and generates.

Verify that you have a workflow schema in the **Schema** tab of the workflow editor, and that you have created links between the elements.

1. Navigate to **Library** > **Workflows**, and select your workflow.
2. Select the **Schema** tab.
3. Select a workflow element.
4. On the **General** tab of the element configuration window, expand the **Inputs/Outputs** option.
5. Add the appropriate workflow input or output parameters from the list of existing parameters and variables.
6. To save your changes to the workflow schema, click **Save**.

You defined the input parameters that the element receives and the output parameters that it generates, and bound them to workflow variables and parameters.

You can create forks in the path of the workflow by defining decisions.

Decisions

Workflows can implement decision functions that define different courses of action according to a Boolean true or false statement.

Decisions are forks in the workflow. Workflow decisions are made according to inputs provided by you, by other workflows, by applications, or by the environment in which the workflow is running. The value of the input parameter that the decision element receives determines which branch of the fork the workflow takes. For example, a workflow decision might receive the power status of a given virtual machine as its input. If the virtual machine is powered on, the workflow takes a certain path through its logical flow. If the virtual machine is powered off, the workflow takes a different path.

Decisions are always Boolean functions. The only possible outcomes for each decision are true or false.

Custom Decisions

Custom decisions differ from standard decisions in that you define the decision statement in a script. Custom decisions return true or false according to the statement you define, as the following example shows.

```
if (decision_statement){
 return true;
}else{
 return false;
}
```

Create Decision Element Links

Decision elements differ from other elements in a workflow. They have only true or false output parameters. Decision elements have no exception linking.

Verify that the **Schema** tab of the workflow editor contains elements, including at least one decision element that is not linked to other elements.

1. Place the mouse pointer on a decision element to link it to two other elements that define two possible branches in the workflow.
   A blue arrow and a red arrow appear on the element's right.
2. Place the pointer on the blue arrow, and while keeping the left mouse button pressed, move the pointer to the target element.
   A green arrow appears between the two elements and the target element turns green. The green arrow represents the true path the workflow takes if the input parameter or variable received by the decision element matches the decision statement.
3. Release the left mouse button.
   The green arrow remains between the two elements. You have defined the path the workflow takes when the decision element receives the expected value.
4. Place the pointer on the decision element, hold down the left mouse button, and move the pointer to the target element.
   A dotted red arrow appears between the two elements and the target element turns green. The red arrow represents the false path that the workflow takes if the input parameter or variable received by the decision element does not match the decision statement.
5. Release the left mouse button.
   The dotted red arrow remains between the two elements. You have defined the path the workflow takes when the decision element receives unexpected input.

You have defined the possible true or false paths that the workflow takes depending on the input parameter or variable the decision element receives.

Exception Handling

Exception handling catches any errors that occur when a schema element runs. Exception handling defines how the schema element behaves when the error occurs.

All elements in a workflow, except for decisions and start and end elements, contain a specific output parameter type that serves only for handling exceptions. If an element encounters an error during its run, it can send an error signal to an exception handler. Exception handlers catch the error and react according to the errors they receive. If the exception handlers you define cannot handle a certain error, you can bind an element's exception output parameter to an Exception element, which ends the workflow run in the failed state.

Exceptions act as a try and catch sequence within a workflow element. If you do not need to handle a given exception in an element, you do not have to bind that element's exception output parameter.

The output parameter type for exceptions is always an errorCode object.

Create Exception Bindings

Elements can set bindings that define how the workflow behaves when it encounters an error in that element.

Verify that the **Schema** tab of the workflow editor contains elements.

1. Navigate to **Library** > **Workflows**, and select your workflow.
2. Navigate to the **Schema** tab.
3. Select the element for which you want to define exception binding.
4. On the **General** tab, expand the **Exception Handling** option.
5. Select a workflow variable you want to use for exception handling.
6. To finish editing the workflow, click **Save**.

You defined how the element handles exceptions.

Foreach Elements and Composite Types

You can insert a Foreach element in the workflow that you develop to run a subworkflow that iterates over arrays of parameters or variables. To improve the understanding and readability of the workflow, you can group several workflow parameters of different types that are logically connected in a single type that is called a composite type.

Using Foreach Elements

A Foreach element runs a subworkflow iteratively over an array of input parameters or variables. You can select the arrays over which the subworkflow is run, and can pass the values for the elements of such an array when you run the workflow. The subworkflow runs as many times as the number of elements that you have defined in the array.

If you have a configuration element that contains an array of variables, you can run a workflow that iterates over these variables in a Foreach element.

For example, suppose that you have 10 virtual machines in a folder that you want to rename. To do this, you must insert a Foreach element in a workflow and define the Rename virtual machine workflow as a subworkflow in the element. The Rename virtual machine workflow takes two input parameters, a virtual machine and its new name. You can promote these parameters as input to the current workflow, and as a result, they become arrays over which the Rename virtual machine workflow will iterate. When you run your workflow, you can specify the 10 virtual machines in the folder and their new names. Every time the workflow runs, it takes an element from the array of the virtual machines and an element from the array of the new names for the virtual machines.

Using Composite Types

A composite type is a group of more than one input parameter or variables that are connected logically but are of different types. In a Foreach element, you can bind a group of parameters as a composite value. In this way, the Foreach element takes the values for the grouped parameters at once in every subsequent run of the workflow.

For example, suppose that you are about to rename a virtual machine. You need the virtual machine object and its new name. If you have to rename multiple virtual machines, you need two arrays, one for the virtual machines and one for their names. These two arrays are not explicitly connected. A composite type lets you have one array where each element contains both the virtual machine and its new name. In this way, the connection between those two parameters in case of multiple values is specified explicitly and not implied by the workflow schema.

Note:

You cannot run a workflow that contains composite types from vSphere.

Define a Foreach Element

If you want to run a subworkflow multiple times by passing different values for its parameters or variables in every subsequent run, you can insert a **Foreach element** in the parent workflow.

When you insert a**Foreach element**, you must select at least one array over which the**Foreach element**iterates. An array element can have different values for each subsequent workflow run.

If the subworkflow has output parameters, you must select the output parameters of the**Foreach element**in which to accumulate workflow outputs, so that the subworkflow can iterate over them as well.

1. Navigate to **Library** > **Workflows**, and select your workflow.
2. In the workflow editor, select the **Schema** tab.
3. From the **Generic** menu, drag a **Foreach element** in the workflow schema.
4. Create variables for the items in the array.
5. Under **Workflow**, select the workflow you want to add to the **Foreach element**.
   The input and output parameters of the selected workflow are added to the **Foreach element**.
6. Add an iteration error handler.
7. Add a variable you want to use for exception handling.
8. To complete the configuration, click **Save**.

You defined a **Foreach element** in your workflow. The **Foreach element** runs a workflow that takes as parameters every element from the array of parameters or variables that you have defined.

For parameters or variables that are not defined as arrays, the workflow takes the same value in every subsequent run.

Rename Virtual Machines by Using a Foreach Element

You can use a **Foreach element** to rename several virtual machines at once. You have to insert a **Foreach element** in a workflow and promote the vm and the newName parameters as input to the current workflow. In this way, when you run the workflow, you specify the virtual machines to rename and the new names for the virtual machines. The virtual machines are included as elements in the array that you created for the vm parameter. The new names for the virtual machines are included in the array that you created for the newName parameter.

Add a Switch Activity to a VCF Operations Orchestrator Workflow

You can add a basic switch activity to a workflow schema that defines the switch cases based on workflow variables or parameters.

Every switch activity can have multiple switch cases. Every switch case is defined by a condition related to a variable or a parameter. If the condition is fulfilled, the workflow run switches to a corresponding workflow element that you define. If none of the specified conditions are fulfilled, the workflow run switches to a default workflow element that you define.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Drag a **Switch** element to the appropriate element in the workflow schema.
3. In the **Cases** tab, add or delete switch cases.
4. Define the condition for each switch case.
5. Select the corresponding workflow element for each switch case.

6. Select the default workflow element to switch to.
7. Click **Save**.

You defined the switch case conditions and workflow paths.

Managing Actions in VCF Operations Orchestrator

You can modify your VCF Operations orchestrator workflows by adding actions scripts.

VCF Operations orchestrator provides libraries of predefined actions and an action editor for custom action scripts. Actions represent individual functions that you use as building blocks in workflows.

Actions are JavaScript functions. Actions can take multiple input parameters and have a single return value. Actions can call on any object in the VCF Operations orchestrator API, or objects in any API that you import into VCF Operations orchestrator by using a plug-in.

When a workflow runs, an action takes input parameters from the workflow's variables. These variables can be either the workflow's initial input parameters, or variables that other elements in the workflow set when they run.

The action editor includes an autocomplete feature for scripts and an API Explorer featuring the available scripting types and their documentation.

Create Actions in VCF Operations Orchestrator

You can use VCF Operations orchestrator to create, edit, and delete action scripts.

Before creating a Python, Node.js, or PowerShell script, verify that you are familiar with the core concepts for developing VCF Operations orchestrator compatible scripts that use these runtimes. See Core Concepts for Using Python, Node.js, and PowerShell Scripts in VCF Operations Orchestrator.

You can use the following runtimes when creating actions:

- Python 3.10
- Node.js 20
- PowerCLI 13 (PowerShell 7.4)

  Note:
  The PowerCLI runtime includes PowerShell and the following modules: VMware.PowerCLI, PowerNSX, PowervRA.
- PowerShell 7.4

  Note:
  The PowerCLI 12 and Node.js 18 runtimes are deprecated and will be removed in a future release.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Actions**.
3. Click **New Action**.
4. On the **General** tab, enter the name and module name of the action.

   Note:
   The name and module name must be unique for every action. The action name must be a valid JavaScript function. The action name must be a single word that can only contain letters, numbers, and the dollar ("$") and underscore ("_") symbols. The module name must consist of words separated by the dot (".") character.
5. Optional: Create a description, version number, tags, and group permissions for the action.
6. On the **Script** tab, add action inputs, select the return type of the output, and write the script.
7. To finish editing the action, click **Create**.

   A message states that the action is saved.

Requesting User Interactions While a Workflow Runs

A workflow can sometimes require additional input parameters from an outside source while it runs. These input parameters can come from another application or workflow, or the user can provide them directly.

For example, if a certain event occurs while a workflow runs, the workflow can request human interaction to decide what course of action to take. The workflow waits before continuing, either until the user responds to the request for information, or until the waiting time exceeds a possible timeout period. If the waiting time exceeds the timeout period, the workflow returns an exception.

The default variables for user interactions aresecurity.group and timeout.date. When you set the security.group variable to a given LDAP user group, you limit the permission to respond to the user interaction request to members of that user group.

When you set the `timeout.date` variable, you set a time and date until which the workflow waits for the information from the user. You can set an absolute date, or you can create a scripted workflow element to calculate a time relative to the current time.

Workflows requiring further user interaction suspend operations until the requested input parameters are provided by the user. Workflows define which users can provide the requested information and send requests for interaction accordingly. Workflows waiting for user input are displayed in the **Recent Workflow Runs** panel of the VCF Operations orchestrator dashboard and the top-right notification menu.

Add a User Interaction to a Workflow

You request input parameters from users during a workflow run by adding a **User interaction** schema element to the workflow. When a workflow encounters a **User interaction** element, it suspends its run and waits for the user to provide the required data.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab and drag a **User interaction** element to the appropriate position in the workflow schema.
3. Enter a name and description for the user interaction.
4. Under **Options**, configure the security group, security assignees, and timeout date variables for the user interaction.
5. To save your workflow, click **Save**.

You added a user interaction element to a workflow. When the workflow reaches this element, it waits for information from the user before continuing its run.

Set the User Interaction security.group Variable

The security.group variable of a user interaction element sets which users or groups of users have permission to respond to the user interaction.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Select the **User interaction** element.
4. Expand the **Options** menu.
5. Click **Select variable** for thesecurity.groupsource parameter to set which users can respond to the user interaction.
6. Click **Create New**.
7. Name the parameter.
8. Under **Value**, enter the name of the LDAP user group you want to add to the element.
9. Click **Create**.
10. To save the changes to your workflow, click **Save**.

You set the security.group variable for the user interaction.

To configure the timeout period of the user interaction, set thetimeout.date variable.

- To set the timeout to an absolute date and time, see Set the timeout.date Variable to an Absolute Date.
- To create a function to calculate a timeout that is relative to the current date and time, see Calculate a Relative Timeout for User Interactions.

Set the timeout.date Variable to an Absolute Date

You set the timeout.date variable for a user interaction to set how long the workflow waits for a user to respond to a user interaction.

Add a user interaction element to the workflow schema.

You set an absolute time and date in the Date object. When the time on the given date arrives, the workflow that is waiting for a user interaction times out and ends in the Failed state. For example, you can set the user interaction to timeout at midday on February 12th. To calculate a timeout that is relative to the current time and date, see Calculate a Relative Timeout for User Interactions.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Select the **User interaction** element.
4. Expand the **Options** menu.
5. Click **Select variable** for thetimeout.datesource parameter to set which users can respond to the user interaction.
6. Click **Create New**.
7. Name the parameter.
8. Under **Value**, use the calendar to select an absolute date and time until which the workflow waits for the user to respond.
9. Click **Create**.
10. To save the changes to your workflow, click **Save**.

You set the timeout.date variable to an absolute date. The workflow times out if the user does not respond to the user interaction before this time and date.

Calculate a Relative Timeout for User Interactions

You can calculate in a Date object a relative time and date at which a user interaction times out.

Add a user interaction element to the workflow schema.

You can set an absolute time and date in a Date object. When the time on the given date arrives, the request for a user interaction times out. Alternatively, you can create a workflow element that calculates and generates a relative Date object according to a function that you define. For example, you can create a relative Date object that adds 24 hours to the current time.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the Schema tab.
3. Place a Scriptable task element before the workflow element that requires the relativeDate object for its timeout.date variable.
4. Enter a name and description for the scripted workflow element.
5. Create a Date variable for the Scriptable task element.
   a) Under **Inputs/Outputs**, create a variable.
   b) Name the variable *timerDate*.
   c) Select Date from the list of variable types.
   d) Leave the value text box empty, because a scripted function provides this value.
   e) Click **Save**.
6. Define a function to calculate and generate a Date object named timerDate in the scripting pad in the Scripting tab.

   For example, you can create a Date object by implementing the following JavaScript function, in which the timeout period is a relative delay in milliseconds.

   ```
   timerDate = new Date();
   System.log( "Current date : '" + timerDate + "'" );
   timerDate.setTime( timerDate.getTime() + (86400 * 1000) );
   System.log( "Timer will expire at '" + timerDate + "'" );
   ```

   The preceding example JavaScript function defines a Date object that obtains the current date and time by using the getTime method and adds 86,400,000 milliseconds, or 24 hours. The **Scriptable task** element generates this value as its output parameter.
7. To finish editing your workflow, click **Save**.

You created a function that calculates a time and date relative to the current time and date and generates a Date object. A **User interaction** element can receive this Date object as an input parameter to set the timeout period until which it waits for input from the user. When the workflow arrives at the **User Interaction** element, it suspends its run and waits either until the user provides the required information, or for 24 hours before it times out.

You must bind the Date object to the **User interaction** element's timeout.date variable. See Set the timeout.date Variable to a Relative Date.

Set the timeout date Variable to a Relative Date

You can set the timeout.date variable of a **User interaction** element to a relative time and date by binding it to a Date object. You define the object in a scripted function.

- Add a user interaction element to the workflow schema.
- Create a scripted function that calculates a relative time and date and encapsulates it in a Dateobject in the workflow. See Calculate a Relative Timeout for User Interactions.

If you create a relative Date object in a scripted function, you can bind the timeout.date variable of a user interaction to this Date object. For example, if you bind the timeout.date variable to a Date object that adds 24 hours to the current time, the user interaction times out after waiting for 24 hours.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Select your **User interaction** element.
4. Under **Options**, configure timeout.date to use the timerDate variable.
5. To finish editing your workflow, click **Save**.

You set the timeout.date variable to a relative date and time that a scripted function calculates.

Define User Interaction Exception Behavior

If a user does not provide the input parameters within the timeout period, the user interaction returns an exception. You can define the exception behavior in a scripted function.

Add a user interaction element to the workflow schema.

If you do not define the action for the workflow to take if the user interaction times out, the workflow ends in the Failed state. Defining the exception behavior is a good workflow development practice.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the Schema tab.
3. Select your User interaction element.
4. Create an exception handling variable.
   a) Under **Exception handling**, click **Select variable**.
   b) To create an exception handling variable, click **Create New**.
   c) Name the variable `errorCode`.
   d) Under Value, enter an appropriate error message.
   e) Click**Create**.
5. Drag a Scriptable task element over the user interaction element in the workflow schema.A red dashed arrow, which represents the exception link, appears between the two elements. The Scriptable task element binds automatically to the errorCode variable from the user interaction.
6. Define the exception handling script.
   a) Enter an appropriate name for the **Scriptable task** element.

      For example, `Log timeout`.
   b) In the **Scripting** tab of the **Scriptable task** element, write a JavaScript function to handle the exception.

      For example, to record the timeout in the Orchestrator log, write the following function:

      ```
      System.log("No response from user. Timed out.");
      ```
7. Link and bind the scriptable task element that handles exceptions to the element that follows it in the workflow.For example, link and bind the scriptable task element to a Throw exception element to end the workflow with an error.
8. To finish editing your workflow, click **Save**.

You defined the exception behavior if the user interaction times out.

Create the dialog box in which users provide input parameters. See Create the Input Parameters Dialog Box for the User Interaction.

Create the Input Parameters Dialog Box for the User Interaction

Users provide input parameters during a workflow run in an input parameters dialog box, in the same way that they provide input parameters when a workflow first starts.

Add a user interaction element to the workflow schema.

You create the layout of the dialog box in the input form designer of the user interaction element, not in the **Input Form** tab for the whole workflow. The **Input Form** tab of the whole workflow creates the layout of the input parameters dialog box that appears when you start a workflow. The input form designer of the user interaction element creates the layout of the input parameters dialog box that opens when a workflow arrives at a user interaction element during its run.

1. Navigate to **Library > Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Select your **User interaction** element.
4. Under **Form Inputs**, click **Edit Input Form**.
5. Configure the appearance, values, and parameter constraints of the **User interaction** input parameters.
6. To organize your input parameters under separate tabs, click **Add Tab**.
7. To finish editing your workflow, click **Save**.

You created the input parameters dialog box in which users provide input parameters to respond to a user interaction during a workflow run.

Calling Workflows Within Workflows

Workflows can call on other workflows during their run. A workflow can start another workflow either because it requires the result of the other workflow as an input parameter for its own run, or it can start a workflow and let it continue its own run independently. Workflows can also start other workflow at a given time in the future, or start multiple workflows simultaneously.

There are four ways to call other workflows from within a workflow. Each way is represented by a different workflow schema element.

Synchronous Workflows

Managed by the **Workflow element** schema element. A workflow can start another workflow synchronously. The called workflow runs as a part of the calling workflow's run, and runs in the same memory space as the calling workflow. The calling workflow starts another workflow, then waits until the end of the called workflow's run before it starts running the next element in its schema. Usually, you call a workflow synchronously because the calling workflow requires the output of the called workflow as an input parameter for a subsequent schema element. For example, a workflow can call the **Start virtual machine and wait** workflow to start a virtual machine, and then obtain the IP address of this virtual machine to pass to another element or to a user by email.

Asynchronous Workflows

Managed by the **Asynchronous element** schema element. A workflow can start a workflow asynchronously. The calling workflow starts another workflow, but the calling workflow immediately continues running the next element in its schema, without waiting for the result of the called workflow. The called workflows run with input parameters that the calling workflow defines, but the lifecycle of the called workflow is independent from the lifecycle of the calling workflow. Asynchronous workflows allow you to create chains of workflows that pass input parameters from one workflow to the next. For example, a workflow can create various objects during its run. The workflow can then start asynchronous workflows that use these objects as input parameters in their own runs. When the original workflow has started all the required workflows and run its remaining elements, it ends. However, the asynchronous workflows it started continue their runs independently of the workflow that started them.

To make the calling workflow, wait for the result of the called workflow, either use a nested workflow or create a scriptable task that retrieves the state of the workflow token of the called workflow and then retrieves the result of the workflow when it completes.

Scheduled Workflows

Managed by the **Schedule workflow** schema element. A workflow can call a workflow but defer starting that workflow until a later time and date. The calling workflow then continues its run until it ends. Calling a scheduled workflow creates a task to start that workflow at the given time and date. When the calling workflow has run, you can view the scheduled workflow under **Activity Scheduled**.

Scheduled workflows only run once. You can schedule a workflow to run recurrently by calling the Workflow.scheduleRecurrently method in a scriptable task element in a synchronous workflow.

Nested Workflows

Managed by the **Nested workflows** schema element. A workflow can start several workflows simultaneously by nesting several workflows in a single schema element. All the workflows listed in the nested workflow element start simultaneously when the calling workflow arrives at the nested workflows element in its schema. Each nested workflow starts in a different memory space from the memory space of the calling workflow. The calling workflow waits until all the nested workflows have completed their runs before it starts running the next element in its schema. As a result, the calling workflow can use the results of the nested workflows as input parameters when it runs its remaining elements.

Call a VCF Operations Orchestrator Workflow Synchronously

Calling a workflow synchronously runs the called workflow as a part of the run of the calling workflow. The calling workflow can use the called workflow's output parameters as input parameters when it runs its subsequent schema elements.

You call workflows synchronously from another workflow by using the **Workflow element** schema element.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Drag a **Workflow element** schema element from the **Generic** menu to the appropriate position in the workflow schema.
4. Under **Workflow**, select the workflow you want to call.
5. If prompted, promote workflow input parameters or variables for use by the workflow.
6. Under **Inputs**, bind the required input parameters to the workflow.
7. Under **Outputs**, bind the required output parameters to the workflow.
8. Under **Exception handling**, define the exception behavior of the workflow.
9. To finish editing your workflow, click **Save**.

You called a workflow synchronously from another workflow. When the workflow reaches the synchronous workflow during its run, the synchronous workflow starts, and the initial workflow waits for it to complete before continuing its run.

Call a VCF Operations Orchestrator Workflow Asynchronously

Calling a workflow asynchronously runs the called workflow independently of the calling workflow. The calling workflow continues its run without waiting for the called workflow to complete.

You call workflows asynchronously from another workflow by using the **Asynchronous Workflow** element.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Drag an **Asynchronous element** schema element from the **Generic** menu to the appropriate position in the workflow schema.
4. Under **Workflow**, select the workflow you want to call.
5. If prompted, promote workflow input parameters or variables for use by the workflow.
6. Under **Inputs**, bind the required input parameters to the workflow.
7. Under **Options Output Options**, bind the required output parameters to the workflow.
8. Under **Exception handling**, define the exception behavior of the workflow.
9. To finish editing your workflow, click **Save**.

You called a workflow asynchronously from another workflow. When the workflow reaches the asynchronous workflow during its run, the asynchronous workflow starts, and the initial workflow continues its run without waiting for the asynchronous workflow to finish.

Schedule a VCF Operations Orchestrator Workflow

You can call a workflow from a workflow and schedule it to start at a later time and date.

You schedule workflows in another workflow by using the **Schedule workflow** element.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Drag a **Schedule workflow** element from the **Generic** menu to the appropriate position in the workflow schema.
4. Under **Workflow**, select the workflow you want to call.
5. If prompted, promote workflow input parameters or variables for use by the workflow.
6. Under **Inputs**, bind the required input parameters to the workflow.
7. Under **Options Output Options**, bind the required output parameters to the workflow.
8. Under **Exception handling**, define the exception behavior of the workflow.
9. To finish editing your workflow, click **Save**.

You scheduled a workflow to start at a given time and date from another workflow.

Call Several Workflows Simultaneously

Calling several workflows simultaneously runs the called workflows synchronously as part of the run of the calling workflow. The calling workflow waits for all the called workflows to complete before it continues. The calling workflow can use the results of the called workflows as input parameters when it runs its subsequent schema elements.

You call several workflows simultaneously from another workflow by using the **Nested workflows** element. You can use nested workflows to run workflows with user credentials that are different from the credentials of the user of the calling workflow.

1. Navigate to **Library** > **Workflows** and select your workflow.
2. Select the **Schema** tab.
3. Drag a **Nested workflows** element from the **Generic** menu to the appropriate position in the workflow schema.
4. Click **Add Workflow**, and add a workflow to the **Nested workflows** element.
5. Repeat for all the workflows you require.
6. To finish editing your workflow, click **Save**.

You called several workflows simultaneously from a workflow.

Prerequisites for calling a remote workflow from within another workflow

If the workflow that you develop calls another workflow that resides on a remote Orchestrator server, certain prerequisites must be fulfilled so that the remote workflow can run successfully.

- All input parameters of the remote workflow must be resolvable on the remote Orchestrator server.
- All output parameters of the remote workflow must be resolvable on the local Orchestrator server.

To ensure that the parameters of the remote workflow are resolvable, the inventory objects that the workflow uses must be available both in the remote and the local Orchestrator servers. In case the remote workflow uses objects from a plug-in, the same plug-in must be available on both Orchestrator servers. The inventories of the remote plug-in and the local plug-in must be identical. In case the remote workflow uses library objects in Orchestrator, like workflows and actions, the same workflows and actions must exist in the inventories of the remote and the local Orchestrator servers.

For example, suppose that you insert the **Rename virtual machine** workflow in a **Nested workflow** element in the test workflow that you develop. You want to run the **Rename virtual machine** workflow in a remote Orchestrator server. When you run the test workflow, the **Rename virtual machine** workflow is called within the run of the test workflow. You specify a virtual machine to rename from the inventory of the local Orchestrator server. Because the **Rename virtual machine** workflow runs on the remote Orchestrator server, the same virtual machine must be available in the inventory of that server. Otherwise, the**Rename virtual machine**workflow cannot resolve its vm input parameter. Therefore, the vCenter Server plug-in on the local and the remote Orchestrator servers must be connected to the same vCenter Server instance.

Developing Long-Running Workflows

A workflow in a waiting state consumes system resources because it constantly polls the object from which it requires a response. If you know that a workflow might wait for a long time before it receives the response it requires, you can add long-running workflow elements to the workflow.

Every running workflow consumes a system thread. When a workflow reaches a long-running workflow element, the long-running workflow element sets the workflow into a passive state. The long-running workflow element then passes the workflow information to a single thread that polls the system for all long-running workflow elements running in the server. Rather than each long-running workflow element constantly attempting to retrieve information from the system, long-running workflow elements remain passive for a set duration, while the long-running workflow thread polls the system on its behalf.

You set the duration of the waiting period in one of the following ways.

- Set a timer, encapsulated in a Date object, that suspends the workflow until a certain time and date. You implement long-running workflow elements that are based on a timer by including a **Waiting timer** element in the schema.
- Define a trigger event, encapsulated in a Trigger object, that restarts the workflow after the trigger event occurs. You implement long-running workflow elements that are based on a trigger by including a **Waiting event** element or a **User interaction** element in the schema.

Implement a Timer-Based Long-Running Workflow

To implement a timer-based long-running workflow, first you set a timer that suspends the workflow until a specific time and date, and then you add a **Waiting timer** element to a workflow.

Set a Relative Time and Date for Timer-Baser Workflows

You can set the timer.date variable of a **Waiting timer** element to a relative time and date by binding it to a Date object. You define the Date object in a scripted function.

When the time on the given date arrives, the timer-based long-running workflow reactivates and continues its run. For example, you can set the workflow to reactivate at midday on February 12. Alternatively, you can create a workflow element that calculates and generates a relative Date object according to a function that you define. For example, you can create a relative Date object that adds 24 hours to the current time.

1. Drag a **Scriptable task** element from the **Generic** menu in the left pane to the workflow schema, and place it before the element that requires the relative Date object for its timeout.date variable.
2. Click the **Scriptable task** element.
3. Enter a name and description for the scripted workflow element in the **Details**properties tab in the right pane.
4. On the **Inputs/Outputs** properties tab, create a workflow variable.
    1. Name the variable timerDate.
    2. Select Date from the list of variable types.
    3. Leave the variable value text box blank. A scripted function provides this value.
    4. Click**Create**.
5. Click the**Scripting**tab for the scripted workflow element.
6. Define a function to calculate and generate a Date object named timerDate in the scripting pad in the **Scripting** tab.
   For example, you can create a Date object by implementing the following JavaScript function, in which the timeout period is a relative delay in milliseconds.

   ```
   timerDate = new Date();
   System.log( "Current date : '" + timerDate + "'" );
   timerDate.setTime( timerDate.getTime() + (86400 * 1000) );
   System.log( "Timer will expire at '" + timerDate + "'" );
   ```

   The preceding example JavaScript function defines a Date object that obtains the current date and time by using the getTime method and adds 86,400,000 miliseconds, or 24 hours. The Scriptable task element generates this value as its output parameter.
7. Click **Save** and confirm your selection.

You created a function that calculates and generates a Date object. A **Waiting timer** element can receive this Date object as an input parameter, to suspend a long-running workflow until the date encapsulated in this object. When the workflow arrives at the **Waiting timer** element, it suspends its run and waits for 24 hours before continuing.

Next, you must add a **Waiting timer** element to a workflow to implement a timer-based long-running workflow.

Create a Timer-Based Long-Running Workflow

If you know that a workflow has to wait for a response from an outside source for a predictable time, you can implement it as a timer-based long-running workflow. A timer-based long-running workflow waits until a given time and date before resuming.

You implement a workflow as a timer-based long-running workflow by using the **Waiting timer** element.

1. Drag a **Waiting timer** element from the **Generic** menu in the left pane to the workflow schema, and position it where you want to suspend the workflow run.
   If you implement a scriptable task to calculate the time and date, the scriptable task element must precede the **Waiting timer** element.
2. Click the **Waiting timer** element.
3. Enter a description of the reason for implementing the timer in the **Details** properties tab in the right pane.
4. Click the **Inputs** tab.

The timer.date parameter appears in the list of variables.

5. Bind the timer.date parameter to an appropriate Date object.
    1. Select a predefined Date object from the proposed list, for example one defined by a **Scriptable task** element elsewhere in the workflow.
    2. Alternatively, create a Date object that sets a specific date and time for the workflow to await.
6. Create a Date object that sets a specific date and time that the workflow awaits.
    1. Click the **Select variable** text box, and then click **Create New**.
       The **New Variable** dialog box appears.
    2. Enter a name and description for the variable.
    3. Click the **Value** text box to set the variable value.
       A calendar appears.
    4. Use the calendar to set a date and time at which to restart the workflow.
    5. Click **Create**.
7. Click **Save** and confirm your selection.

You defined a timer that suspends a timer-based long-running workflow until a set time and date.

Next, you can create a long-running workflow that waits for a trigger event before continuing.

Implement a Trigger-Based Long-Running Workflow

To implement a trigger-based long-running workflow, first you define a trigger event that restarts the workflow, and then you bind the trigger event to a **Waiting event** element or a **User interaction** element in a workflow.

Create a trigger object

Trigger objects monitor event triggers that plug-ins define. For example, the vCenter plug-in defines these events as Task objects. When the task ends, the trigger sends a message to a waiting trigger-based long-running workflow element, to restart the workflow.

- • Create a workflow.
  • Open the workflow for editing in the workflow editor.
  • Add some elements to the workflow schema.
- In the workflow, declare a VC:Task object as a variable or input parameter, such as a VC:Task object from a workflow or workflow element that starts or clones a virtual machine.

The time-consuming event for which a trigger-based long-running workflow waits must return a VC:Task object. For example, the startVM action to start a virtual machine returns a VC:Task object, so that subsequent elements in a workflow can monitor its progress. A trigger-based long-running workflow trigger event requires this VC:Task object as an input parameter.You create a Trigger object in a JavaScript function in a **Scriptable task** element. The **Scriptable task** element can be part of the trigger-based long-running workflow that waits for the trigger event. Alternatively, it can be part of a different workflow that provides input parameters to the trigger-based long-running workflow. The trigger function must implement the createEndOfTaskTrigger() method from the Orchestrator API.

> **Important:**
>
> You must define a timeout period for all triggers, otherwise the workflow can wait indefinitely.

1. Drag a **Scriptable task** element from the **Generic** menu in the left pane to the workflow schema.One of the elements that precedes the **Scriptable task** must generate a VC:Task object as its output parameter.
2. Click the **Scriptable task** element.
3. Enter a name and description for the trigger in the **Details** properties tab in the right pane.
4. Click the **Inputs/Outputs** tab.
5. Select or create an input variable of the type **VC:Task**.
   This VC:Task object represents the time-consuming event that another workflow or element starts.
6. Select or create an input parameter of the **Number** type to define a timeout period in seconds.

7. Create an output parameter with the following properties.
    1. Create the Name property with the value trigger.
    2. Create the Type property with the value Trigger.
    3. Click **Create**.
8. Define any exception behavior in the **Exception handling** menu.
9. On the **Scripting** tab, define a function to generate a Trigger object.
   For example, you can create a Trigger object by implementing the following JavaScript function.
   ```
   trigger = task.createEndOfTaskTrigger(timeout);
   ```
   The createEndOfTaskTrigger() method returns a Trigger object that monitors a VC:Task object named task.
10. Click Save and confirm your selection.

You defined a workflow element that creates a trigger event for a trigger-based long-running workflow. The trigger element generates a Trigger object as its output parameter, to which a **Waiting Event** element can bind.

Next, you must bind this trigger event to a **Waiting Event** element in a trigger-based long-running workflow.

Create a Trigger-Based Long-Running Workflow

If you know that a workflow has to wait for a response from an outside source during its run, but do not know how long that wait is, you can implement it as a trigger-based long-running workflow. A trigger-based long-running workflow waits for a defined trigger event to occur before resuming.

You implement a workflow as a trigger-based long-running workflow by using the **Waiting event** element. When the trigger-based long-running workflow arrives at the **Waiting event** element, it stops its run and waits in a passive state until it receives a message from the trigger. During the waiting period, the passive workflow does not consume a thread, but rather the long-running workflow element passes the workflow information to the single thread that monitors all long-running workflows in the server.

1. Drag a **Waiting event** element from the **Generic** menu in the left pane to the workflow schema, and place it where you want to suspend the workflow run.
   The scriptable task that declares the trigger must immediately precede the **Waiting event** element.
2. Click the **Waiting event** element.
3. Enter a name and description of the waiting event in the **Details** properties tab in the right pane.
4. Click the **Inputs** tab.
   The trigger.ref parameter appears in the list of inputs.
5. Click the **Select variable** text box to bind the input parameter to an appropriate Trigger object.
   The Trigger
6. object represents a trigger event that another workflow or workflow element defines.
7. Define any exception behavior in the **Exception handling** tab.
8. Click **Save** and confirm your selection.

You defined a workflow element that suspends a trigger-based long-running workflow, that waits for a specific trigger event before restarting.

You can run a workflow.

Find Object References in Workflows

As a workflow developer, you can use object reference information to optimize your development life cycle.

Develop a workflow that includes at least one object reference.

With VCF Operations orchestrator, you can find object reference information. This feature has two functions:

- **Find Dependencies**: find information about object dependencies in your workflows. Dependencies can include other workflows, actions, resource elements, and configuration elements.

Note:
Found dependencies can also include commented out code. For example, a workflow can include a line of code referencing an action that is commented out. The addition of commented out code in the list of found dependencies is expected behavior.

- **Find Usages**: learn if the selected workflow is used in other workflows in the VCF Operations orchestrator library.

You can access information about object references from the workflow editor or from the VCF Operations orchestrator library in either Card View, List View, or Tree View.

The following procedure demonstrates how you can access object references from the workflow editor.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Workflows**, and select your workflow.
3. To find information about object dependencies, click **Find Dependencies**.

   Note:
   While on the dependencies pop-up window, you can select referenced objects from the list. Selecting an object opens a separate VCF Operations orchestrator tab where you can view the details of the selected object or edit it.
4. To find information about where the selected workflow is used, click **Find Usages**.

Managing Resource Elements

Workflows can use objects that you create independently of VCF Operations orchestrator as attributes. To use external objects as attributes in workflows, you import them into the server as resource elements.

Objects that VCF Operations orchestrator workflows can use as resource elements include image files, scripts, XML templates, HTML files, and so on. Any workflows that run in the VCF Operations orchestrator server can use any resource elements that you import into VCF Operations orchestrator.

After you import an object into VCF Operations orchestrator as a resource element, you can make changes to the object in a single location, and propagate those changes automatically to all the workflows that use this resource element.

The maximum size for a resource element is 16 MB.

You can import, export, restore, update, and delete a resource element.

Managing Configuration Elements

A configuration element is a list of variables, that you can use to configure constants across a whole VCF Operations orchestrator server deployment.

You can use configuration elements to make variables available to all the workflows, actions, and policies running on the VCF Operations orchestrator server.

If you create a package containing a workflow, an action, or a policy that uses a variable from a configuration element, VCF Operations orchestrator automatically includes the configuration element in the package.

If you import a package containing a configuration element into another VCF Operations orchestrator server, you can import the configuration element variable values as well. For example, if you create a workflow that requires variable values that depend on the VCF Operations orchestrator server on which it runs, setting those variables in a configuration element lets you export that workflow, so that another VCF Operations orchestrator server can use it. Configuration elements therefore allow you to exchange workflows, actions, and policies between servers more easily.

Create a configuration element

With configuration elements, you can set common variables across an VCF Operations orchestrator server. All elements that are running in the server can use the variables you set in a configuration element.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Assets** > **Configurations**.
3. Select **New Configuration**.
4. Enter the configuration element name.
5. Select the **Variables** tab.
6. To create a local variable, click **New**.
   a) Enter the variable name.
   b) Select the variable type.

      Note:
      To create an array of configuration variables, select the **Array** check box.
   c) Optional: Enter a value for the configuration variable.
   d) Click **Save**.
7. To finish creating a configuration element, click **Save**.

You can use the configuration element to provide variables to workflows, actions, or policies.

VCF Operations orchestrator Input Form Designer

If a workflow requires input parameters, it opens a dialog box in which users enter the required values. You can organize the content, layout, and presentation of this dialog box with the input form designer.

The input form designer is located in the **Input Form** tab of the workflow editor. This designer consists of a navigation menu, design canvas, and properties menu. You can drag inputs and generic elements from the left menu to the design canvas. In the canvas, you can set the position of the input parameters, organize them into separate input tabs, and configure the input parameter properties.

Note:
You cannot use content from the **Variables** tab of the workflow editor in the input form designer. You can only use parameters from the **Input/Output** tab.

Generic elements	You can add generic elements, like drop-down menus and password text boxes, to the input form designer. Generic elements do not correspond to actual input parameters, but can be bound to input parameters.

Create the Workflow Input Parameters Dialog Box in the VCF Operations orchestrator

You can use the input form designer to create and customize the workflow input parameter dialog box.

Verify that the workflow has a defined list of input parameters.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Workflows**.
3. Select your custom workflow.
4. Click the **Input Form** tab.
5. Optional: Create tabs for use in the input dialog box.
   You can use tabs to organize the structure of your dialog box.
6. Select your input parameters.
7. Edit the properties of the input parameters.
   For more information on input parameter properties, see Input Parameter Properties in VCF Operations orchestrator Workflows.

8. Optional: Add generic elements to the canvas and bind them to input parameters.
9. Optional: Add external validation to the input parameters. For more information, see Validate VCF Operations orchestrator workflow inputs using actions.
10. Click **Save**.

You created the layout of the workflow dialog box and set the properties of the input parameters.

Input Parameter Properties in VCF Operations orchestrator Workflows

You can set parameter properties to constrain the input parameters that users provide when they run VCF Operations orchestrator workflows.

With VCF Operations orchestrator, you can define the parameter properties used to quantify the input parameter values used in workflows. The parameter properties you define impose limits on the types and values of the input parameters that users can provide in VCF Operations orchestrator workflows.

Parameter properties validate the input parameters and modify the presentation of the text boxes that appear in the input parameters dialog box. Some parameter properties can create dependencies between parameters.

Parameter Property	Description
Label	Set the input parameter label.
Display type	Set the input text box display type.
Visibility	Set the visibility of the input parameter.
Read-only	Set the input text box as read-only.
Custom help	Set the input parameter signpost description.
Default value	Set the default value of the input parameter.
Step	Used for number type inputs. Set by how much the value of the input parameter increases per click.
Required	Sets if the input parameter value is mandatory or not.
Regular expression	Validates the input by using a regular expression.
Minimum value	Set the minimum value or length of the parameter.
Maximum value	Set the maximum value or length of the parameter.
Match text box	Set the input parameter value to match the value of another input parameter.
Value source	Set the value source of the parameter properties in the **Appearance**, **Value**, and **Constraints** tabs. **Note:** You can import the value of external actions by using **External source**. The filtering of available actions is done by parameter type.

Validate VCF Operations orchestrator workflow inputs using actions

Use external actions to validate the inputs of your custom workflows.

Create a custom workflow with input parameters. For more information, see Create a Workflow in VCF Operations Orchestrator.

You can use the input form designer to create external validations for your workflow inputs. External validations use action scripts that return a string value when the input parameter value contains an error. If the input parameter value is valid, the external validation returns nothing.

1. In VCF Automation, open the **Orchestrate** tab.
2. Create a validation action.
   a) Navigate to **Library** > **Actions**.
   b) Click **New Action**.
   c) Enter the required information on the **General** tab.
   d) Enter the validation action input parameters.

      Note:
      The names of the validation action input parameters must be identical to the names of the workflow input parameters that are being validated.
   e) Enter the script of the validation action on the **Script** tab.

```
if (in_1=="invalid") {
            return "in_1 can't be invalid!";
}
        if (in_2=="invalid") {
            return "in_2 can't be invalid!";
}

//inputs are valid, return nothing
```

      Note:
      The preceding script is a simple example and does not represent the full scope of the validation scripts that can be used.
   f) Click **Create**.
3. Apply external validation.
   a) Navigate to **Library** > **Workflows**.
   b) Select your custom workflow.
   c) Select the **Input Form** tab.
   d) On the **Validations** tab, drag and drop the **Orchestrator validation** element into the canvas.
   e) In the canvas, select the validation element, enter a validation label, and select the validation action.
   f) Optional: Create additional validation elements.
   g) Click **Save**.
4. Run the workflow.

   If the validation encounters an error, it returns a string. If the validation is successful, the validation returns nothing and the workflow run continues.

You created an external validation for your custom VCF Operations orchestrator workflow.

Testing and Debugging VCF Operations Orchestrator Workflows

You can validate, debug, and monitor VCF Operations orchestrator workflows before and after you run them in production. Use the procedures in this section to identify structural errors, troubleshoot runtime issues with breakpoints and external IDEs, and analyze workflow performance with metrics and profiling.

Validating Workflows

VCF Operations orchestrator provides a workflow validation tool. Validating a workflow helps identify errors in the workflow and checks that the data flows from one element to the next correctly.

By default, VCF Operations orchestrator always performs a workflow validation when you run a workflow.

When you validate a workflow, the validation tool creates a list of any errors or warnings. Clicking an error in the list highlights the workflow element that contains the error.

If you run the validation tool in the workflow editor, the tool provides suggested quick fixes for the errors it detects. Some quick fixes require additional information or input parameters. Other quick fixes resolve the error for you.

Workflow validation checks the data bindings and connections between elements. Workflow validation does not check the data processing that each element in the workflow performs. As a result, a valid workflow might run incorrectly and produce erroneous results if a function in a schema element is incorrect.

Validate a Workflow and Fix Validation Errors in VCF Operations orchestrator

You must validate a workflow before you can run it. You can only fix validation errors if you have opened the workflow for editing.

Verify that you have a complete workflow to validate, with schema elements linked and bindings defined.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Workflows** and select the workflow you want to validate.
3. Click **Edit**.
4. Click **Validate** from the top menu.
   If the workflow is valid, a confirmation message appears. If the workflow is invalid, a list of errors appears.
5. For an invalid workflow, click an error message and take appropriate steps to resolve the problem.
   The validation tool highlights the schema element in which the error occurs by adding a red icon to it. Where possible, the validation tool displays a quick fix action.
   - If you agree with the proposed quick fix action, click it to perform that action.
   - If you disagree with the proposed quick fix action, close the Workflow Validation dialog box and fix the schema element manually.

     Important:
     Always check that the fix that VCF Operations orchestrator proposes is appropriate.

   For example, the proposed action might be to delete an unused attribute, when in fact that attribute might not be bound correctly.
6. Repeat the preceding steps until you have eliminated all validation errors.

You validated a workflow and fixed the validation errors.

You can run the workflow.

Debug Workflow Scripts in VCF Operations orchestrator

You can debug workflow runs by inserting breakpoints in the script of workflow items.

When a breakpoint is reached, you have several options to continue the debugging process. When you debug an element from the workflow schema, you can view the general information about the workflow run, modify the workflow variables, add expressions to watch, and view log messages.

Note:
Perform all script debugging in a non-production environment.

1. In VCF Automation, open the **Orchestrate** tab.
2. Navigate to **Library** > **Workflows** and select your workflow.
3. Open the workflow schema, select a workflow element, and click the **Scripting** tab.
4. To insert a breakpoint, click the red circle to the left of the line number.

   Note:
   You can only insert breakpoints in workflow elements with scripting.
5. To run the workflow in the debugging mode, click **Debug**.
   If the workflow requires input parameters, you must provide them.

6. When the workflow run is paused after reaching a breakpoint, select one of the available options.

Option	Description
Continue	Resumes the workflow run until another breakpoint is reached or the workflow run finishes.
Step into	You can use this option to step into a workflow element. You cannot step into a nested workflow element when you debug a workflow in the workflow editor.
Step over	Skips the current element in the schema and pauses the workflow run on the next element.

   Note:
   You can instruct the debugger to ignore the current breakpoint by clicking the breakpoint. This changes the breakpoint symbol to a green triangle.

7. Optional: On the **Debugger** tab, insert expressions to watch.
   You can use expressions to follow the completion of specific variables.
8. Optional: On the **Debugger** tab, modify the values of variables.

Debug Workflows by Schema Element

As a workflow designer, you can debug individual schema elements.

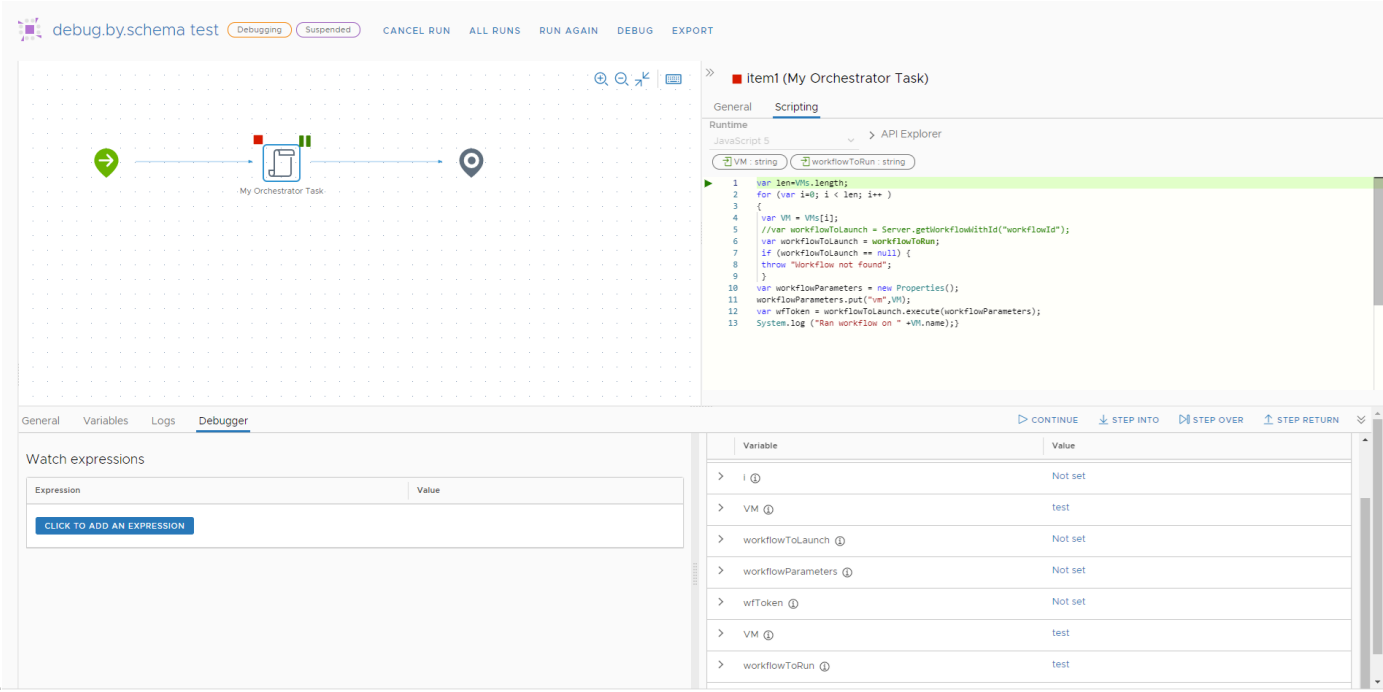
1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Workflows** and select your workflow.
3. Select the **Schema** tab.
4. Select the workflow element you want to debug, and click the debug button on the top-left of the element.

   Note:
   By adding a breakpoint to a **Workflow Element** schema element, you can debug child workflows directly from the parent workflow. When the debugger reaches the **Workflow Element** schema element, it opens the schema view of the child workflow.

5. Repeat for any other schema elements you want to debug.
6. Click **Debug**.
7. Enter the requested input parameter values, and click **Run**.
   The workflow run begins, and is suspended when the debugger reaches a schema element with a breakpoint.
8. When at a breakpoint, select one of the following options:

Option	Description
Continue	Resumes the workflow run until another breakpoint is reached or the workflow run finishes.
Step into	Step into the current workflow function. If the debugger cannot go deeper into the current line of the function, it performs a **Step over** operation.
Step over	The debugger continues into the next line of the current function.
Step return	The debugger goes into the line that will perform when the current function returns.

9. Optional: On the **Variables** tab, edit the value of your workflow variables.



Running VCF Operations orchestrator Workflows

A VCF Operations orchestrator workflow runs according to a logical flow of events.

When you run a workflow, each schema element in the workflow runs according to the following sequence.

1. The workflow binds the workflow token variables and input parameters to the input parameters of the schema element.
2. The schema element runs.
3. The output parameters of the schema element are copied to the workflow token variables and workflow output parameters.
4. The workflow token variables and output parameters are stored in the database.
5. The next schema element starts running.

This sequence repeats for each schema element until the end of the workflow.

Workflow Token Check Points

When a workflow runs, each schema element is a check point. After each schema element runs, VCF Operations orchestrator stores workflow token variables in the database, and the next schema element starts running. If the workflow stops unexpectedly, the next time the Orchestrator server restarts, the currently active schema element runs again, and the workflow continues from the start of the schema element that was running when the interruption occurred. However, VCF Operations orchestrator does not implement transaction management or a rollback function.

End of Workflow

The workflow ends if the current active schema element is an end element. After the workflow reaches an end element, other workflows or applications can use the output parameters of the workflow.

Running a workflow on a selection of objects

You can automate repetitive tasks by running a workflow on a selection of objects. For example, you can create a workflow that takes a snapshot of all the virtual machines in a virtual machine folder, or you can create a workflow that powers off all the virtual machines on a given host.

You can use one of several methods to run a workflow on a selection of objects. Which method you choose depends on the specific workflow and can affect the performance of that workflow.

- Run the **Run a workflow on a selection of objects** workflow.
  To access the workflow, navigate to **Library > Workflows** in Orchestrator and enter the name of the workflow in the workflow search bar.
  Running this workflow is the simplest method and requires no workflow development, but it can only run workflows that take a single input parameter.
- Create a workflow that calls the **Start workflows in a series** or **Start workflows in parallel** workflows.
  This method allows you to run workflows that take more than one input parameter. The calling workflow must create a properties array to pass the input parameters to the **Start workflows in a series** or **Start workflows in parallel** workflow. These two workflows are only meant for use in other workflows. Do not run them directly.
  For instructions on implementing this method, go to Implement the Start Workflows in a Series and Start Workflows in Parallel Workflows.
- Create a workflow that obtains an array of objects and runs a workflow on each object in the array in a loop of workflow elements.
- Run a workflow from JavaScript by calling the Workflow.execute() method in a For loop in a scripted element in a workflow.
  This method is faster than running a workflow in a loop of workflow elements, but it is less flexible and limits the potential for reuse. Most importantly, running a workflow in a scripted loop loses the checkpointing that Orchestrator performs when it starts each element in a workflow run. As a consequence, if the Orchestrator server stops while the scripted loop is running, when the server restarts, the workflow resumes at the beginning of the scripted element, repeating the whole loop. If the Orchestrator server stops while running a workflow with a loop of workflow elements, the workflow resumes at the specific element in the loop that was running when the server stopped.

Implement the Start Workflows in a Series and Start Workflows in Parallel Workflows

You can use the **Start workflows in a series** and **Start workflows in parallel** workflows to run a workflow on a selection of objects.

Open a workflow for editing in the workflow editor.

You cannot run the **Start workflows in a series** and **Start workflows in parallel** workflows directly. You must include them in another workflow that you create. To use the **Start workflows in a series** and **Start workflows in parallel** workflows to run a workflow on a selection of objects, you must obtain the objects on which to run the workflow. You pass these objects and any other input parameters that the workflow requires to the workflow as an array of properties. The **Start workflows in a series** and **Start workflows in parallel** workflows emit the results of running the workflow on the selection of objects as an array of WorkflowToken objects.

You implement the **Start workflows in a series** and **Start workflows in parallel** workflows in the same way. The **Start workflows in a series** workflow runs the workflow on each object sequentially. The **Start workflows in parallel** workflow runs the workflow on all the objects simultaneously.

1. In the workflow schema, add a **Scriptable task** or an **Action element** to obtain a list of objects on which to run the workflow.

   For example, to run a workflow on all the virtual machines in a virtual machine folder, you can add the `getAllVirtualMachinesByFolder` action to the workflow.

2. Link the scripted element or action and bind the input and output of the scripted element or action to workflow inputs or variables.

   For example, you can bind the vmFolder input of the getAllVirtualMachinesByFolder action to a workflow input parameter and the actionResult output to a workflow variable in the calling workflow.

3. Add a **Scriptable task** element to cast the list of objects into a properties array.

   For example, if the objects on which to run the workflow are an array of virtual machines, allVMs, returned by the actionResult output of the getAllVirtualMachinesByFolder action, you can write the following script to cast the objects into a properties array.

```
propsArray = new Array();

for each (var vm in allVMs) {
 var prop = new Properties();
 prop.put("vm", vm);
 propsArray.push(prop);
}
```

4. Bind the inputs and outputs of the scriptable task to workflow variables.

   In the example scriptable task element in step 3, you bind the input to the allVMs array of virtual machines and you create the propsArray output variable as an array of Properties objects.

5. Add a **Workflow element** to the workflow schema.
6. Select either of the**Start workflows in a series**or**Start workflows in parallel**workflows and link the workflow element to the other elements.
7. To run on the objects, bind the wf input of the **Start workflows in a series** or **Start workflows in parallel** workflow to the workflow.

   For example, to remove any snapshots of all the virtual machines returned by the getAllVirtualMachinesByFolder action, select the **Remove all snapshots** workflow.

8. Bind the parameters input of the **Start workflows in a series** or **Start workflows in parallel** workflow to the array of Properties objects that contains the objects on which to run the workflow.

   For example, bind the parameters input to the propsArray variable defined in step 4.

9. Bind the workflowTokensoutput of the **Start workflows in a series** or **Start workflows in parallel** workflow to a variable in the workflow.
10. Continue adding more elements that use the results of running the **Start workflows in a series** or **Start workflows in parallel** workflow.

You created a workflow that uses either of the **Start workflows in a series** or **Start workflows in parallel** workflows to run a workflow on a selection of objects.

Resuming a Failed Workflow Run

If a workflow fails, VCF Operations orchestrator provides an option to resume the workflow run from the last failed activity.

You can change the parameters of the workflow and attempt to resume it, or retain the parameters and make changes to external components that affect the workflow run. For example, if a workflow run fails due to a problem in a third-party system, you can make changes to the system and resume the workflow run from the failed activity, without changing the workflow parameters.

Set the Behavior for Resuming a Failed Workflow Run

You can set the behavior for resuming a failed run for each custom workflow. The default workflows in the library use the default system setting for resuming a failed workflow run.

You can change the default system behavior creating a system property. See Set custom properties for resuming failed workflow runs.

1. Navigate to **Library** > **Workflows** and select your workflow.

2. On the **Summary** tab, select an option from the **Resume workflow from failed behavior** drop-down menu.

Option	Description
System default	Follows the default behavior.
Enabled	If a workflow run fails, a pop-up window displays an option to resume the workflow run.
Disabled	If a workflow run fails, it cannot be resumed.

3. Click Save and confirm your selection.

Set custom properties for resuming failed workflow runs

By default, VCF Operations orchestrator is not set up to resume failed workflow runs. You can enable VCF Operations orchestrator to resume failed workflow runs and set a custom timeout period after which failed workflow runs cannot be resumed.

1. Log in to VCF Operations orchestrator as `root`.
2. Select **System Properties**, and click **New**.
3. Under **Key**, enter `com.vmware.vco.engine.execute.resume-from-failed.timeout-sec`.
4. Under **Value**, enter the custom timeout period in seconds.

   Note:

   The value you set overrides the default timeout setting of 86400 seconds.
5. Under **Description,** enter a description for the new system property.
6. Click **Add**.

Managing a Failed Workflow Run

You can resume a workflow run from the last failed activity if resuming a failed run is enabled for the workflow.

When the resume failed workflow feature is enabled, you can change the parameters of the workflow and try to resume it. You can also retain the parameters and change external components that affect the workflow run. If you do not select an option, the workflow run times out and cannot be resumed. For modifying the timeout period, see Set custom properties for resuming failed workflow runs.

In this use case, you create a simple workflow designed specifically to fail during its run. This workflow demonstrates the basic functionality of the resume failed workflow feature.

1. Navigate to **Library** > **Workflows** and click **New Workflow**.
2. Enter the name resume workflow test and click **Create**.
3. On the **Summary** tab, click the **Resume workflow from failed behavior** drop-down menu.
4. Select **Enabled**.
5. On the **Inputs/Outputs** tab, click **New**.
6. Create an input parameter with the following properties.

Name	fail
Type	boolean

7. On the **Schema** tab, drag three **Scriptable task** elements into the schema diagram.
8. Name the schema elements before failure,failure, and after failure.
9. On the **Scripting** tab of the before failure element, enterSystem.log("Before failure");.
10. Add the fail input parameter to the failure element.
11. On the **Scripting** tab of the after failure element, enterSystem.log("After failure");.
12. Save the workflow, and click **Run**.
13. Toggle on the **fail** check-box, and click **Run**.
    The workflow run reaches the **Waiting** state.
14. On the top-right, select **Answer**.

15. To cancel the workflow run, select **Cancel** and click **Answer**.
16. To resume the workflow run, select **Resume**, and navigate to the **Parameters** tab.
17. Toggle off the **fail** check-box, and click **Answer**.
    The workflow run reaches the **Completed** state.

You now know how to manage failed workflow runs.

Using Workflow Token Replay in VCF Operations orchestrator

You can use the token replay feature to view the transitions between items in workflow runs.

The token replay feature records contextual information for each transition between workflow items. For each workflow item, token replay records when the workflow run started, ended, and what variables were changed at the end of the workflow item run. Token replay also references the generated script log messages for each workflow item.

> **Note:**
> Data about workflow item transitions is stored in the VCF Operations orchestrator PostgreSQL database. This data is removed from the database when the workflow run is deleted.

1. Optional: In VCF Automation, open the Orchestrate tab. Enable token replay for all workflow runs on your VCF Operations orchestrator server.
   a) In VCF Automation, open the Orchestrate tab.
   b) Select **System Settings** > **Performance and Features**.
   c) Enable the Run Replay Status toggle.
   d) Click **Save**.

   > **Note:**
   > It can take up to 5 minutes for the VCF Operations orchestrator server to refresh the extension.
2. Navigate to **Activity** > **Workflow Runs**.
3. Select a workflow run.
4. Select a workflow run item from the left menu.
   The **Variable** and **Logs** tabs now display information specific for that workflow item.

Managing Policies in VCF Operations Orchestrator

Policies are specialized objects that are used to monitor events and automatically trigger workflows or scripts in response.

Policies are a series of rules, gauges, thresholds, and event filters that run certain workflows or scripts when specific predefined events occur in VCF Operations orchestrator or in the technologies that VCF Operations orchestrator accesses through plug-ins. VCF Operations orchestrator constantly evaluates the policy rules while the policy is running. For instance, you can implement policy gauges and thresholds that monitor the behavior of vCenter objects of the `VC:HostSystem` and `VCirtualMachine` types.

Create and apply VMware Cloud Foundation Operations orchestrator policies

You can use policies to monitor the activity of the VMware Cloud Foundation Operations orchestrator system for specific events.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Policies**.
3. Select **New Policy**.
4. Enter a policy name and version number.
5. On the **Variables** tab, click **New**.
   a) Enter the variable name.
   b) Select the variable type.
      To create an array of policy variables, select the **Array** check box.
   c) Enter the variable value.
      To import the value of a configuration element variable, you can use **Bind to configuration**.
   d) Click **Save**.
6. On the **Definition** tab, add policy elements and set event handlers.
   For more information on policy elements, see Policy Elements in VCF Operations orchestrator.
7. Click **Save**.

To start a policy, select the policy and click **Run**. Enter the policy run name and, if prompted, the required input parameters.

To view the policy status, navigate to **Activity** > **Policy Runs**.

Policy Elements in VCF Operations orchestrator

You can use policy elements to run predefined VCF Operations orchestrator workflows or scripts when an event occurs.

You can add a policy element to trigger workflow or script runs as a response to events triggered by objects. With the periodic event element, you can schedule workflow or script runs. With the root element, you can set the start or stop behavior of policies. Policy elements can have event handlers that define when policy elements must run.

> **Note:**
> Event handlers that activate policy elements can be either workflows or action scripts. If you add both a workflow and a script to an event handler, the policy ignores the script trigger and only uses the workflow trigger.

Event Handler	Description
OnInit	The policy element is triggered every time you start the policy.
OnExit	The policy element is triggered every time you stop the policy.
OnExecute	Used by the periodic event element. Triggers the policy element during the time specified in the periodic event element.

> **Note:**
> Technologies plugged in to the VCF Operations orchestrator database can possess unique event handlers. For example, through the SNMP plug-in, you can use the **OnTrap** event handler when creating SNMP-based policy elements.

Policy elements are configured on the **Definition** tab of the policy edit window.

Manage Policy Runs in VCF Operations orchestrator

You can use VCF Operations orchestrator to manage the policy priority and server start-up behavior of policies for when the VCF Operations orchestrator server is restarted.

Create and run a policy. For more information, see Create and apply VMware Cloud Foundation Operations orchestrator policies.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Activity** > **Policy Runs**.
3. Click the policy run you want to manage.
4. Click **Stop**.
   The policy state changes to **Stopped**.
5. On the **General** tab, set the policy priority and server start-up behavior.
6. To restart the policy, click **Run**.
   The policy state changes to **Running**.

Running and Debugging Actions

You can improve your actions by running and debugging them directly from the action editor.

You can run and debug actions directly from the action editor of VCF Operations orchestrator. With this feature, you can guarantee that your actions perform as expected when they are integrated into your workflows.

Run Actions in VCF Operations orchestrator

As a workflow designer, you want to run your actions before integrating them into a workflow.

Create an action. See Create Actions in VCF Operations Orchestrator.

1. In VCF Automation, open the **Orchestrate** tab.
2. Navigate to **Library** > **Actions** and select the action you want to run.
3. Click **Run**.

   Note:

   The above procedure is based on the list view of the UI. If you are using the card view, navigate to the action you want to run, open the **Actions** drop-down, and select **Run**.
4. Enter the required input parameters, and click **Run**.
   After the action run finishes, click the **Results/Inputs** tab. If the action run encountered an error, it is displayed on this tab in a red color. You can view the details of the action run from the **Action Results** element.

   Note:
   The results of the action run are not saved.

Debug Actions in the VCF Operations orchestrator

As a workflow designer, you can debug actions by inserting breakpoints into your script.

Create an action. See Create Actions in VCF Operations Orchestrator.

VCF Operations orchestrator includes a built-in debugging tool that you can use to debug the script and input properties of your action. The debug process can be initiated in the action editor by inserting breakpoints into the script lines of your action.

Note:
The built-in debugging tool only works with actions that use the default JavaScript runtime.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Library** > **Actions** and select the action you want to debug.

   Note:

   The above procedure is based on the list view of the UI. If you are using the card view, navigate to the action you want to run, open the **Actions** drop-down, and select **Run**.

3. In the action editor, add breakpoints to the lines of your action script you want to debug.
4. Click **Debug**.
5. Enter the input parameters of your action, and click **Run**.
   An action run in debug mode begins.
6. When the action run is paused after reaching a breakpoint, select one of the following options:

Option	Description
Continue	Resumes the action run until another breakpoint is reached or the action run finishes.
Step into	Step into the current action function. If the debugger cannot go deeper into the current line of the function, it performs a **Step over** operation.
Step over	The debugger continues into the next line of the current function.
Step return	The debugger goes into the line that will perform when the current function returns.

7. Optional: On the **Debugger** tab, add expressions.
8. Optional: On the **Debugger** tab, edit the value of your variables.

Debugging VCF Operations Orchestrator Runtime Scripts with External IDE

VCF Operations orchestrator supports debugging Python and Node.js scripts in an external IDE.

VCF Operations orchestrator supports the following debugging scripts in external IDEs.

Runtime	Status
Python	Supported
Node.js	Supported
PowerShell	Not supported
PowerCLI	Not supported

Metric Data in VCF Operations orchestrator

As an administrator, you can use workflow profiling and the System Dashboard metrics to troubleshoot the VCF Operations orchestrator system and workflows.

The profiling feature gathers metric data about workflow runs. Workflow profiling is enabled by default. To deactivate automatic profiling, navigate to **System Settings** > **Performance and Features** and deselect the Workflow Profiling Status toggle.

The other source for metric data in VCF Operations orchestrator is the System Dashboard which provides system-level metrics. For more information, see Using the VCF Operations orchestrator System Dashboard.

Profile Workflows in VCF Operations orchestrator

You can profile your workflow runs to troubleshoot and optimize your VCF Operations orchestrator environment.

Run a workflow.

You can use the profiling feature of VCF Operations orchestrator to gather useful metric data about your workflow runs. This data can be used to optimize the performance of your workflows. By default, workflow runs are profiled automatically. To do a manual profiling run, find your workflow in the library and select **Actions** > **Profile**.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Activity** > **Workflow Runs**.
3. Select a workflow run.

   On the workflow run schema, you can see data about the individual workflow items. Data includes total run duration, max duration, and number of item runs. You can filter this information from the drop-down menu on the top right of the page.
4. Select the **Performance** tab.

   This tab provides you with metric data on workflow run CPU times, run duration, token size, and workflow item data.

   Note:
   If the workflow run is suspended, for example, when the workflow is waiting for further input, the CPU times metric only captures the runtime thread that occurred before completion.

Use the data gathered from profiling to optimize your workflow.

Using the VCF Operations orchestrator System Dashboard

As an administrator, you can use the VCF Operations orchestrator System Dashboard to gather useful metric data about the nodes of your VCF Operations orchestrator environment.

You can access the System Dashboard from by clicking the **System** tab on the VCF Operations orchestrator dashboard page. Provided data includes:

- Node status
- Node properties
- Cluster settings
- Threads info
- Heap memory
- Non-heap memory
- File system use
- Authentication data
- Orchestrator database connection pool
- Process input arguments

This data can be used to monitor the state of individual nodes of your VCF Operations orchestrator environment and troubleshoot problems. To navigate between individual nodes, click the tab associated with a node on the top of the System Dashboard.

Publishing and Scheduling VCF Operations Orchestrator Workflows

Use this section to package, schedule, and track changes to VCF Operations Orchestrator workflows and related content objects. After building and testing a workflow, you bundle it for distribution across environments, automate its scheduled execution, track its change history, and synchronize it with a Git repository for collaborative development.

Audience

Workflow developers and administrators who are ready to deploy completed workflows to production or distribute them across VCF Operations Orchestrator environments.

Quick navigation

I want to...	Go to...
Bundle workflows for deployment to another environment	
Move packages between VCF Operations Orchestrator instances	
Run a workflow automatically at a set time or interval	

I want to...	Go to...
Set up a recurring workflow run	
Review change history for a workflow or action	
Roll back a workflow to a previous version	
Synchronize content with a Git repository	
Reset content to a previous Git commit	

Before You Begin

Complete the following prerequisites before starting any task in this section.

- You have built and tested the workflow or action you want to publish. See Test and debug workflows.
- You have the **Workflow Designer** or **Administrator** role in VCF Operations Orchestrator. Users with the **Executor** role cannot create or modify packages.
- If you are distributing packages to a remote VCF Operations Orchestrator server, verify that the target server is running and accessible, and that you have administrator credentials for it.
- If you are configuring Git integration, verify that you have a GitHub or GitLab repository created and that you have a personal access token or SSH key with write access to the target repository.

Managing Packages

Use VCF Operations orchestrator to create, export, and import packages. Packages can be used to export workflow objects for use on other VCF Operations orchestrator instances.

Packages can contain workflows, actions, policies, configuration elements, or resources elements.

When you add an element to a package, VCF Operations orchestrator checks for dependencies and adds any dependent elements to the package. For example, if you add a workflow that uses actions or other workflows, VCF Operations orchestrator adds those actions and workflows to the package.

> **Note:**
> VCF Operations orchestrator dependencies added to a package can also include commented out code. For example, a workflow can include a line of code referencing an action that is commented out. In such a scenario, the action is added to the package regardless. The addition of commented out code in the package is expected behavior.

When you import a package, the server compares the versions of the different elements of its contents to matching local elements. The comparison shows the differences in versions between the local and imported elements. The user can decide whether to import the package, or can select specific elements to import.

For most objects created in VCF Operations orchestrator, aside from resource elements, packages are the only way to export and import these objects.

Packages use digital rights management to control how the receiving server can use the contents of the package. VCF Operations orchestrator signs packages and encrypts the packages for data protection. Packages can track which users export and redistribute elements by using X509 certificates.

Create a Package in VCF Operations orchestrator

You can export and import workflows, policies, actions, plug-in references, resource elements, and configuration elements in packages. All dependent elements related to package objects are added to the package automatically, to ensure compatibility between versions. To delete dependent elements, you must first remove the related package object.

Verify that the VCF Operations orchestrator server contains objects like workflows, actions, and policies, that you can add to a package.

For most objects created in VCF Operations orchestrator, aside from resource elements, packages are the only way to export and import these objects.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Assets** > **Packages**.
3. Click **New Package**.
4. On the **General** tab, enter a name and description for the package.

   Note:
   You cannot use special characters when naming packages in VCF Operations orchestrator.
5. On the **Content** tab, click **Add**.
6. Select the objects that you want to add to the package and click **Add**.

   Note:
   Dependent elements are added to the package automatically, but are not displayed in the **Content** tab during package creation. To view dependent elements, select the **Content** tab after package creation.
7. To finish creating the package, click **Create**.

Export a Package in VCF Operations orchestrator

You can use VCF Operations orchestrator to export packages to another VCF Operations orchestrator environment.

Create a package containing the VCF Operations orchestrator objects you want to export. For more information, see Create a Package in VCF Operations orchestrator.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Assets** > **Packages**.
3. Click **Export** on the package.
4. Optional: Select additional export options.

Option	Description
Add configuration attribute values to package	Export the attribute values of the configuration elements.
Add configuration SecureString attribute values to package	Export the `SecureString` configuration attribute values.
Add global tags to package	Export the global tags.

5. Set the access rights for users who import the package.

Option	Description
View contents	The user can view the package content.
Add to package	The user can add content from the imported package to other packages.
Edit contents	The user can edit the package content.

6. Click **Ok**.

   Note:
   Files with the `.package` extension are saved to a default folder on your local machine. To set a custom folder, you can change the storage settings in your browser.

You exported the package. You can now use the exported objects on another VCF Operations orchestrator environment.

Import a Package in VCF Operations orchestrator

Use VCF Operations orchestrator to import workflow packages. By importing packages, you can reuse objects from one VCF Operations orchestrator server on another server.

- Back up any standard VCF Operations orchestrator objects that you have modified.
- On the remote server, create and export a package with the objects you want to import.

1. In VCF Automation, open the Orchestrate tab.
2. Navigate to **Assets** > **Packages**.
3. Click **Import**, browse to the `.package` file that you want to import, and click **Open**.
4. Review the imported package information.
   a) The **General** tab contains information about the imported package like the name, description, number of contained items, and certificate information.

      You might be prompted to indicate that you trust the publisher certificate of the source VCF Operations orchestrator instance before you can import the file.
   b) The **Package elements** tab lists the objects included in the import file. If the version of an object in the package is later than the version on the server, the system selects that object version for import. Earlier versions of VCF Operations orchestrator elements must be selected manually.
   c) Deselect **Import Configuration Attribute Values** if you do not want to import the attribute values of the configuration elements from the package.
   d) From the drop-down menu, select whether you want to import tags.
5. Click **Import**.

Schedule Workflows in VCF Operations orchestrator

You can use scheduling to automate your VCF Operations orchestrator workflow runs.

When you schedule workflow runs, you set the date, time, and intervals at which the scheduled task runs.

1. In VCF Automation, open the **Orchestrate** tab.
2. Select your workflow from the **Library** menu, and on the workflow panel, click **Schedule**.
3. Configure the scheduled task parameters in the **General**, **Scheduling**, and **Workflow** categories.

   Note:
   The **Workflow** parameter category is visible only for workflows that require input parameters.

Parameter	Description
Name	The name of the scheduled task.
Description	A short description detailing the purpose of the scheduled task.
Starting user	The user credentials used to authenticate the scheduled workflow run. The value of the parameter can be either the current user, Orchestrator system user, or API token.
Start	The date and time of the first scheduled run of the workflow.
Start if in the past	Select whether to start the workflow, if the scheduled time is in the past. **Yes** starts the scheduled workflow immediately. **No** starts the workflow at the next scheduled recurrence.
Schedule	Set the recurrence pattern and event trigger entries of the scheduled task.

4. Click **Save**.

You have created a scheduled task for the workflow. Scheduled workflows appear under **Activity** > **Scheduled**. You can delete scheduled tasks by clicking **Delete** on the schedule panel.

Edit Scheduled Task in VCF Operations orchestrator

Scheduled tasks can be edited to change parameters such as the starting user, date, time, and recurrence of the scheduled workflow.

Create a scheduled workflow task.

1. In VCF Automation, open the Orchestrate tab.
2. Select your scheduled task from **Activity** > **Scheduled**.
3. Click **Open** on the workflow panel.
4. Edit the scheduled task parameters.

   Note:
   Input parameters set when creating the scheduled task are read-only and cannot be edited. To change these parameters, create a new scheduled task for this workflow.

Parameter	Description
Starting user	Change the authentication of the scheduled task. Click **Use Current User** to set the scheduled task authentication to the current user. Editing this parameter can be useful for use cases where the authentication provider for VCF Operations orchestrator has changed and the previous user credentials used when creating the scheduled task are no longer valid. The new user authentication is set after changes to the scheduled task are saved.
Name	The name of the scheduled task.
Description	A short description detailing the purpose of the scheduled task.
Start	The date and time of the first scheduled run of the workflow.
Start if in the past	Select whether to start the workflow, if the scheduled time is in the past. **Yes** starts the scheduled workflow immediately. **No** starts the workflow at the next scheduled recurrence.
Schedule	Set the recurrence pattern and event trigger entries of the scheduled task.

5. To finish editing the scheduled task, click **Save**.

VCF Operations orchestrator applies the updated parameters to the scheduled task. The workflow runs on the new schedule from the next execution point.

Managing VCF Operations orchestrator Version History

VCF Operations orchestrator maintains a version history record for VCF Operations orchestrator content. Using the version history, you can compare different VCF Operations orchestrator content versions and revert to a previous version.

VCF Operations orchestrator creates a version history record for each VCF Operations orchestrator object when you save the object. Subsequent changes to VCF Operations orchestrator objects create a new version history record. The previous versions history records are preserved and can be used to track changes to the object and revert the object to a previous version. Reverting an object to a previous version creates a new version history record.

VCF Operations orchestrator tracks the version history of the following VCF Operations orchestrator objects:

- Workflows
- Actions
- Packages
- Policies
- Resource elements
- Configuration elements

Note:
Generated workflows do not appear in the workflow version history. For example, the workflows generated by the **Generate CRUD workflows for a table** workflow do not appear on the **Version History** tab and cannot be

pushed to any configured Git repositories. To include these workflows in VCF Operations orchestrator version history, duplicate the generated workflows.

You can access the version history of an object from the **Version History** tab of the object editor page. If you are attempting to edit an object at the same time as another user, a merge conflict might occur. To resolve the merge conflict, click **Resolve** to the right of the error message. On the **Resolve Conflicts** window you have three options:

- **Use theirs**. Resolve the merge conflict by using the changes made by the other user.
- **Use ours**. Resolve the merge conflict by using your changes.
- **Resolve**. Resolve the merge conflict by editing the displayed change model. If the provided model is invalid, this option is unavailable.

Prerequisites

- Create a workflow.
- Configure a connection to a GitHub or GitLab repository. See Git Branching for VCF Operations orchestrator Content Management.
- Push a local change set to the configured Git repository.

Visual comparison between workflow versions

1. In VCF Automation, open the **Orchestrate** tab.
2. Navigate to **Library** > **Workflows** and select one of your workflows.
3. Edit the content of the workflow. For example, you can add an extra **Scriptable task** element on the **Schema** tab.
4. Click **Save**.
5. Select the **Version History** tab.
6. From the top-right, select **Diff Visually**.

You can now perform a visual comparison between two selected workflow versions. You can select which versions to compare from the **Version** drop-down menu. You can restore a workflow to another version by selecting **Use Version**.

Restore a workflow to an earlier version

1. In VCF Automation, open the **Orchestrate** tab.
2. Navigate to **Library** > **Workflows** and select a workflow.
3. Select the **Version History** tab.
4. To view a comparison between the versions, select a workflow version and select another version from the **Diff against** drop-down menu.
5. To restore the workflow to another version, click **Restore**.

The workflow state is reverted to the state of the selected version.

Reset your VCF Operations orchestrator content inventory to a previous state with Git

You can reset your VCF Operations orchestrator content to a previous state by selecting a specific commit.

1. In VCF Automation, open the **Orchestrate** tab.
2. Navigate to **Administration** > **Git History**.
3. Select a change set you want to reset to and click **Reset to this**.
4. Confirm that you want to reset to this specific commit and click **Ok**.

The VCF Operations orchestrator content inventory is reset to the state specified in the commit. Relevant VCF Operations orchestrator content is reverted to a previous version. If the content did not exist when the commit was pushed, it is removed from the inventory.

To restore the VCF Operations orchestrator inventory to the latest state saved to the Git repository, perform a `Pull` command from the **Git History** window.

Using the VCF Operations orchestrator Plug-ins

With the VCF Operations orchestrator plug-ins, you can access and control external technologies and applications, such as virtualization management tools, email systems, databases, directory services, and remote control interfaces. Exposing an external technology in a VCF Operations orchestrator plug-in lets you incorporate objects and functions in workflows and run workflows on the objects of that external technology.

VCF Operations orchestrator provides a standard set of preinstalled plug-ins, including plug-ins for vCenter and VCF Automation, to allow you to orchestrate tasks in the different environments that the plug-ins expose. In addition, the open plug-in architecture lets you develop plug-ins to access other applications. VCF Operations orchestrator implements open standards to simplify integration with external systems.

Plug-ins extend the VCF Operations orchestrator scripting engine with new object types and methods and publish notification events from the external system that triggers events in VCF Operations orchestrator and in the plugged-in technology. Plug-ins provide an inventory of JavaScript objects that you can access on the Inventory page of VCF Operations orchestrator. Each plug-in contains packages of workflows and actions that you can run to automate the typical use cases of the integrated product.

VCF Operations orchestrator Architecture

The standard set of plug-ins is automatically installed with the VCF Operations orchestrator server. You might need to configure some of the plug-ins, for example, the vCenter plug-in, before you start using them.

VCF Operations orchestrator connects to an authentication provider to manage user accounts and to a preconfigured PostgreSQL database to store information from the workflows that it runs. You can access VCF Operations orchestrator, the objects it exposes, and the default workflows through the VCF Operations orchestrator, or through web services. You use the VCF Operations orchestrator to monitor and configure workflows and services.

Access the VCF Operations orchestrator API Explorer

You use the VCF Operations orchestrator API Explorer as an in-product reference guide to JavaScript objects exposed by VCF Operations orchestrator and all installed plug-ins. To access the API explorer, navigate to the **API Explorer** page on the Orchestrate page in VCF Automation.

Managing VCF Operations orchestrator plug-ins

On the **System Settings** page of VCF Operations orchestrator, you can view a list of all plug-ins that are installed in VCF Operations orchestrator and perform basic management actions.

Change the plug-In logging level

Instead of changing the logging level for VCF Operations orchestrator, you can change it only for specific plug-ins.

Activate or deactivate a plug-in

You can activate or deactivate a plug-in by selecting the plug-in and clicking **Enable/Disable**.

This action does not remove the plug-in file.

Configuring the VCF Operations orchestrator plug-ins

The default VCF Operations orchestrator plug-ins come with configuration workflows. You can run these workflows to register endpoints for management.

The configuration workflows have the *`configuration`* tag. For example, to access workflows that are used to manage AMQP brokers and subscriptions, enter the tags *`AMQP`* and *`Configuration`* in the search text box of the workflow library.

Install, update, or delete a plug-in

You can install, update, or delete third-party plug-ins on the **System Settings** page in VCF Operations orchestrator.

Install a VCF Operations Orchestrator Plug-In

You can install third-party plug-ins in VCF Operations orchestrator.

Download the plug-in file to your local machine. The preferred file format for VCF Operations orchestrator plug-ins is `.vmoapp`, but `.dar` files are also supported.

Note:

Do not rename the `.dar` file after you download it.

To install the plug-in, you use the VCF Operations orchestrator UI.

1. In VCF Operations orchestrator, select the **System Settings**.
2. On the **Plug-ins** card, click **Open**.
3. Click **Add Plug-in**.
4. In the **General** pane, click **Select** and add the plug-in file.
5. Select a log level.
6. Click **Next**.
7. Review the plug-in information, if applicable, accept the end-user license agreement, and click **Install**.
8. Click **Upload**.
   The plug-in is installed or updated and the VCF Operations orchestrator server service is restarted.

Verify that the correct plug-in information is listed on the **Plug-ins** page.

Delete a plug-in

You can delete third-party plug-ins from VCF Operations orchestrator.

1. In VCF Operations orchestrator, select the **System Settings**.
2. On the **Plug-ins** card, click **Open**.
3. Find the plug-in you want to delete and click **Delete**.
4. Confirm that you want to delete the plug-in, and click **Delete**.

Plug-ins installed with the VCF Operations orchestrator server

VCF Operations orchestrator includes a collection of standard plug-ins. Each plug-in expoes an external product API to the VCF Operations orchestrator platform. Plug-ins provide inventory classes, additional object types for the scripting engine, and publish notification events from the external system. Each plug-in also provides a library of workflows for automating the typical use cases of the integrated external products.

You can see the list of installed plug-ins on the **Manage Plug-Ins** page in VCF Operations orchestrator.

Table 1845: Plug-Ins Installed with VCF Operations orchestrator

Plug-In	Purpose	Status	Configuration
Active Directory	Provides interaction between VCF Operations orchestrator and Microsoft Active Directory.		See Configuring and the Auto Deploy Plug-In.
AMQP	Interact with Advanced Message Queuing Protocol (AMQP) servers, also known as brokers.		See Configuring the AMQP Plug-In.
Auto Deploy	Perform different operations associated with vCenter server ESXi hosts.		See Configuring and the Auto Deploy Plug-In.

Plug-In	Purpose	Status	Configuration
Configuration	Provides workflows for configuring and managing the VCF Operations orchestrator server keystores and trusted certificates.		None
Dynamic Types	Define dynamic types and create and use objects of these dynamic types.		See Using the Dynamic Types Plug-In.
Enumeration	Provides common Enumerated Types that can be used in workflows by other plug-ins.		See Time Zone Codes
HTTP-REST	Manage REST Web services through an interaction between VCF Operations orchestrator and REST hosts.		See Configuring the HTTP-REST Plug-In.
Library	Provides workflows that act as basic building blocks for customization and automation of client processes. The workflow library includes templates for life-cycle management, provisioning, disaster recovery, hot backup, and other standard system management processes. You can copy and edit the templates to modify them according to your needs.		None
Mail	Uses Simple Mail Transfer Protocol (SMTP) to send email from workflows.		Set the default values for the `EmailMessage` object to use. See Define the default SMTP connection.
Multi-Node	Contains workflows for hierarchical management, management of VCF Operations orchestrator instances, and scale-out of VCF Operations orchestrator activities.		See Using the Multi-Node plug-in.
Net	Uses the Jakarta Apache Commons Net Library. Provides implementations of the Telnet, FTP, POP3, and IMAP protocols. The POP3 and IMAP protocols is used for reading email. With the Mail plug-in, the Net plug-in provides complete email sending and receiving capabilities in workflows.		
PowerShell	Manage PowerShell hosts and run custom PowerShell operations.		See Using the PowerShell plug-in.
SNMP	Connect and receive information from SNMP-enabled systems and devices.		
SOAP	Manage SOAP Web services by providing interaction between VCF Operations orchestrator and SOAP hosts.		See Configuring the plug-in.

Plug-In	Purpose	Status	Configuration
SQL	Provides the Java Database Connectivity (JDBC) API, which is the industry standard for database-independent connectivity between the Java programming language and a wide range of databases. The databases include SQL databases and other tabular data sources, such as spreadsheets or flat files. The JDBC API provides a call-level API for SQL-based database access from workflows.		
SSH	Provides an implementation of the Secure Shell v2 (SSH-2) protocol. Allows remote command and file transfer sessions with password and public key-based authentication in workflows. Supports keyboard-interactive authentication. Optionally, the SSH plug-in can provide remote file system browsing directly in the VCF Operations orchestratorClient inventory.		See Add an SSH Host to VCF Operations Orchestrator.
vCenter	Provides access to the vCenter API so that you can incorporate all the vCenter objects and functions into the management processes that you automate by using VCF Operations orchestrator.		See Configuring the vCenter Server Plug-In in VCF Operations Orchestrator.
vCloud Suite API (vAPI)	Provides access to the API services exposed by any vAPI provider.		
VCF Automation	Integrates VCF Operations orchestrator with VCF Automation organizations.		
VCF Automation for VM Apps	Integrates VCF Operations orchestrator with VCF Automation for VM Apps organizations.		See Configuring the vCenter Server Plug-In.
XML	A complete Document Object Model (DOM) XML parser that you can implement in workflows. Alternatively, you can use the ECMAScript for XML (E4X) implementation in the VCF Operations orchestrator JavaScript API.		

Plug-In Components

The components of each plug-in, such as workflow categories and API modules, use different naming conventions.

Table 1846: Names of Plug-In Components

Plug-In Name in the Configuration UI	Workflow Categories	API Module
Active Directory	Computer Configuration Organizational Unit User User Group	`AD`
AMQP	Configuration	`AMQP`
Auto Deploy	Answer Files Configuration Reprovision Host Rules Rule Set Compliance	`AutoDeploy`
Configuration	Configuration	`Configurator`
Dynamic Types	Configuration	`DynamicTypes`
Common enumerated types	None	`Enums`
HTTP-REST	Configuration	`REST`
Library	Locking Orchestrator Tagging	Not applicable.
Mail	Mail	`Mail`
Multi-Node	Servers Configuration Remote Execution Remote Management Tasks Workflows	`VCO`
Net	None	`Net`
PowerShell	Configuration Generate Templates	`PowerShell`
SNMP	Device Management Query Management Trap Host Management	`SNMP`
SOAP	Configuration	`SOAP`
SQL	JDBC SQL	`SQL`
SSH	SSH	`SSH`
Support	None	`Support`
vAPI	VAPI	`VAPI`
vCenter	vCenter	`VC`
VCF Automation	Configuration Infrastructure	`VCFA`
VCF Automation for VM Apps	Configuration Infrastructure Sample REST Operations	`VRA`
XML	XML	`XML`

Time Zone Codes

When implementing common enumerated types in workflows, you can use time zone codes as possible values for the `Enums:MSTimeZone` enumeration.

Time Zone Code	Time Zone Name	Description
000	Dateline Standard Time	(GMT-12:00) International Date Line West
001	Samoa Standard Time	(GMT-11:00) Midway Island, Samoa
002	Hawaiian Standard Time	(GMT-10:00) Hawaii
003	Alaskan Standard Time	(GMT-09:00) Alaska
004	Pacific Standard Time	(GMT-08:00) Pacific Time (US and Canada); Tijuana
010	Mountain Standard Time	(GMT-07:00) Mountain Time (US and Canada)
013	Mexico Standard Time 2	(GMT-07:00) Chihuahua, La Paz, Mazatlan
015	U.S. Mountain Standard Time	(GMT-07:00) Arizona
020	Central Standard Time	(GMT-06:00) Central Time (US and Canada)
025	Canada Central Standard Time	(GMT-06:00) Saskatchewan
030	Mexico Standard Time	(GMT-06:00) Guadalajara, Mexico City, Monterrey
033	Central America Standard Time	(GMT-06:00) Central America
035	Eastern Standard Time	(GMT-05:00) Eastern Time (US and Canada)
040	U.S. Eastern Standard Time	(GMT-05:00) Indiana (East)
045	S.A. Pacific Standard Time	(GMT-05:00) Bogota, Lima, Quito
050	Atlantic Standard Time	(GMT-04:00) Atlantic Time (Canada)
055	S.A. Western Standard Time	(GMT-04:00) Caracas, La Paz
056	Pacific S.A. Standard Time	(GMT-04:00) Santiago
060	Newfoundland and Labrador Standard Time	(GMT-03:30) Newfoundland and Labrador
065	E. South America Standard Time	(GMT-03:00) Brasilia
070	S.A. Eastern Standard Time	(GMT-03:00) Buenos Aires, Georgetown
073	Greenland Standard Time	(GMT-03:00) Greenland
075	Mid-Atlantic Standard Time	(GMT-02:00) Mid-Atlantic
080	Azores Standard Time	(GMT-01:00) Azores
083	Cape Verde Standard Time	(GMT-01:00) Cape Verde Islands
085	GMT Standard Time	(GMT) Greenwich Mean Time: Dublin, Edinburgh, Lisbon, London
090	Greenwich Standard Time	(GMT) Casablanca, Monrovia
095	Central Europe Standard Time	(GMT+01:00) Belgrade, Bratislava, Budapest, Ljubljana, Prague
100	Central European Standard Time	(GMT+01:00) Sarajevo, Skopje, Warsaw, Zagreb
105	Romance Standard Time	(GMT+01:00) Brussels, Copenhagen, Madrid, Paris
110	W. Europe Standard Time	(GMT+01:00) Amsterdam, Berlin, Bern, Rome, Stockholm, Vienna
113	W. Central Africa Standard Time	(GMT+01:00) West Central Africa
115	E. Europe Standard Time	(GMT+02:00) Bucharest
120	Egypt Standard Time	(GMT+02:00) Cairo
125	FLE Standard Time	(GMT+02:00) Helsinki, Kyiv, Riga, Sofia, Tallinn, Vilnius
130	GTB Standard Time	(GMT+02:00) Athens, Istanbul, Minsk
135	Israel Standard Time	(GMT+02:00) Jerusalem
140	South Africa Standard Time	(GMT+02:00) Harare, Pretoria
145	Russian Standard Time	(GMT+03:00) Moscow, St. Petersburg, Volgograd
150	Arab Standard Time	(GMT+03:00) Kuwait, Riyadh
155	E. Africa Standard Time	(GMT+03:00) Nairobi
158	Arabic Standard Time	(GMT+03:00) Baghdad
160	Iran Standard Time	(GMT+03:30) Tehran
165	Arabian Standard Time	(GMT+04:00) Abu Dhabi, Muscat
170	Caucasus Standard Time	(GMT+04:00) Baku, Tbilisi, Yerevan

Time Zone Code	Time Zone Name	Description
175	Transitional Islamic State of Afghanistan Standard Time	(GMT+04:30) Kabul
180	Ekaterinburg Standard Time	(GMT+05:00) Ekaterinburg
185	West Asia Standard Time	(GMT+05:00) Islamabad, Karachi, Tashkent
190	India Standard Time	(GMT+05:30) Chennai, Kolkata, Mumbai, New Delhi
193	Nepal Standard Time	(GMT+05:45) Kathmandu
195	Central Asia Standard Time	(GMT+06:00) Astana, Dhaka
200	Sri Lanka Standard Time	(GMT+06:00) Sri Jayawardenepura
201	N. Central Asia Standard Time	(GMT+06:00) Almaty, Novosibirsk
203	Myanmar Standard Time	(GMT+06:30) Yangon (Rangoon)
205	S.E. Asia Standard Time	(GMT+07:00) Bangkok, Hanoi, Jakarta
207	North Asia Standard Time	(GMT+07:00) Krasnoyarsk
210	China Standard Time	(GMT+08:00) Beijing, Chongqing, Hong Kong SAR, Urumqi
215	Singapore Standard Time	(GMT+08:00) Kuala Lumpur, Singapore
220	Taipei Standard Time	(GMT+08:00) Taipei
225	W. Australia Standard Time	(GMT+08:00) Perth
227	North Asia East Standard Time	(GMT+08:00) Irkutsk, Ulaan Bataar
230	Korea Standard Time	(GMT+09:00) Seoul
235	Tokyo Standard Time	(GMT+09:00) Osaka, Sapporo, Tokyo
240	Yakutsk Standard Time	(GMT+09:00) Yakutsk
245	A.U.S. Central Standard Time	(GMT+09:30) Darwin
250	Cen. Australia Standard Time	(GMT+09:30) Adelaide
255	A.U.S. Eastern Standard Time	(GMT+10:00) Canberra, Melbourne, Sydney
260	E. Australia Standard Time	(GMT+10:00) Brisbane
265	Tasmania Standard Time	(GMT+10:00) Hobart
270	Vladivostok Standard Time	(GMT+10:00) Vladivostok
275	West Pacific Standard Time	(GMT+10:00) Guam, Port Moresby
280	Central Pacific Standard Time	(GMT+11:00) Magadan, Solomon Islands, New Caledonia
285	Fiji Islands Standard Time	(GMT+12:00) Fiji Islands, Kamchatka, Marshall Islands
290	New Zealand Standard Time	(GMT+12:00) Auckland, Wellington
300	Tonga Standard Time	(GMT+13:00) Nuku'alofa

Configure Kerberos Authentication for VCF Operations orchestrator Plug-Ins

You can use Kerberos authentication for VCF Operations orchestrator plug-ins.

Configure the `krb5.conf` file in an external VCF Operations orchestrator instance

1. Create or edit the `krb5.conf` file at `/data/vco/usr/lib/vco/app-server/conf/`.
   A `krb5.conf` file has the following structure:

```
[libdefaults]
default_realm = YOURDOMAIN.COM
[realms]
YOURDOMAIN.COM = {
kdc = dc.yourdomain.com
default_domain = yourdomain.com
}
[domain_realm]
.yourdomain.com=YOURDOMAIN.COM
yourdomain.com=YOURDOMAIN.COM
```

The `krb5.conf` must contain specific configuration parameters with their values.

Kerberos configuration tags	Details
default_realm	The default Kerberos realm that a client uses to authenticate against an Active Directory server. Must be in uppercase letters.
kdc	The domain controller that acts as a Key Distribution Center (KDC) and issues Kerberos tickets.
default_domain	The default domain that is used to produce a fully qualified domain name. This tag is used for Kerberos 4 compatibility.

To allow ticket forwarding to other external systems, add the `forwardable = true` flag. For additional information, see the Oracle documentation on the krb5.conf file.

By default, the Java Kerberos configuration uses the UDP protocol. To use only the TCP protocol, you must specify the `udp_preference_limit` parameter with a value `1`.

Note:
The Kerberos authentication requires a Fully Qualified Domain Name (FQDN) host address.

Important:
When you add or modify the `krb5.conf` file, you must restart the VCF Operations orchestrator server service.

If you have a clustered VCF Operations orchestrator environment, make sure that the `krb5.conf` file exists in all three appliances with the same configuration before you restart the VCF Operations orchestrator pods.

2. Change permissions.

```
chmod 644 krb5.conf
```

3. Redeploy the VCF Operations orchestrator pod.

```
kubectl -n prelude get pods
```

Look for an entry similar to `vco-app-<ID>`.

4. Destroy the pod.

```
kubectl -n prelude delete pod vco-app-<ID>
```

A new pod is automatically deployed to replace the pod you destroyed.

Configure the `krb5.conf` file in an embedded VCF Operations orchestrator instance

When configuring the `krb5.conf` file for embedded VCF Operations orchestrator instances there are several differences from the external VCF Operations orchestrator instance that must be considered:

- Configuration properties must be set from the user interface under **System Settings** > **Configuration Properties**. Using the `vracli vro properties set` command is no longer supported.
- Configuration files and third-party drivers cannot be copied from the appliance. They must be copied directly to the `vco-app` container. The following is an example script which can be used to copy your files and drivers:

```
#!/bin/bash

# Script to download a file inside the vco-server-app container of the vco-app pod
# Usage: ./download_to_vco.sh <url> <destination_path>

# Check if correct number of arguments is provided
if [ "$#" -ne 2 ]; then
    echo "Usage: $0 <url> <destination_path>"
    echo "Example: $0 https://example.com/file.zip /tmp/file.zip"
    exit 1
fi

URL="$1"
DEST_PATH="$2"
NAMESPACE="prelude"
```

```
POD_NAME="vco-app"
CONTAINER_NAME="vco-server-app"

echo "Targeting Pod: $POD_NAME"
echo "Namespace: $NAMESPACE"
echo "Container: $CONTAINER_NAME"
echo "Downloading: $URL -> $DEST_PATH"

# Check if the pod exists exactly as named, otherwise try to find a pod starting with the name
if ! kubectl get pod "$POD_NAME" -n "$NAMESPACE" > /dev/null 2>&1; then
    echo "Pod '$POD_NAME' not found directly. Searching for pods starting with '$POD_NAME'..."
    # Try to find a pod that starts with the name (assuming it might be a deployment)
    FOUND_POD=$(kubectl get pods -n "$NAMESPACE" -o jsonpath="{.items[?(@.metadata.name=='$POD_NAME' ||
 @.metadata.generateName=='$POD_NAME-')].metadata.name}" | head -n 1)

    # If using grep/awk is more reliable for simple prefix matching:
    if [ -z "$FOUND_POD" ]; then
        FOUND_POD=$(kubectl get pods -n "$NAMESPACE" --no-headers | awk '{print $1}' | grep "^$POD_NAME" |
 head -n 1)
    fi

    if [ -n "$FOUND_POD" ]; then
        echo "Found pod: $FOUND_POD"
        POD_NAME="$FOUND_POD"
    else
        echo "Error: Could not find pod '$POD_NAME' or any pod starting with '$POD_NAME' in namespace
 '$NAMESPACE'."
        exit 1
    fi
fi

# Execute commands in the container:
# 1. Create destination directory
# 2. Download file (curl -L follows redirects, -f fails on HTTP errors)
# 3. Set permissions to 644
echo "Executing remote download and setup..."
DIR_PATH=$(dirname "$DEST_PATH")

kubectl exec -n "$NAMESPACE" "$POD_NAME" -c "$CONTAINER_NAME" -- /bin/sh -c "mkdir -p \"$DIR_PATH\" && curl
 -f -L -o \"$DEST_PATH\" \"$URL\" && chmod 644 \"$DEST_PATH\""

# Check the exit code of kubectl (which reflects the exit code of the command chain)
if [ $? -eq 0 ]; then
    echo "Success: File downloaded to $DEST_PATH"
else
    echo "Error: Download failed."
    exit 1
fi
```

- The configuration file is located `/usr/lib/vco/app-server/conf/` and the database drivers are located at `/var/run/vco/plugins/SQL/`. All other paths are identical to the external instance, with the exception that `/data/vco` is excluded from the filepath.
- The file-logs are available in `/var/log/services-logs/prelude/vco-app/file-logs/` . The console logs, used for Kerberos troubleshooting are available in support bundles or by using the following command:

  ```
  kubectl -n prelude logs vco-app-0 -c vco-server-app
  ```

Persistent volumes for embedded instance that are not backed up or restored

The following persistent volumes are not backed up or restored in embedded VCF Operations orchestrator instances:

- The `krb5.conf` file
- The `js-io-rights.conf` file
- DB plug-in drivers
- The `com.vmware.scripting.rhino-class-shutter`-file
- Certificates for legacy plugins (not implementing ISSLService)

Enable Kerberos debug logging

You can troubleshoot VCF Operations orchestrator plug-in problems by modifying the Kerberos configuration file used by the plug-in.

The Kerberos configuration file is located in the `/data/vco/usr/lib/vco/app-server/conf/` directory of the VCF Operations orchestrator Appliance.

1. Log in to the VCF Operations orchestrator Appliance command line as **root**.
2. Run the `kubectl -n prelude edit deployment vco-app` command.
3. In the deployment file, locate and edit the `-Djava.security.krb5.conf=/usr/lib/vco/app-server/conf/krb5.conf` string.
   ```
   -Djava.security.krb5.conf=/usr/lib/vco/app-server/conf/krb5.conf -Dsun.security.krb5.debug=true
   ```
4. Save the changes and exit the file editor.
5. Run the `kubectl -n prelude get pods` command. Wait until all pods are running.
6. To monitor the Kerberos login, run the following command.
   ```
   tail -f /services-logs/prelude/vco-app/console-logs/vco-server-app.log
   ```

   Note:

   The preceeding `tail` command is valid only for external VCF Operations orchestrator instances. For embedded VCF Operations orchestrator in VCF Automation, log on to the container to monitor the Kerberos login.
7. Alternatively, you can enable debug logging in the VCF Operations orchestrator configurator by adding the `sun.security.krb5.debug = true` system property.

Using the Active Directory Plug-In

The VCF Operations orchestrator plug-in for Microsoft Active Directory allows interaction between VCF Operations orchestrator and Microsoft Active Directory. You can use the plug-in to run VCF Operations orchestrator workflows that automate Active Directory processes.

The Active Directory plug-in contains a set of standard workflows. You can also create custom workflows that implement the plug-in API to automate tasks in your Active Directory environment.

Configuring the Active Directory Plug-In

To connect to a Microsoft Active Directory instance by using the Active Directory plug-in, you must configure the connection parameters for the Microsoft Active Directory instance by running the configuration workflows included in the plug-in.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `active_directory` and `configuration` tags in the workflow search box.

Workflow Name	Description
Add an Active Directory server	Adds a new Active Directory domain configuration.
Configure Active Directory plug-in options	Configures the search limitation options of the Active Directory plug-in.
Update an Active Directory server	Modifies an existing Active Directory server configuration.

Workflow Name	Description
Remove an Active Directory server	Removes an Active Directory server configuration.
Configure Active Directory server (Deprecated)	Creates or updates the default Active Directory server configuration. Use Update an Active Directory server.
Reset configuration (Deprecated)	Deletes the default Active Directory server configuration. Use Remove an Active Directory server.

Active Directory Plug-in and Object Class Mapping

VCF Operations orchestrator plug-in for Microsoft Active Director uses the following mapping for Active Directory object classes:

Plug-in Type	Object Class
`AD_Computer`	`computer`
`AD_User`	`user`
`AD_UserGroup`	`group`
`AD_Group`	`container`
`AD_OrganizationalUnit`	`organizationalUnit`
`AD_Unknown`	Applies for all objects that do not use the preceding classes.

Note:

If the Active Directory object has more than one of the preceding classes, the first class in the order is preferred. The inventory of the Active Directory plug-in represents the container hierarchy, not the hierarchy of the groups.

Using the Active Directory Plug-In Workflow Library

The Active Directory plug-in workflow library contains workflows that allow you to run automated processes related to the management of Microsoft Active Directory objects.

Using the Active Directory Plug-In Inventory

The Active Directory plug-in exposes all objects in the connected Microsoft Active Directory instance in the **Inventory** view.

To display the workflows that are available for an Active Directory inventory object, navigate to **Administration** > **Inventory** > **Active Directory** in the VCF Operations orchestrator Client.

Active Directory Plug-In Workflows

The Active Directory plug-in contains a set of standard workflows that cover the most common LDAP functionality. You can use the workflows as building blocks for creating complex custom solutions. By combining standard workflows, you can automate multistep processes in the Active Directory environment.

Client-Side Load Balancing for the Active Directory Plug-In

You can use client-side load balancing and failover to improve the stability of your Active Directory plug-in configuration.

You can configure client-side load balancing when running the **Add an Active Directory server** and **Update an Active Directory server** workflows. Client-side load balancing is possible through the ServerSet Java class.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `active_directory` and `configuration` tags in the workflow search box.
3. Run the **Add an Active Directory server** or **Update an Active Directory server** workflow.
4. Select the **Alternative hosts** tab.
5. From the drop-down menu, select **Single Server**, **Round-Robin DNS Server**, **Round-Robin**, or **Failover**.

Option	Description
Single Server	A server set implementation that connects to only one server.
Round-Robin DNS Server	A server set where server handles the case in which a given host name may resolve to multiple IP addresses. This server set does strictly require DNS server setup. The ordering mechanism for selecting an address is round-robin.
Round-Robin	A server set where load is distributed evenly between several directory servers. If a server is unavailable, the connection will move to the next server in the set.
Failover	A server set where server connections are established in order. This implementation can establish connections between separate server sets. Useful for providing high availability in complex environments.

6. When you finish configuring the workflow run, click **Run**.

Using the AMQP Plug-In

The AMQP plug-in allows you to interact with Advanced Message Queuing Protocol (AMQP) servers also known as brokers. You can define AMQP brokers and queue subscriptions as inventory objects by running configuration workflows, and perform AMQP operations on defined objects.

The plug-in contains a set of standard workflows related to managing AMQP brokers and calling AMQP operations.

Configuring the AMQP Plug-In

You can configure AMQP by running the configuration workflows included in the plug-in. The Configuration workflow category contains workflows that allow you to manage AMQP brokers.

To access these workflows in the VCF Operations orchestrator, navigate to **Library** > **Workflows** and enter the `amqp` and `configuration` tags in the workflow search box.

Workflow Name	Description
Add a broker	Adds an AMQP broker.
Remove a broker	Removes an AMQP broker.
Remove a subscription	Removes an AMQP message subscription.
Subscribe to queues	Creates a subscription element.
Update a broker	Updates broker properties.
Validate a broker	Validate a broker by attempting to start a connection.

TLS1.3 support in the AMQP plug-in

TLS1.3 support depends on the AMQP broker.

For RabbitMQ, see the list of supported versions at https://www.rabbitmq.com/ssl.html#tls1.3. If you follow the RabbitMQ guide, you must also add `ssl_options.password = <the_private_key_password>` to the Rabbit MQ configuration.

If you want to use TLS1.3 in the AMQP plug-in, add the relevant system parameter in the VCF Operations orchestrator by running the following command:

```
vracli vro properties set -k o11n.plugin.amqp.ssl-context-protocol -v TLSv1.3
```

Add a Broker

You can run a workflow to add an AMQP broker.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `amqp` and `configuration` tags in the workflow search box.
3. Locate the **Add a broker** workflow and click **Run**.
4. On the **AMQP broker properties** tab, enter the name of the broker.
5. On the **AMQP connection properties** tab, provide the information required for the broker connection.

Option	Action
Host	Enter the address of the host.
Port	Enter the port of the AMQP broker service. The default port is 5672.
Virtual host	Enter the address of the virtual host. The default value provided is /.
Use SSL	Select whether to use SSL certificates.
Accept all certificates	Select whether to accept all SSL certificates without validation.
User name	Enter the user name for the broker.
Password	Enter the password for the broker.

6. Click **Run**.

After the workflow runs successfully, the AMQP broker appears in the **Inventory** view.

You can run a Validate a broker workflow. If an error occurs, use the Update a broker workflow to change the properties of the broker before validating again.

Subscribe to Queues

You can run a workflow to create a new subscription element.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.
- Verify that the AMQP broker has all queues included in the subscription declared.

1. Navigate to **Library** > **Workflows** and enter the `amqp` and `configuration` tags in the workflow search box.
2. Locate the **Subscribe to queues** workflow and click **Run**.
3. On the **Subscription** tab, enter the name of the queue to display.
4. On the **AMQP Broker** tab, select the broker to which you want to add the subscription.
5. On the **Queues** tab, select all the queues for message subscription.
6. Click **Run**.

After the workflow runs successfully, a child of the broker appears in the **Inventory** view.

You can create a policy.

Update a Broker

You can run a workflow to update the broker properties.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `amqp` and `configuration` tags in the workflow search box.
2. Locate the **Update a broker** workflow and click **Run**.
3. On the **AMQP Broker** tab, select the broker that you want to update.
   Current properties of the broker appear on the **New AMQP connection properties** tab.
4. On the **New AMQP connection properties** tab, edit the properties that you want.
5. Click **Run**.

Using the AMQP plug-in workflow library

The AMQP workflow category contains workflows that allow you to run AMQP operations.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `amqp` tag in the workflow search box.

Workflow Name	Description
Bind	Creates a binding in a specified broker.
Declare a queue	Adds a queue to a specified broker.
Declare an exchange	Adds an exchange to a specified broker.
Delete a queue	Deletes a queue from a specified broker.
Delete an exchange	Deletes an exchange from a specified broker.
Receive a text message	Receives a text message from a specified broker.
Send a test message	Sends a text message using a specified broker.
Unbind	Unbinds binding in a specified broker.

Declare a Binding

You can run a workflow to create a binding in a specified broker.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `amqp` tag in the workflow search box.
2. Locate the **Bind** workflow and click **Run**.
3. On the **AMQP Broker** tab, select a broker in which you want to create a binding.
4. On the **Binding Properties** tab, provide information about the binding.

Option	Action
Queue name	Enter the name of the queue.
Exchange name	Enter the name of the exchange.
Routing key	Enter the routing key.

5. Click **Run**.

Declare a Queue

You can run a workflow to add a queue to a specified broker.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `amqp` tag in the workflow search box.
2. Locate the **Declare a queue** workflow and click **Run**.
3. On the **AMQP Broker** tab, select a broker to which you want to add the queue.
4. On the **Queue Properties** tab, define the queue properties.
   a) In the **Name** text box, enter the name of the queue to display.
   b) Select whether the queue is durable.

Option	Description
Yes	The queue is removed after a broker restart.
No	The queue remains after a broker restart.

   c) Select whether an exclusive client is set for the specific queue.

Option	Description
Yes	Sets one client for this specific queue.
No	Sets more clients for this specific queue.

   d) Select whether to delete the queue with activated subscription automatically.

Option	Description
Yes	Automatically deletes the queue when no more clients are connected to it. The queue remains until at least one client subscribes to it.
No	Does not delete the queue.

5. Click **Run**.

Declare an Exchange

You can run a workflow to add an exchange in a specified broker.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `amqp` tag in the workflow search box.
2. Locate the **Declare an exchange** workflow and click **Run**.
3. On the **AMQP Broker** tab, select a broker to which you want to add the exchange.
4. On the **Exchange Properties** tab, define the exchange properties.
   a) In the **Name** text box, enter the name of the queue to display.
   b) Select the exchange type.

Option	Description
direct	Makes a direct match between the routing key provided in the message and the routing criteria used when a queue is bound to this exchange.
fanout	Forwards any message sent to this exchange to all queues bound to it. Queues that are bound to this exchange contain no arguments.
headers	Queues are bound to this exchange with a table of arguments that can contain headers and values. A special argument named x-match determines the matching algorithm.
topic	Performs a wildcard match between the routing key and the routing pattern specified in the binding.

   c) Select whether the exchange is durable.

Option	Description
Yes	The exchange remains after a broker restart.
No	The exchange is removed after a broker restart.

   d) Select whether to delete the exchange with activated subscription automatically.

Option	Description
Yes	Automatically deletes the exchange when no more queues are bound to it. The exchange remains until at least one queue is bound to it.
No	Does not delete the exchange.

5. Click **Run**.

Send a Text Message

You can run a workflow to send a text message using a specified broker.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `amqp` tag in the workflow search box.
2. Locate the **Send a text message** workflow and click **Run**.
3. On the **AMQP Broker** tab, select a broker from which you want to send a message.
4. On the **Exchange** tab, specify the name of the exchange and the routing key.
5. On the **Message** tab, enter the message you want to send.
6. Click **Run**.

Delete a Binding

You can run a workflow to delete a binding in a specified broker.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an AMQP broker from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `amqp` tag in the workflow search box.
2. Locate the **Unbind** workflow and click **Run**.
3. On the **AMQP Broker** tab, select a broker to remove the binding from.
4. On the **Binding Properties** tab, enter the name of the queue, the name of the exchange, and the routing key.
5. Click **Run**.

Using the Auto Deploy Plug-In

You can use the preinstalled Auto Deploy plug-in for VMware Cloud Foundation Operations orchestrator to manage the ESXi hosts on your vCenter Server. You use the plug-in to perform different operations associated with your vCenter ESXi hosts, such as applying image profiles and host profiles or changing the location where the host is stored.

> **Note:**
>
> Starting with the 9.0 release, the Auto Deploy plug-in for VMware Cloud Foundation Operations orchestrator is deprecated and will be removed in a future release.

An Image profile includes the vSphere Installation Bundles (VIBs) that define the installation and upgrade process for your ESXi host. Image profiles are managed through the vSphere Auto Deploy service and defined through the vSphere Image Builder service. For more information on image profiles, see Image Profiles.

Image profiles and their associated VIBs are stored in software depots. A software depot is a hierarchy of files and folders and can be available through an HTTP or HTTPS URL address (online depot) or a ZIP file (offline depot).

A host profile includes the configuration profile used by a specific ESXi host. Host profiles can be extracted from one ESXi host and applied to a different host that you want to apply the same configuration to. For more information on host profiles, see vSphere Host Profiles.

Configuring and the Auto Deploy Plug-In

You must configure the Auto Deploy plug-in before you can begin using it to manage your ESXi hosts.

- Enable the Auto Deploy service in vCenter.
  - Log in to vCenter and then navigate to **Home** > **Auto Deploy**. By default, only users with an administrator role have the privileges to use the **vCenter Auto Deploy Service**.
  - On the Auto Deploy page, click **Enable Auto Deploy and Image Builder** to activate the service. If the **Image Builder** service is already active, select the **Configure** tab and click **Enable Auto Deploy Service**.
- Add a vCenter to your VCF Operations orchestrator deployment.
  Log in to the VCF Operations orchestrator and run the **Add a vCenter Server instance** workflow included with the vCenter plug-in for VCF Operations orchestrator.

  > **Note:**
  > Each instance of the Auto Deploy plug-in is associated with a single vCenter instance.

1. Log in to the VCF Operations orchestrator.
2. Add a vCenter host and image profile depot.
   a) Select the **Add an Auto Deploy host** workflow.
   b) Select the vCenter that you have added to your VCF Operations orchestrator deployment and click **Run**.
   c) Select the **Add a depot** workflow.
   d) Enter a depot name, enter the depot URL address, and click **Run**.

      You have added a depot to your Auto Deploy plug-in. You can view the image profiles hosted on the depot by navigating to the **Inventory** page and expanding the **Auto Deploy** entry.

Create and Apply an Auto Deploy Rule

Modify your ESXi host by creating and applying an Auto Deploy rule.

Configure the Auto Deploy plug-in. See Configuring and the Auto Deploy Plug-In.

The following use case describes how you can use the Auto Deploy plug-in for VMware Cloud Foundation Operations orchestrator to create and apply a simple deploy rule that adds an image profile to one of your ESXi hosts and changes the vSphere Client folder where the host is located.

1. Log in to the VCF Operations orchestrator.
2. Create a deployment rule.
   a) Select the **Create a deploy rule** workflow.
   b) Select your Auto Deploy host, enter a name for your rule, and select an image profile.
   c) Optional: Apply an extracted host profile.

      By extracting a host profile from an ESXi host, you can apply the host configuration, including settings like virtual switch configurations, to any host to which you apply the deploy rule. To extract a host profile, log in to the vSphere Client, right-click the host, and select **Host Profiles** > **Extract Host Profile**.
   d) Optional: Select a host pattern.

      Selecting a pattern can be useful when you want to specify the type of host to which you want to apply the deploy rule. For example, you can specify that you want the rule to apply to hosts that use a specific hardware vendor.
   e) Under **Type**, select **Host Folder**.
   f) Under **Host folder**, select the vCenter folder where you want to move the host when the rule is activated and the host is re-provisioned.
   g) Click **Run**.
3. Validate that the rule is added to the vCenter.
   a) Log in to the vSphere Client as an **administrator**.
   b) Under the top-left menu, select **Auto Deploy** and navigate to the **Deploy Rules** tab.
4. Activate the deploy rule.
   a) Log in to the VCF Operations orchestrator.
   b) Select the **Activate a deploy rule and a working set** workflow.
   c) Select your deploy rule, and click **Run**.
   d) Optional: Under **Position**, select the rule order.

      Note:
      An ESXi host can match more than one vSphere Auto Deploy rule criteria. In this case, the rule order set in the **Position** field is considered.
   e) Click **Run**.

      After the workflow finishes running, the status of the rule in the vSphere Client changes to **Active**. To apply the rule to a host, you must set the host to run in maintenance mode, and then re-provision the host.

5. Set the host to maintenance mode.
   a) Select the **Enter maintenance mode** workflow included with the vCenter plug-in for VMware Cloud Foundation Operations orchestrator.
   b) Select your host.
   c) Under **VSAN host decommission mode**, select **noAction**.
   d) Click **Run**.
6. Re-provision your host.
   a) Select the **Reprovision a host by a simple reboot** workflow.
   b) Select your Auto Deploy host and ESXi host to which you want to apply the deploy rule.
   c) Click **Run**.

   The new image profile is applied and the host is moved to the vSphere Client folder specified in step 3.
7. Exit maintenance mode.
   a) Select the **Exit maintenance mode** workflow.
   b) Select your host, and click **Run**.

You have created and applied a deploy rule to one of your vCenter ESXi hosts.

Troubleshooting

If you add or modify an object, such as a deploy rule, directly from the vCenter instance for which you have an Auto Deploy host, the plug-in can't detect and fetch changes in the vCenter.

To re-fetch all new or modified objects, remove the Auto Deploy host and then add it back again.

Using the Configuration Plug-In

You can modify the VCF Operations orchestrator server configuration settings by running workflows from the Configuration plug-in.

With the Configuration plug-in, you can configure and manage the VCF Operations orchestrator server keystores and trusted certificates.

SSL Trust Manager Workflows

The SSL Trust Manager category contains workflows that you can use for deleting and importing SSL certificates.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `configuration` and `ssl_trust_manager` tags in the workflow search box.

Workflow Name	Description
Delete a trusted certificate	Deletes an SSL certificate from the server trust store.
Import certificate from URL	Imports an SSL certificate from a URL into the server trust store.
Import a certificate from URL using authenticated proxy server	Imports an SSL certificate from a URL that is reachable through an authenticated proxy server.
Import certificate from URL using proxy server	Imports an SSL certificate from a URL that is reachable through a proxy server.
Import certificate from URL with certificate alias	Imports an SSL certificate from a URL into the server trust store.
Import trusted certificate from a file	Imports an SSL certificate from a file into the server trust store.

Keystore Workflows

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `configuration` and `keystores` tags in the workflow search box.

Workflow Name	Description
Add certificate	Adds a certificate to a keystore.
Add key	Adds a key.
Create a keystore	Creates a new keystore.
Delete a keystore	Deletes a keystore.
Delete certificate	Deletes a certificate from a keystore.
Delete entry	Deletes an entry.
Delete key	Deletes a key.

Using the Dynamic Types Plug-In

The VCF Operations orchestratorDynamic Types plug-in allows you to define dynamic types, create objects of these types, and set relations between them. You can use the Dynamic Types plug-in to expose third-party objects as custom types in the VCF Operations orchestrator Scripting API.

Define dynamic types

The definition of a dynamic type contains the descriptions of its properties and a set of finder workflows and actions which can be used to find dynamic objects of this type. Runtime instances of dynamic types are called dynamic objects. You can run workflows on the dynamic objects you create and perform different operations on them.

Each dynamic type must be defined in a namespace. Namespaces are helper dynamic objects that let you group dynamic types in containers.

1. Define a new dynamic type and its properties by running the Define Namespace and Define Type workflows from the Dynamic Types plug-in. In result, you obtain a set of finder and inventory workflows for finding objects of the new dynamic type and their relations with other objects.
2. Modify the new finder and inventory workflows, so that they receive their input from the third-party REST API.
   a. Create REST operations by using the Add a REST Operation workflow from the HTTP-REST plug-in and map these operations to the corresponding REST API methods.
   b. Modify the finder and inventory workflows to invoke these REST operations and consume their outputs.

Using the HTTP-REST Plug-In

The HTTP-REST plugin allows you to manage REST Web services by providing interaction between VCF Operations orchestrator and REST hosts. You can define REST services and their operations as inventory objects by running configuration workflows, and perform REST operations on the defined objects.

The plug-in contains a set of standard workflows related to managing REST hosts and invoking REST operations. You can also generate custom workflows to automate tasks in a REST environment.

Security Hardening

Sensitive information is considered internal to the corresponding HTTP-REST plug-in objects and intended for use only in scenarios involving these objects. Therefore, specific scripting properties and methods that might be used to access sensitive information are removed. Instead, you can store sensitive information in VCF Operations orchestrator configuration elements or external password vaults.

Persistent and Transient REST Hosts

The HTTP-REST plug-in supports two types of REST hosts that you can use to make requests to REST endpoints - persistent hosts and transient hosts.

Differences between persistent and transient hosts

The following table compares the two types of REST hosts.

Persistent Hosts	Transient Hosts
Stored in the VCF Operations orchestrator database.	Not stored in the VCF Operations orchestrator database. Transient hosts are virtual objects which reside in memory while a script is being executed.
Stored in the VCF Operations orchestrator inventory. Persistent hosts can also be viewed in form drop-down menus of the RESTHost type.	Not stored in the VCF Operations orchestrator inventory.
Available after restart, failover, and upgrade. When a workflow token gets interrupted, it can continue from where it left off if the workflow item takes a persistent REST host as input. Use persistent hosts as inputs/outputs of workflow items. You can create them at the beginning of the scripting and delete them if you don't need them anymore.	Not available after restart and failover. When a workflow gets interrupted, it can't restore a workflow item input which carries a transient REST host. Use transient hosts in scripting when you make isolated requests against a server which you otherwise don't use.
Can be exported and imported as resource elements.	Transferable across different VCF Operations orchestrator instances because they are created and managed entirely from scripting. Use transient hosts when you work on multiple environments with no need to migrate persistent hosts.
Each persistent host has a dedicated HTTP client that is used for managing requests to the endpoint.	Hosts reuse the same HTTP client instance.
Parallel requests are supported for persistent and transient hosts. • If you activate parallel requests, each request is executed with a separate context, and the state, including cookies, is not preserved between requests. • If support for parallel requests is deactivated, consecutive requests share the same HTTP context.	

Considerations for transient hosts

When creating transient hosts, consider the following.

- Transient hosts passed between workflow items as input/output might not work in all cases. Transient hosts rely on workflow cache, which doesn't work when asynchronous workflows are started, for example. Nested workflows might also fail.
- Only `GET` and `HEAD` requests get redirected automatically. URL redirection uses the `default` strategy.
- Host name verification is not supported.
- Client certificate authentication is not supported.

Troubleshooting

If you use transient hosts without support for parallel requests, you might experience scripting regressions after upgrading your VCF Operations orchestrator environment or upgrading the HTTP-REST plug-in to version 2.4.1.19272162 or later. Using different transient host instances to run requests, which depend on one another for cookies, is not supported.

To avoid this problem, use one of the following methods.

1. Instead of transient hosts, use persistent hosts and operations. You can create persistent REST hosts in one of two ways.
   a. Create a REST host pointing to the server using the **Add a REST host** workflow.

Instead of using transient hosts, use the REST host as input everywhere where you need to create a request to it.

- Do not create transient operations pointing to this host. Create regular operations instead.
- Support for parallel requests must be deactivated, otherwise cookies are not preserved.

This approach is not recommended if you make multiple requests in parallel to this host in your workflows.

b. Create a REST host per workflow run from the scripting, and then delete it.
Use this method if you make parallel requests to the server. For example, if you have two parallel requests, create two different hosts.

1. Clone a workflow.
2. Add a scripting element which creates the host that you want to use for future requests.
3. Use the host as the output of the workflow and as input to all other scripting that makes requests to that host.
4. To clean the state, add an element at the end of the scripting which deletes the host you created.

2. Use one transient host for all dependent requests in a given workflow and pass it between workflow items as input/output as needed.
Passing transient hosts between multiple workflow elements is not officially supported, but expected to work. Note that during restart, the workflow state might be lost and the workflow might not resume successfully.
If you use transient hosts and you want to make requests which depend on each other for cookies, you must use the same transient host instance for all requests. If the requests span multiple workflow items, create the host in the first workflow item and then pass it as input to the rest.
3. Use your current transient hosts, but modify the failing requests to include the necessary cookies by adding the respective headers.
You might have to parse the cookies from the previous response and use them in subsequent requests.

Configuring the HTTP-REST Plug-In

You can configure HTTP-REST by running the configuration workflows included in the plug-in. The Configuration workflow category contains workflows that help you to manage REST hosts.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `http-rest` and `configuration` tags in the workflow search box.

Workflow Name	Description
Add a REST host	Adds a REST host to the plug-in inventory.
Add a REST host by Swagger spec as a string	Adds a REST host based on a Swagger spec web resource provided as a string.
Add a REST host by Swagger spec from a URL	Adds a REST host based on a Swagger spec available at a specific URL.
Add a REST operation	Adds an operation to a REST host.
Add schema to a REST host	Adds an XSD schema to a REST host.
Clone a REST host	Creates a clone of a REST host.
Clone a REST operation	Creates a clone of a REST operation.
Reload plug-in configuration	Refreshes the list of REST hosts in the plug-in inventory.
Remove a REST host	Removes a REST host from the plug-in inventory.
Remove a REST operation	Removes an operation from a REST host.
Remove schemas form a REST host	Removes all associated XSD schemas from a REST host.
Update a REST host	Updates a REST host in the plug-in inventory.
Update a REST operation	Updates an operation on a REST host.

Add a REST Host

You can run a workflow to add a REST host and configure the host connection parameters.

1. Log in to the VCF Operations orchestrator Client as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `http-rest` and `configuration` tags in the workflow search box.
3. Locate the **Add a REST host** workflow and click **Run**.
4. On the **Host Properties** tab, enter the properties of the new host.
   a) In the **Name** text box, enter the name of the host.
   b) In the **URL** text box, enter the address of the host.

      Note:
      The Kerberos authentication requires a Fully Qualified Domain Name (FQDN) host address.
   c) In the **Connection timeout** text box, enter the number of seconds before a connection times out.
   d) In the **Operation timeout** text box, enter the number of seconds before an operation times out.
   e) Select whether you want to accept the REST host certificate and add it to the VCF Operations orchestrator server trust store.
   f) Select whether you want to be able to run parallel requests.
   g) Select a redirect strategy.

Option	Description
defaultRedirect	Redirects only when the `HEAD` and `GET` methods are used.
alwaysRedirect	Redirects all HTTP methods, including `POST`, `PUT`, `DELETE`, and so on.
neverRedirect	Forbids all redirects and does not follow 3xx status codes with any HTTP method.

5. On the **Host Authentication** tab, select the authentication type.

Option	Description
None	No authentication is required.
OAuth 1.0	On the **OAuth 1.0** tab, provide the required authentication parameters.
OAuth 2.0	On the **OAuth 2.0** tab, provide the authentication token and select a token sending strategy. • If you select **Authorization header**, the token is sent in the authorization header of each request made to the host as a bearer token. • If you select **Query parameter**, the token is sent in the `oauth_token` query parameter of each request made to the host. Alternatively, you can configure the authentication settings using the VCF Operations orchestrator scripting API. For instructions, see Using the Scripting API to Configure or Update a REST Host Authentication.
Basic	Provides basic access authentication. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator client retrieves credentials from the user who is logged in.

Option	Description
Digest	Provides digest access authentication that uses encryption. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator client retrieves credentials from the user who is logged in.
NTLM	Provides NT LAN Manager (NTLM) access authentication within the Window Security Support Provider (SSPI) framework. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator client retrieves credentials from the user who is logged in. On the **NTLM** tab, provide the NTLM settings.
Kerberos	Provides Kerberos access authentication. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator client retrieves credentials from the user who is logged in.

6. Optional: On the **Proxy Settings** tab, select whether to use a proxy server.
   a) Enter the address and the port of the proxy server.
   b) Select the proxy authentication type.

Option	Description
None	No authentication is required.
Basic	Provides basic access authentication. Supported only for REST hosts configured with HTTP. On the **Proxy Credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator client retrieves credentials from the user who is logged in.

7. On the **SSL** tab, select whether you want the target hostname to match the name stored in the server certificate.
8. Optional: Select a keystore entry to use to authenticate against the server. The keystore entry must be of the `PrivateKeyEntry` type.
9. Click **Run**.

After the workflow runs successfully, the REST host appears in the **Inventory** view.

You can add operations and XSD schema to the REST host. You can execute requests to the REST host based on the created REST operations using the default REST plug-in workflows, or using the VCF Operations orchestrator scripting.

You can also add a REST host with client certification authentication. To add a REST host with client certificate authentication:

1. (Optional) Create a dedicated keystore for storing certificate private keys by running the **Create a Keystore** workflow.
2. Add the certificate private key to the keystore by running the **Add key** workflow.

3. Add the REST host by running the **Add a Rest Host** workflow.
   a. Under **Host Authentication Type**, enter `None`.
   b. Select the previously added private key as the value for **PrivateKeyEntry in SSL** section of the workflow input form. Alternatively, if you want to add the key from the scripting, specify the ID by using the internal VCF Operations orchestrator ID for the element with the `keystore://<keystore_ID>/<key_name>` format.
   c. Run the workflow.

Add a REST Operation

You can run a workflow to add an operation to a REST host from the plug-in inventory.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a REST host from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `http-rest` and `configuration` tags in the workflow search box.
2. Locate the **Add a REST operation** workflow and click **Run**.
3. Select the parent host to which you want to add the operation.
4. In the **Name** text box, enter the name of the operation.
5. In the **Template URL** text box, enter only the operation part of the URL.
   You can include placeholders for parameters that are provided when you run the operation.

   The following is an example URL syntax.

   ```
   /customer/{id}/orders?date={date}
   ```
6. Select the HTTP method that the operation uses.
   If you select **POST** or **PUT**, you can provide a Content-Type request header for the method.
7. Click **Run**.

You can run workflows on the operation from the **Inventory** view.

Add a Schema to a REST Host

You can run a workflow to add an XSD schema to a REST host from the plug-in inventory.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a REST host from the **Inventory** view.

The XSD schema describes the XML documents that are used as input and output content from Web services. By associating such a schema with a host, you can specify the XML element that is required as an input when you are generating a workflow from a REST operation.

1. Navigate to **Library** > **Workflows** and enter the `http-rest` and `configuration` tags in the workflow search box.
2. Locate the **Add a schema to a REST host** workflow and click **Run**.
3. On the **Host** tab, select the host to which you want to add the XSD schema.
4. On the **XSD Schema Details** tab, select whether to load the schema from URL.

Option	Action
Yes	Enter the URL of the schema.
No	Provide the schema content.

5. Click **Run**.

Generate a New Workflow from a REST Operation

You can create a custom workflow from a REST operation.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a REST host from the **Inventory** view.

You can integrate custom-generated workflows into high-level workflows. For more information about workflow development, see the *Developing Workflows with VCF Operations orchestrator* guide.

1. Navigate to **Library** > **Workflows** and enter the `http-rest` tag in the workflow search box.
2. Locate the **Generate a new workflow from a REST operation** workflow and click **Run**.
3. Select the REST operation from the list of available operations.
   If the operation takes input and XSD schemas are added to its host, you can specify the request input type.
4. In the **Name** text box, type the name of the workflow to generate.
5. Select the workflow folder in which to generate the new workflow.
   You can select any existing folder from the workflow library.
6. Click **Run**.

Invoking a REST Operation

To make REST requests, you can either invoke a configured REST operation or invoke a REST operation dynamically by using a configured REST operation as a template and replacing any of the parameters at runtime.

There are several ways to invoke a REST operation.

- Configure REST hosts and associate REST operations with them by running the **Add a REST Host** and **Add a REST Operation** workflows. The registered REST hosts and REST operations are persistent and can be found in the **Inventory** and **Resources** views.
- Invoke a REST operation without previously configuring REST hosts and adding REST operations by running the **Invoke a dynamic REST operation** workflow from **Library** > **Workflows**. With this workflow, you can provide REST host base URL and operation parameters. The data is not persistent and is not available in the **Inventory** and **Resources** views.
- Configure REST hosts, associate REST operations with them, and use the configured REST hosts and REST operations as templates for further use, by running the **Invoke a REST host with dynamic params** and **Invoke a REST operation with dynamic params** workflows from **Library** > **Workflows**. You can replace some of the parameters of already configured REST hosts and REST operations when you run the workflows. The original REST hosts and REST operations are not affected.

Invoke a REST Operation

Call a REST operation directly.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a REST host from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `http-rest` and `configuration` tags in the workflow search box.
2. Locate the **Invoke a REST operation** workflow and click **Run**.
3. On the **Operation** tab, select the REST operation from the list of available operations.
4. Provide the input parameters and content that the operation requires.
5. Click **Run**.

Using the Library Plug-In

You can use the Library plug-in workflows as templates for customization and automation of client processes, and to troubleshoot VCF Operations orchestrator. The Library plug-in provides workflows in the **Locking**, **Orchestrator**, and **Tagging** workflow categories.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the tags in the workflow search box.

Locking Workflows

Use the `locking` tag.

Workflow Name	Description
Display all locks	Shows all locks.
Locking test	A test workflow that creates a lock.
Locking test (x5)	A test workflow that creates five locks.
Release all locks	Releases all locks.

Task Workflows

Use the `tasks` tag.

Workflow Name	Description
Create recurrent task	Creates a recurrent task and returns the newly created task.
Create task	Schedules a workflow to run at a later time and date, as a task.

Orchestrator Workflows

Use the `orchestrator` and `workflows` tags in the workflow search box.

Workflow Name	Description
Refresh stale workflow runs in waiting state	Processes all workflow runs that are in waiting state for the specified remote server and updates the workflow state according to the remote workflow run. You can use this workflow if there is data loss between the workflow runs, for example, when there is loss of connectivity between the VCF Operations orchestrator servers.
Start workflows in a series	Runs a workflow multiple times in a series, one instance after the other. You provide workflow parameters in an array. You also provide a property list, with one property per workflow input, for each instance of the workflow that starts. The number of properties in the array define the number of workflow runs.
Start workflows in parallel	Runs a workflow multiple times, with different parameters. You provide workflow parameters in an array. You also provide a property list, with one property per workflow input, for each instance of the workflow that starts. The number of properties in the array define the number of workflow runs.

Tagging Workflows

Use the `tagging` tag.

Workflow Name	Description
Find objects by tag	Finds objects by the tags assigned to them. You provide the names and values of the tags and the workflow returns a list of the objects to which these tags apply.
List workflow tags	Lists the tags assigned to the workflow you specified as an input parameter.
Tag workflow	Assigns a tag to a workflow. You must specify the workflow you want to tag and the tag name and value.
Tagging example	Demonstrates workflow tagging.
Untag workflow	Removes a tag from a workflow. You must specify the workflow you want to untag and the tag you want to remove from the workflow.

Using the Mail Plug-In

You can send email messages from workflows by using the Mail plug-in, which uses the Simple Mail Transfer Protocol (SMTP). For example, you can create a workflow to send an email to a given address if the workflow requires user interaction or when it completes its run.

Using the Mail plug-in sample workflows

You can call the sample workflows of the Mail plug-in from custom workflows to implement the email functionality to the custom workflows. You can run an example workflow to test the interaction between VCF Operations orchestrator and your SMTP server.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `mail` tag in the workflow search box.

> **Note:**
> Before you can access the workflows, verify that the user account that you are logged in with has the necessary permissions to run Mail workflows.

Workflow Name	Description
Configure mail	Defines the connection to the SMTP server, the SMTP authentication account, and the address and display name of the sender.
Retrieve messages	Retrieves the messages of a given email account by using the POP3 protocol.
Retrieve messages (via MailClient)	Retrieves the messages of a certain email account, without deleting them, by using the new scripting API provided by the `MailClient` class.
Send notification	Sends an email with specified content to a given email address. If optional parameters are not specified, the workflow uses the default values set through the Configure mail workflow.
Send notification to mailing list	Sends an email with specified content to a given email address list, CC list, and BCC list. If optional parameters are not specified, the workflow uses the default values set through the Configure mail workflow.

Define the default SMTP connection

You can set the default email account that can authenticate against an SMTP server to send and receive email notifications.

> **Note:**
> Avoid load balancers when configuring mail in VCF Operations orchestrator. Otherwise, you might receive `SMTP_HOST_UNREACHABLE` error.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `mail` tag in the workflow search box.
3. Locate the **Configure mail** workflow and click **Run**.
4. On the **Host** tab, enter the IP address or domain name of your SMTP server and a port number to match your SMTP configuration.
   The default SMTP port is 25.
5. On the **Credentials** tab, enter a user name and password for authentication.
   Enter a valid email account and an associated password. VCF Operations orchestrator uses the email account to send emails.
6. On the **Email Content** tab, enter a sender's email address and name.
   The sender information appears in all emails sent by VCF Operations orchestrator.
7. Click **Run**.

Using the Multi-Node plug-in

The Multi-Node plug-in workflow library contains workflows for hierarchical orchestration, management of VCF Operations orchestrator instances, and scale-out of VCF Operations orchestrator activities.

Multi-Node plug-in schema

The Multi-Node plug-in creates a primary-secondary relation between VCF Operations orchestrator servers, which extends in the areas of package management and workflow execution.

The plug-in contains a set of standard workflows for hierarchical orchestration, management of VCF Operations orchestrator instances, and scale-out of VCF Operations orchestrator activities.

Access the plug-in API

VCF Operations orchestrator provides an API Explorer to allow you to search the Multi-Node plug-in API and see the documentation for JavaScript objects that you can use in scripted elements.

Access the API Explorer from either the VCF Operations orchestrator Client or from the **Scripting** tabs of the workflow, policy, and action editors.

You can copy code from API elements and paste it into scripting boxes. For more information about API scripting, see *Developing with VCF Operations orchestrator*.

Configuring the Multi-Node plug-in

You must use the VCF Operations orchestrator Client to configure the Multi-Node plug-in.

The Servers Configuration workflow category contains workflows that allow you to configure the connected VCF Operations orchestrator servers.

To access these workflows, navigate to **Library** > **Workflows** and enter the `orchestrator` and `servers_configuration` tags in the workflow search box.

Workflow Name	Description
Add an Orchestrator server	Adds a VCF Operations orchestrator server to the plug-in inventory.
Delete an Orchestrator server	Removes a VCF Operations orchestrator server from the plug-in inventory and deletes all created proxies for this server.
Update an Orchestrator server	Updates a VCF Operations orchestrator server from the plug-in inventory by changing its details.

Add an VCF Operations orchestrator server

You can run a workflow to establish a connection to a new VCF Operations orchestrator server.

Verify that the primary and secondary VCF Operations orchestrator servers are the same version.

1. Log in to the VCF Operations orchestrator Client as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `orchestrator` and `servers_configuration` tags in the workflow search box.
3. Locate the **Add an Orchestrator server** workflow and click **Run**.
4. On the **Server details** tab, provide the host address and the port of the remote VCF Operations orchestrator server.
   a) Select whether the certificate is accepter silently and added to the trusted store.
   b) Select whether to generate proxy workflows for the remote VCF Operations orchestrator server.
5. On the **Connection settings** tab, provide the connection settings.
   a) In the **Connection timeout** text box, enter the number of seconds within which VCF Operations orchestrator must connect to the remote server, otherwise the connection times out.
   b) In the **Socket timeout** text box, enter the number of seconds within which the request must succeed before it times out.
   c) In the **Retry timeout** text box, enter the number of seconds within which the proxy workflows wait to receive a notification from the remote VCF Operations orchestrator server if there is no connectivity.
6. On the **Connection mode** tab, select whether the connection is shared.

Option	Description
No	The credentials of the logged-in user are used to connect to the remote VCF Operations orchestrator server.
Yes	All users can access the remote VCF Operations orchestrator server using the same credentials. Provide the credentials for the shared connection.

7. Click **Run**.

Using proxy workflows

You can use proxy workflows to manage the interaction between the local VCF Operations orchestrator server and workflows on a remote VCF Operations orchestrator server.

You can use the Multi-Node plug-in to generate local workflows which interact with remote workflows. These local workflows are called proxy workflows. A proxy workflow takes the input parameters from the inventory of the Multi-Node plug-in. When you run the proxy workflow, it converts the parameters to the types required by the remote workflow. When the remote workflow finishes its run, the output parameters are converted back to the local representation on the primary VCF Operations orchestrator server.

Synchronous proxy workflows

The synchronous type of proxy workflows preserve the API and the operation contract of the remote workflows.

The schema of all synchronous proxy workflows is the same, but contains different scripting.



The synchronous proxy workflow completes the run after the remote workflow completes and provides output parameters.

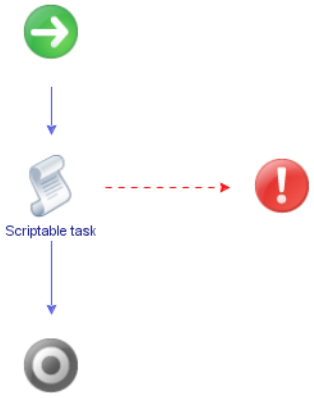
The local workflow consumes no server resources while waiting for the results of the remote workflow.

At the end of a successful run the output parameters of the proxy workflow contain a local representation of the remote workflow token. The output parameters can be used directly by other workflows on the local VCF Operations orchestrator server when they are of simple type, such as, boolean, number, string, and similar.

Asynchronous proxy workflows

You can use asynchronous proxy workflows to optimize the run of remote workflows.

The schema of all asynchronous proxy workflows is the same, but contains different scripting.



An asynchronous proxy workflow returns immediately a result that is a local wrapper of the remote workflow token object. The proxy workflow uses this token to check the state of the run and to retrieve the output parameters when the remote workflow completes its run. The output parameters can be used directly by other workflows on the local VCF Operations orchestrator server when they are of simple type, such as, boolean, number, string, and similar.

Remote execution workflows

The Remote Execution workflow category contains workflows that allow you to manage proxy workflows.

Remote Execution Standart Workflows

To access these workflows, navigate to **Library** > **Workflows** and enter the `orchestrator` and `remote_execution` tags in the workflow search box.

Workflow Name	Description
Create a multi proxy action	Creates a multi-proxy action to run workflows on multiple servers.
Create a proxy workflow	Creates a proxy workflow that you can use to start a workflow on a remote VCF Operations orchestrator server.
Create proxy workflows from a folder	Creates proxy workflows for all workflows in a folder on the remote VCF Operations orchestrator server.

Server Proxies

To access these workflows, navigate to **Library** > **Workflows** and enter the `orchestrator`, `remote_execution` and `server_proxies` tags in the workflow search box.

Workflow Name	Description
Create proxy workflows for an Orchestrator server	Creates proxy workflows on the local VCF Operations orchestrator server by mirroring the remote server's structure.
Delete proxy workflows for an Orchestrator server	Removes the proxy workflows for the local VCF Operations orchestrator server and deletes all generated workflows.
Refresh proxy workflows for an Orchestrator server	Regenerates all proxy workflows for the local VCF Operations orchestrator server from the remote server.

Using the Multi-Node plug-in inventory

The Multi-Node plug-in mirrors all inventories of the connected VCF Operations orchestrator servers in the **Inventory** view.

The inventory for a single remote server consists of two major parts, system objects and plug-in objects. Both objects are wrappers of the remote objects into locally usable types:

System object System objects are under a top-level group called **System**. They contain configurations, packages, workflows, actions, and related folders. Remote system objects have individual wrapper types.

Plug-in objects Plug-in objects mirror the inventories of all plug-ins attached to the remote VCF Operations orchestrator server. Remote plug-in objects are all wrapped into a single local type **VCO:RemotePluginObject**.

Remote management workflows

The Remote Management workflow category contains workflows that allow you to manage packages and workflows on remote VCF Operations orchestrator instances.

Remote Management Packages

To access these workflows, navigate to **Library** > **Workflows** and enter the `orchestrator`, `remote_management` and `packages` tags in the workflow search box.

Workflow Name	Description
Delete a package	Deletes a package and its contents from a remote VCF Operations orchestrator server.
Delete a package by name	Deletes a package and its contents by name on a remote VCF Operations orchestrator server.
Deploy a package from a local server	Deploys a package from a local VCF Operations orchestrator server to remote VCF Operations orchestrator servers.
Deploy a package from a remote server	Deploys a package from one remote VCF Operations orchestrator server to a list of remote VCF Operations orchestrator servers.
Deploy packages from a local server	Deploys packages from a local VCF Operations orchestrator server to remote VCF Operations orchestrator servers.

Remote Management Workflows

To access these workflows, navigate to **Library** > **Workflows** and enter the `orchestrator`, `remote_management` and `workflows` tags in the workflow search box.

Workflow Name	
Delete a remote workflow	Deletes a workflow from a remote VCF Operations orchestrator server.
Delete all finished workflow runs	Deletes all finished workflow runs from a remote workflow.
Deploy a workflow from a local server	Deploys a workflow from a local VCF Operations orchestrator server to a list of remote VCF Operations orchestrator servers.
Deploy a workflow from a remote server	Deploys a workflow from a remote VCF Operations orchestrator server to a list of other remote VCF Operations orchestrator servers.

Multi-Node plug-in use cases

The Multi-Node plug-in use cases include user scenarios such as importing a package from the local VCF Operations orchestrator server to the remote servers, using multi proxy actions, as well as information about maintenance of remote and proxy workflows.

Create a multi-proxy action

You can run the Create a multi-proxy action workflow to run a workflow on several servers.

You can create an action, so that you can run a workflow on a remote VCF Operations orchestrator server at a later stage.

1. Log in to the VCF Operations orchestrator Client as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `orchestrator` and `remote_execution` tags in the workflow search box.
3. Locate the **Create a multi-proxy action** workflow and click **Run**.
4. On the **Action details** tab, enter the action name and module.

   The action name must contain only alpha-numeric characters without spaces.

   A new action is created even if another action with the same name exists.

5. On the **Workflow details** tab, select whether the workflow is local or remote.

Option	Description
Yes	Select the remote workflow that you want to use for this action.
No	Select the local workflow that you want to use for this action.

6. Click **Run**.

The generated action accepts the same parameters as the source workflow but promotes the parameters to an array in case of multi-selection of objects. The values in the array are indexed.

Maintenance of remote and proxy workflows

If the remote and proxy workflows change, you might want to update the proxies or to delete them if you do not need them anymore. For maintenance purposes, the Multi-Node plug-in provides workflows that allow you to update or delete proxy and remote workflow information.

To access the workflows for managing the proxy workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `orchestrator`, `remote_execution`, and `servers_proxies` tags in the workflow search box.

Workflow Name	Description
Refresh proxy workflows for an Orchestrator server	Regenerates all proxy workflows for the local VCF Operations orchestrator server from the remote server.
Delete proxy workflows for an Orchestrator server	Removes the proxy workflows for the local VCF Operations orchestrator server and deletes all generated workflows.

To access the workflows for further maintenance of the proxy workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `orchestrator`, `remote_management`, and `workflows` tags in the workflow search box.

Workflow Name	Description
Delete all finished workflow runs	Deletes all finished workflow runs from a remote workflow.
Delete a remote workflow	Deletes a workflow from a remote VCF Operations orchestrator server.
Deploy a workflow from a local server	Deploys a workflow from a local VCF Operations orchestrator server to a list of remote VCF Operations orchestrator servers.

Deploy a package from a local server

You can run a workflow to deploy a package from a local VCF Operations orchestrator server to remote VCF Operations orchestrator servers.

In this example, you can deploy a package from a local server to an array of remote servers.

1. Navigate to **Library** > **Workflows** and enter the `orchestrator` and `remote_management` tags in the workflow search box.
2. Locate the **Deploy a package from a local server** workflow and click **Run**.
3. Select the package to deploy from the local storage.
4. Select the remote servers to deploy the package to.
5. Select whether you want to overwrite the remote server packages.

Option	Description
Yes	The packages on the remote server are replaced, discarding the version of the packaged elements.
No	A version check of the server and the deploying packages is performed. The packages are deployed after a successful check.

6. Click **Run**.

After running the workflow, the status information is displayed in the log view and in the inventory of the plug-in.

Using the Net Plug-In

You can use the Net plug-in to implement the Telnet, FTP, POP3, and IMAP protocols in workflows. The POP3 and IMAP implementations allow downloading and reading email. In combination with the Mail plug-in, the Net plug-in provides full email sending and receiving capabilities in workflows.

Using the PowerShell plug-in

The PowerShell plug-in allows interaction between VCF Operations orchestrator and Windows PowerShell. The PowerShell plug-in workflow library contains workflows that allow you to manage PowerShell hosts and run custom PowerShell operations.

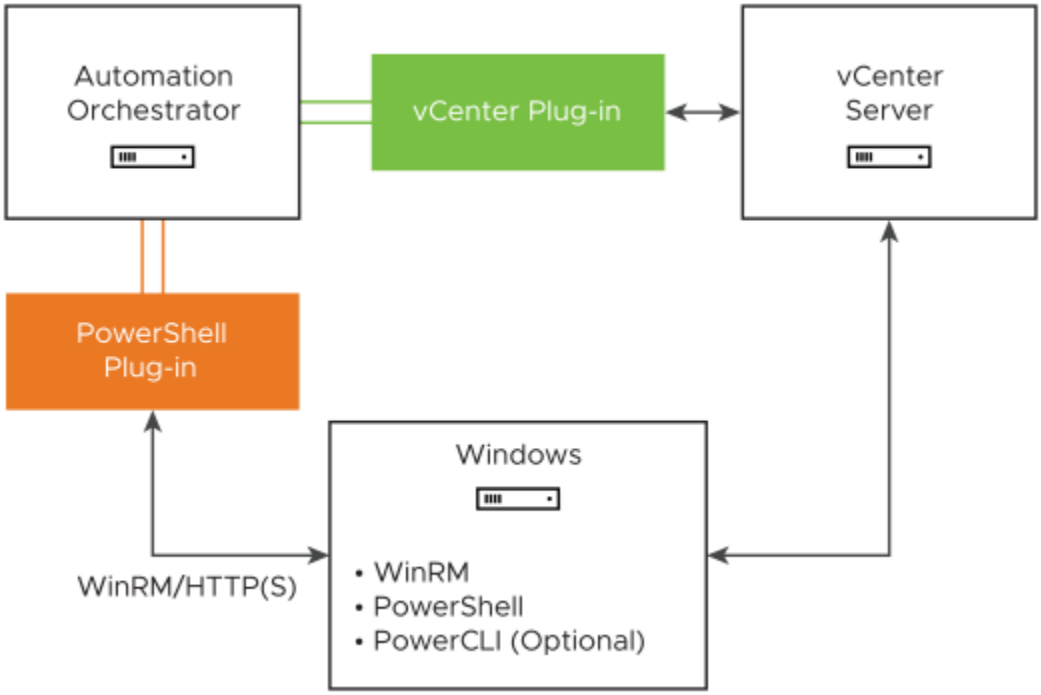
You use the plug-in to call PowerShell scripts and cmdlets from VCF Operations orchestrator actions and workflows, and to work with the result. In addition to the standard workflows that come with the plug-in, you can also create custom workflows that implement the plug-in API.

You can use the **Inventory** view in the VCF Operations orchestrator Client to manage the available PowerShell resources. You can use the scripting API of the plug-in to develop custom workflows.

PowerShell plug-in components

The PowerShell plug-in relies on a number of components to function properly.

VCF Operations orchestrator and Windows PowerShell provide the platform for the plug-in, and the plug-in provides interaction between those products. The PowerShell plug-in can also interact with other components, such as vCenter and vSphere PowerCLI.



The plug-in communicates with Windows PowerShell through the WinRM communication protocol. See Configuring WinRM.

Optionally, you can integrate the PowerShell plug-in with vSphere PowerCLI and vCenter. See PowerCLI Integration with the Plug-In.

You can install all components on a local host. The usage, functionality, and communication protocol requirements of the PowerShell plug-in do not change if VCF Operations orchestrator and Windows PowerShell are installed on the same machine.

Access the PowerShell plug-in API

To access the API Explorer from the VCF Operations orchestrator Client, click **API Explorer** in the VCF Operations orchestrator Client navigation pane.

To access the API Explorer from the **Scripting** tabs of the workflow, policy, and action editors, click **Search API** on the left.

You can copy code from API elements and paste it into scripting boxes. For more information about API scripting, see *Developing with VCF Operations orchestrator*.

Using the PowerShell plug-in inventory

The PowerShell plug-in exposes all objects in the connected PowerShell hosts in the **Inventory** view.

Within the inventory of the plug-in, you can monitor PowerShell hosts and their snap-ins and cmdlets. Each remote host can contain snap-ins and each snap-in can contain cmdlets.

Configuring the PowerShell plug-in

You must use the VCF Operations orchestrator Client to configure the PowerShell plug-in.

Configuration Workflows

You use the workflows from the Configuration category to manage PowerShell hosts.

To access these workflows, navigate to **Library** > **Workflows** and enter the `powershell` and `configuration` tags in the workflow search box.

Workflow Name	Description
Add a PowerShell host	Adds a PowerShell host to the plug-in inventory.
Remove a PowerShell host	Removes a PowerShell host from the plug-in inventory.
Update a PowerShell host	Updates the specified PowerShell host in the plug-in inventory.
Validate a PowerShell host	Validates the configuration of the specified PowerShell host.

Sample Workflows

You use the workflows from the Samples category to test basic use cases.

To access these workflows in the workflows library, enter the `powershell` and `samples` tags in the workflow search box.

Workflow Name	Description
Invoke a script via API	Demonstrates how to call a PowerShell script through the available scripting API.
List directory content	Lists the contents of a directory on the PowerShell host file system.
Pipeline execution example	Demonstrates how you can run multiple cmdlets arranged into a pipe.
Toggle virtual machine state	Toggles the power state of a virtual machine.

Configuring WinRM

To establish a connection between the PowerShell plug-in and Windows PowerShell, you must configure WinRM to use one of the supported communication protocols.

The PowerShell plug-in supports Windows Remote Management (WinRM) 2.0 as a management protocol.

The following authentication methods are supported.

Authentication method	Details
Basic	Non-secure authentication mechanism that requires a user name and a password.
Kerberos	Secure authentication protocol that uses tickets to verify the identity of the client and the server.

> **Note:**
> The PowerShell plug-in does not support delegation of user credentials in WinRM and CredSSP is not a supported authentication method.

WinRM Through HTTP

The PowerShell plug-in supports communication with the WinRM host through the HTTP protocol. Although WinRM authenticates the communication, the data transfer is not encrypted and is sent as plain text on the network. You should use the HTTP protocol if IPSec is configured between the machines that communicate.

To use Basic authentication, you must set the `AllowUnencrypted` property to `true` in both the service and client WinRM configuration. For an example of HTTP configuration, see Configure WinRM to Use HTTP.

WinRM Through HTTPS

The PowerShell plug-in supports communication with the WinRM host through the HTTPS protocol. You can use the HTTPS protocol as a more secure communication method.

To use the HTTPS protocol, you must generate a certificate for server authentication and install the certificate on the WinRM host. For an example of HTTPS configuration, see Configure WinRM to use HTTPS.

Kerberos authentication

You can use Kerberos authentication when you add and manage a PowerShell host. With Kerberos authentication, domain users can run commands on remote PowerShell-enabled machines over WinRM.

To configure WinRM on the PowerShell host, run the following command.

```
winrm quickconfig
winrm set winrm/config/service/auth '@{Kerberos="true"}'
winrm set winrm/config/winrs @{MaxMemoryPerShellMB="2048"}
```

For more information, see Configure Kerberos Authentication for VCF Operations orchestrator Plug-Ins.

Configure WinRM to Use HTTP

You can configure the WinRM host to enable communication with the PowerShell plug-in through the HTTP protocol.

You must modify the WinRM configuration by running commands on the WinRM host machine. You can use the same machine as both the WinRM service and WinRM client.

> **Important:**
> If you skip any of the steps when configuring WinRM to use HTTP, the host might not be added, and you might get an error message in the logs such as:
>
> ```
> Caused by: org.dom4j.DocumentException: Error on line -1 of document : Premature end of file.
> Nested exception: Premature end of file.
> at org.dom4j.io.SAXReader.read(SAXReader.java:482)
> at org.dom4j.DocumentHelper.parseText(DocumentHelper.java:278)
> at
>  com.xebialabs.overthere.cifs.winrm.connector.JdkHttpConnector.sendMessage(JdkHttpConnector.java:117)
> ```

1. Run the following command to set the default WinRM configuration values.

   ```
   c:\> winrm quickconfig
   ```

2. Optional: Run the following command to check whether a listener is running, and verify the default ports.

   ```
   c:\> winrm e winrm/config/listener
   ```

   The default ports are 5985 for HTTP, and 5986 for HTTPS.

3. Enable basic authentication on the WinRM service.
   a) Run the following command to check whether basic authentication is allowed.

      ```
      c:\> winrm get winrm/config/service
      ```

   b) Run the following command to enable basic authentication.

      ```
      c:\> winrm set winrm/config/service/auth '@{Basic="true"}'
      ```

4. Run the following command to allow transfer of unencrypted data on the WinRM service.

   ```
   c:\> winrm set winrm/config/service @{AllowUnencrypted="true"}
   ```

5. If the channel binding token hardening level of the WinRM service is set to `strict`, change its value to `relaxed`.

   ```
   c:\> winrm set winrm/config/service/auth @{CbtHardeningLevel="relaxed"}
   ```

6. Enable basic authentication on the WinRM client.
   a) Run the following command to check whether basic authentication is allowed.

      ```
      c:\> winrm get winrm/config/client
      ```

b) Run the following command to enable basic authentication.

```
c:\> winrm set winrm/config/client/auth @{Basic="true"}
```

7. Run the following command to allow transfer of unencrypted data on the WinRM client.

```
c:\> winrm set winrm/config/client @{AllowUnencrypted="true"}
```

8. If the WinRM host machine is in an external domain, run the following command to specify the trusted hosts.

```
c:\> winrm set winrm/config/client @{TrustedHosts="host1, host2, host3"}
```

9. Run the following command to test the connection to the WinRM service.

```
c:\> winrm identify -r:http://winrm_server:5985 -auth:basic -u:user_name -p:password -
encoding:utf-8
```

Configure WinRM to use HTTPS

You can configure the WinRM host to enable communication with the PowerShell plug-in through the HTTPS protocol.

- Configure WinRM to use the HTTP protocol. For more information, see Configure WinRM to Use HTTP.
- Verify that you can access the Microsoft Management Console (`mmc.exe`) on the WinRM host.

The WinRM host requires a certificate so that it can communicate through the HTTPS protocol. You can either obtain a certificate or generate one.

1. Generate a self-signed certificate.

   The following command line contains example syntax for creating a certificate on the WinRM host by using the Powershell Cmdlet `New-SelfSignedCertificate`.

   ```
   New-SelfSignedCertificate -CertStoreLocation cert:\localmachine\my -DnsName
    ("machinename.subdomain.company.com", "machinename") -NotAfter (get-date).AddYears(5) -Provider
    "Microsoft RSA SChannel Cryptographic Provider" -KeyLength 2048
   ```

   In this example, the `-NotAfter` parameter specifies that the certificate will expire in 5 years. If this parameter isn't specified, the certificate expires in 1 year. Learn more about creating self-signed certificates.

2. Add the generated certificate by using the Microsoft Management Console.
   a) Run `mmc.exe`.
   b) Select **File** > **Add/Remove Snap-in**.
   c) From the list of available snap-ins, select **Certificates** and click **Add**.
   d) Select **Computer account** and click **Next**.
   e) Click **Finish**.
   f) Verify that the certificate is installed in **Console Root** > **Certificates (Local Computer)** > **Personal** > **Certificates** and **Console Root** > **Certificates (Local Computer)** > **Trusted Root Certification Authorities** > **Certificates**.

      If the certificate is not installed in the Trusted Root Certification Authorities and Personal folders, you must install it manually.

3. Create an HTTPS listener by using the correct thumbprint and host name.
   The following command line contains example syntax for creating an HTTPS listener.

   ```
   winrm create winrm/config/Listener?Address=*+Transport=HTTPS
   @{Hostname="host_name";CertificateThumbprint="certificate_thumbprint"}
   ```

   Note:
   Omit the spaces in the certificate thumbprint.

   When the certificate expires, the listener must be deleted and created again with the new valid certificate.

4. Test the connection.
   The following command line contains example syntax for testing the connection.

   ```
   winrs -r:https://host_name:port_number -u:user_name -p:password hostname
   ```

Add a PowerShell host

You add a PowerShell host and configure the host connection parameters by running a workflow. You can set up a connection to a remote or a local PowerShell host.

Procedure

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `powershell` and `configuration` tags in the workflow search box.
3. Locate the Add a PowerShell host workflow and click **Run**.
4. In the **Name** text box, enter the name of the host.
5. In the **Host / IP** text box, enter the address of the host.

   Note:
   The Kerberos authentication requires a Fully Qualified Domain Name (FQDN) host address.
6. In the **Port** text box, type the port of the host.

   You use port 5985 for the HTTP or 5986 for the HTTPS protocol.
7. On the **Host Type** tab, specify the PowerShell host type that the plug-in connects to.
   a. Select a transport protocol.

      Note:
      If you use the HTTPS transport protocol, the certificate of the remote PowerShell host is imported into the VCF Operations orchestrator keystore.
   b. Select the authentication type.

      Important:
      If you want to use Kerberos authentication, you must enable it on the WinRM service.
8. On the **User Credentials** tab, select the type of session mode that the plug-in uses to connect to the PowerShell host.

Option	Description
Shared Session	The plug-in uses shared credentials to connect to the remote host. You must provide the PowerShell host credentials for the shared session. The PowerShell host credentials must belong to a member of the local administrators group.
Session per User	The VCF Operations orchestrator client retrieves credentials from the user who is logged in. You must log in with a *`user@domain`* format to VCF Operations orchestrator to use the **Session per User** mode.

9. On the **Advanced Options** tab, from the **Shell Code Page** drop-down menu, select the type of encoding that the PowerShell uses.
10. Click **Run**.

Results

After the workflow runs successfully, the PowerShell host appears in the **Inventory** view.

Troubleshooting

If the workflow fails with the `Cannot Locate KDC(Dynamic Script Module name: addPowerShellHost#30)` error, see Troubleshooting the workflow Add a PowerShell host.

Running PowerShell Scripts

You can run workflows to invoke an external or custom script on a PowerShell host.

Invoke a PowerShell Script

You can run an existing or custom PowerShell script on a host in the plug-in inventory.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a PowerShell host from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `powershell` tag in the workflow search box.
2. Locate the Invoke a PowerShell script workflow and click **Run**.
3. On the **PowerShell Host** tab, select a PowerShell host on which to run the script.
4. On the **Script** tab, in the **Script** text box, type or paste the PowerShell script that you want to run.
5. Click **Run**.

Invoke an External Script

You can run an external PowerShell script on a host in the plug-in inventory.

- Verify that you are logged in to VCF Operations orchestrator as an administrator.
- Verify that you have a connection to a PowerShell host from the **Inventory** view.
- Verify that you have access to other `.ps1` files that the script might reference.

External PowerShell scripts are contained in `.ps1` files. The `.ps1` file that you want to run must be stored on the PowerShell host.

1. Navigate to **Library** > **Workflows** and enter the `powershell` tag in the workflow search box.
2. Locate the **Invoke an external script** workflow and click **Run**.
3. On the **PowerShell Host** tab, select a PowerShell host on which to run the script.
4. On the **External Script** tab, in the **Name** text box, type the filename of the external `.ps1` script that you want to run.

   Note:
   If the `.ps1` file is not in the default folder, you must type the absolute file path. You can use system environment variables to specify script paths. For example, `$env:HOMEPATH\test1.ps1`.
5. In the **Arguments** text box, type the script arguments.
   The valid syntax is the same as used in the PowerShell console.
6. Click **Run**.

Generating Actions

You can run workflows to generate actions based on a PowerShell script or a PowerShell cmdlet. You can use the generated actions as building blocks for custom workflows.

Generate an action from a PowerShell script

You can run a workflow to generate an action from a PowerShell script that you provide. You can optionally generate a sample workflow that can run the generated action.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a PowerShell host from the **Inventory** view.

You can customize the script of the action that you generate by using placeholders. For each placeholder, the workflow creates a corresponding action parameter of type `string` in the generated action. When you run the action, you can provide an actual value as the action parameter to replace the placeholder.

1. In the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `powershell` and `generate` tags in the workflow search box.
2. Locate the **Generate an action from a PowerShell script** workflow and click **Run**.
3. On the **Script** tab, in the **Script** text box, type or paste the PowerShell script from which to generate the action.

   Note:
   You can use `{#ParamName#}` as a placeholder for user input. If the placeholder is of type `string`, you must use double quotes to pass the value of the placeholder to the action.

   The following script is an example of how to link the generated action parameter to a script parameter.

   ```
   param($name={#ParamName#})
   echo $name;
   ```

4. On the **Action** tab, in the **Name** text box, enter a name for the action that you want to generate, and select an existing module in which to generate the action.
5. On the **Workflow** tab, select whether to generate a workflow.

Option	Description
Yes	Generates a sample workflow that can run the generated action. You must select a folder in which to generate the workflow. **Note:** The name of the generated workflow consists of the predefined string Invoke Script and the name of the generated action.
No	A sample workflow is not generated.

6. Click **Submit** to run the workflow.

You can integrate the generated action in custom workflows.

Generate an action for a PowerShell Cmdlet

You can run a workflow to generate an action for a PowerShell cmdlet and parameter set that you provide. With this action, you can use PowerShell functionality in VCF Operations orchestrator. You can optionally generate a sample workflow that runs the generated action.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a PowerShell host from the **Inventory** view.

You can use a large set of data types with the PowerShell script engine. The data types that you can use include primitive types such as `Integer`, `Boolean`, `Char`, any type available from the .NET assembly, or user-defined types. When generating actions based on PowerShell cmdlet definitions, the input and output cmdlet parameters are represented by types that the VCF Operations orchestrator platform supports. The PowerShell plug-in defines the type mappings. In general, primitive types are mapped to the corresponding VCF Operations orchestrator types, and complex types are represented by the `PowerShellRemotePSObject` object.

1. In the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `powershell` and `generate` tags in the workflow search box.
2. Locate the **Generate an action for a PowerShell cmdlet** workflow and click **Run**.
3. On the **Cmdlet** tab, select a PowerShell cmdlet to run when using the action that you generate.
4. Select a parameter set for the cmdlet.

   The parameter set definition values appear in the **Parameter set definition** text box.

   Note:
   You cannot modify the parameter set definition values by editing the string in the **Parameter set definition** text box. You can review the string for information about the parameters that the parameter set contains.

5. On the **Action** tab, in the **Name** text box, enter a name for the action that you want to generate, and select an existing module in which to generate the action.
6. On the **Workflow** tab, select whether to generate a workflow.

Option	Description
Yes	Generates a sample workflow that can run the generated action. You must select a folder in which to generate the workflow. **Note:** The name of the generated workflow consists of the predefined string Execute Cmdlet and the name of the generated action.
No	A sample workflow is not generated.

7. Click **Run**.

You can integrate the generated action in custom workflows.

Passing Invocation Results Between Actions

The PowerShell plug-in supports passing of results as parameters from one PowerShell script invocation to another. To pass results correctly, both invocations must happen in the same session.

PowerCLI Integration with the PowerShell Plug-In

You can use functionality that is available in a third-party snap-in, such as VMware vSphere PowerCLI, with the PowerShell plug-in.

To use the third-party snap-in functionality, the snap-in must be available on the PowerShell host. To load the snap-in in the current session, you must also invoke the `AddPsSnapin` action. When using PowerCLI, you must set the name of the snap-in to `VMware.VimAutomation.Core`.

The PowerShell plug-in does not provide pre-generated actions for third-party snap-ins. You can generate actions for third-party snap-ins by running the Generate an action for a PowerShell cmdlet workflow. See Generate an action for a PowerShell Cmdlet.

The `com.vmware.library.powershell.converter` package contains basic building blocks that allow conversion from a `VC:<SomeObjectType>` object, to the corresponding PowerCLI object. This feature allows workflows from the vCenter plug-in to interact with workflows from the PowerShell plug-in and to pass parameters between the two plug-ins.

Converter Workflows

You can use the sample workflows from the Converter workflow category to test the integration between the PowerShell plug-in and PowerCLI. To test the integration, PowerCLI must be installed on the PowerShell host.

The Converter sample workflows demonstrate the conversion functionality available in the plug-in.

> **Note:**
> The PowerShell plug-in does not support all types that are available in PowerCLI and the vCenter plug-in. Unsupported types return an exception.

To access these workflows, navigate to **Library** > **Workflows** and enter the `powershell` and `converter` tags in the workflow search box.

Workflow Name	Description
Convert PSObject to vCO object	Converts `PowerShellRemotePSObject` to `VC:<SomeObjectType>`.
Convert PSObject to vCO object to PSObject	Converts `PowerShellRemotePSObject` to `VC:<SomeObjectType>` and the reverse.

Workflow Name	Description
Convert vCO object to PSObject	Converts `VC:<SomeObjectType>` to `PowerShellRemotePSObject`.

Working with PowerShell results

You can use objects from the PowerShell plug-in API to work with results that Windows PowerShell returns.

You can use the methods from the `PowerShellInvocationResult` class to retrieve information about a script that you run.

Method	Description
`getErrors()`	Returns a list of errors reported by the PowerShell engine during script invocation.
`getInvocationState()`	Status of the script. The possible values are `Completed` or `Failed`.
`getHostOutput()`	Output of the script as it appears on the PowerShell console.
`getResults()`	Objects returned by the PowerShell engine. The returned object is of type `PowershellRemotePSObject`.

`PowershellRemotePSObject` is a remote representation of objects returned by the PowerShell engine. `PowershellRemotePSObject` contains XML serialization of the result that can be accessed by calling the `getXml()` method.

The PowerShell plug-in also provides an object model that wraps the XML result and provides easy access to particular object properties. The `getRootObject()` method provides access to the object model. In general, the `getRootObject()` method maps the PowerShell types to types available in VCF Operations orchestrator, by using the following rules.

- If the returned object is of a primitive PowerShell type, the object is mapped to the corresponding Orchestrator primitive type.
- If the returned object is of type `collection`, the object is represented as `ArrayList`.
- If the returned object is of type `dictionary`, the object is represented as `Hashtable`.
- If the returned object is of type `complex`, the object is represented as `PSObject`.

Scripting examples for common PowerShell tasks

You can cut, paste, and edit the JavaScript examples to write scripts for common PowerShell tasks.

For more information about scripting, see the *VCF Operations orchestrator Developer's Guide*.

Run a PowerShell Script Through the API

You can use JavaScript to run a PowerShell script through the plug-in API.

This example script performs the following actions.

- Opens a session to a PowerShell host.
- Provides a script to run.
- Checks invocation results.
- Closes the session.

```
var sess;
try {
     //Open session to PowerShell host
     var sess = host.openSession()
     //Set executed script
     var result = sess.invokeScript('dir')
```

```
     //Check for errors
     if (result.invocationState == 'Failed'){
          throw "PowerShellInvocationError: Errors found while executing script \n" +
 result.getErrors();
     }
     //Show result
     System.log( result.getHostOutput() );
} catch (ex){
     System.error (ex)
} finally {
     if (sess) {
     //Close session
     host.closeSession( sess.getSessionId() );
     }
}
```

Work with Result

You can use JavaScript to work with the result of a PowerShell script run.

This example script performs the following actions.

- Checks the invocation state.
- Extracts a value from the result.
- Checks the `RemotePSObject` type.

```
var sess = host.openSession()
sess.addCommandFromString("dir " + directory)
var invResult = sess.invokePipeline();
//Show result
System.log( invResult.getHostOutput() );

//Check for errors
if (invResult.invocationState == 'Failed'){
System.error(invResult.getErrors());
     } else {
     //Get PowerShellRemotePSObject
     var psObject = invResult.getResults();
     var directories = psObject.getRootObject();

     var isList = directories instanceof Array
     if ( isList ){
          for (idx in directories){
               var item = directories[idx];
               if ( item.instanceOf('System.IO.FileInfo') ){//Check type of object
                    System.log( item.getProperty('FullName') );//Extract value from result
               }
          }
     } else {
               System.log( directories.getProperty('FullName') );//Extract value from result
     }
}

host.closeSession( sess.getSessionId());
```

Connect with Custom Credentials

You can use JavaScript to connect to a PowerShell host with custom credentials.

```
var sess;
```

```
try {
     sess = host.openSessionAs(userName, password);

     var invResult = sess.invokeScript('$env:username');

     //Check for errors
     if (invResult.invocationState == 'Failed'){
               System.error(invResult.getErrors());
     } else {
               //Show result
               System.log( invResult.getHostOutput() );
     }
} catch (ex){
     System.error (ex)
} finally {
     if (sess) {
                host.closeSession( sess.getSessionId());
     }
}
```

Troubleshooting

If you encounter problems when using the PowerShell plug-in, you can refer to a troubleshooting topic to understand the problem or solve it, if there is a workaround.

Activate Kerberos Event Logging

For troubleshooting purposes, you might want to activate Kerberos event logging on the Key Distribution Center (KDC) machine.

Back up the Windows registry.

1. Log in to the domain controller that acts as a Key Distribution Center (KDC).
2. Run the registry editor as an **administrator**.
3. In the registry window, expand `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters`.
4. If a `LogLevel` registry key value does not exist, right-click to create it.
   a) Right-click **Parameter**, select **New** > **DWORD (32-bit) Value**, and enter `LogLevel`.
   b) Select **Parameter** and in the right pane, double-click `LogLevel` and enter `1` in the **Value data:** text box.
   The new setting becomes effective without a reboot on Windows Server 2003 and later.

The Kerberos error event entries are recorded in the System Windows Event Log.

To deactivate Kerberos event logging, delete the `LogLevel` registry key value or change its value data to `0`.

Servers Not Found in Kerberos Database

After you add PowerShell servers with Kerberos authentication, the servers might not be found because they are not added correctly.

When you try to connect to a server, the server is not found in Kerberos database.

```
No valid credentials provided (Mechanism level: No valid
credentials provided (Mechanism level: Server not found in Kerberos
database (7)))
```

This error might be caused by several misconfigurations.

- The PowerShell host is not part of a domain.

- The host to realm mapping is not correct.
- The Service Principal Name of the PowerShell host is not built correctly.

Note:
Kerberos authentication does not work when the destination is an IP address.

When you add a PowerShell host using the Kerberos authentication, enter a DNS or NetBIOS destination.

Unable to Obtain a Kerberos Ticket

When you provide wrong credentials, the plug-in fails to obtain a Kerberos ticket.

You are unable to add a host to the plug-in inventory and the result is the following error message.

```
Pre-authentication information was invalid (24)
```

You have provided wrong credentials.

Provide the correct credentials.

Kerberos Authentication Fails Due to Different Time Settings

Inconsistent time settings in the environment that uses Kerberos configuration might lead to authentication failure.

Attempts to use Kerberos for initial authentication of a host or for resource access fail, and the following error message appears.

```
Clock Skew
```

If the system time on the computers in the environment differs with more than 5 minutes from the domain controller, or from one another, the Kerberos authentication fails.

Synchronize the system times in the environment.

Kerberos Authentication Session Mode Fails

When you use Kerberos authentication with Shared Session or Session per User, adding the PowerShell host might fail.

When you attempt to add a PowerShell host to the plug-in inventory using Shared Session or Session per User, the workflow fails with the following error.

```
Null realm name (601) - default realm not specified (Dynamic Script Module name :
addPowerShellHost#16)
```

The default realm is not specified in the Kerberos configuration file `krb5.conf`, neither is provided as a part of the user name.

Provide a default realm in your Kerberos configuration file or include the realm in your user name when authenticating with Kerberos.

Unable to Reach a Key Distribution Center for a Realm

Any misspelling in the `krb5.conf` file might cause a failure when you add a host.

When you are adding a host, the Kerberos authentication is unable to reach a Key Distribution Center (KDC) for *`yourrealm`*.

```
Cannot get kdc for realm YOURREALM.COM
```

The `libdefaults` and `realms` sections in the `krb5.conf` file might be misspelled.

Verify that the `libdefaults` and `realms` sections in your `krb5.conf` file are spelled correctly.

Unable to Locate the Default Realm

VCF Operations orchestrator workflows that require Kerberos authentication might fail if the Kerberos configuration file does not have the correct format or encoding.

Kerberos authentication cannot identify the default realm.

```
Cannot locate default realm
```

The Kerberos configuration file `krb5.conf` that you upload to the VCF Operations orchestrator Appliance has been edited on a non-UNIX operating system. As a result, the format and the encoding might be incorrect.

In order for the VCF Operations orchestrator appliance to read the `krb5.conf` file, the format of the file must be UNIX and the character encoding must be ANSI as UTF-8.

Using the SNMP Plug-In

The SNMP plug-in allows VCF Operations orchestrator to connect and receive information from SNMP-enabled systems and devices. You can define SNMP devices as inventory objects by running workflows, and perform SNMP operations on the defined objects.

You can use the plug-in to connect to SNMP devices such as routers, switches, network printers, and UPS devices. The plug-in can also receive events from vCenter over the SNMP protocol.

The SNMP plug-in provides two methods of communication with the SNMP devices.

- Queries for the values of specific SNMP variables.
- Listening for events (SNMP traps) that are generated from the devices and pushed to the registered SNMP managers.

The plug-in contains a set of standard workflows related to managing SNMP devices, queries, the trap host, and performing SNMP operations. You can also create custom workflows to automate tasks in an SNMP environment.

Managing SNMP Devices

You can run workflows to register SNMP devices with , edit the settings for existing devices, and unregister devices.

SNMP Device Management Workflows

The Device Management workflow category contains workflows that allow you to manage SNMP devices.

To access these workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows** and enter the `snmp` and `device_management` tags in the workflow search box.

Workflow Name	Description
Edit an SNMP device	Edits the configuration of a registered SNMP device.
Register an SNMP device	Registers an SNMP-enabled device to the plug-in inventory.
Unregister an SNMP device	Unregisters an SNMP device from the plug-in inventory.

Register an SNMP Device

You can run a workflow to register an SNMP device and optionally configure advanced connection parameters.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `snmp` and `device_management` tag in the workflow search box.
3. Locate the **Register an SNMP device** workflow and click **Run**.
4. In the **Device address** text box, enter the IP address or DNS name of the SNMP device.

   Note:
   To establish a more reliable connection, use an IP address.
5. Optional: In the **Name** text box, enter a name for the device as you want it to appear in the **Inventory** view.
   If you leave the text box blank, the device address is used to generate a name automatically.
6. Optional: To configure the advanced connection parameters, select the check box, and navigate to the **Advanced** tab.
   a) In the **Port** text box, specify the connection port.
      The default port is 161.

b) From the **Version** drop-down menu, select the SNMP version that you want to use and provide the credentials.

The support for SNMPv3 is limited to the AuthPriv security level with MD5 authentication. The DES passphrase is the same as the MD5 password.

Note:
The support for SNMPv3 is deprecated.

c) In the **Community name** text box, enter a name for the device. The default name is **public**.

7. Click **Run**.

After the workflow runs successfully, the SNMP device appears in the **Inventory** view.

You can add queries to the SNMP device and run workflows from the **Inventory** view.

Managing SNMP Queries

You can add queries to registered SNMP devices, run, copy, and edit existing queries, and remove queries from SNMP devices. You can use SNMP queries as building blocks in more complex workflows.

Add a Query to an SNMP Device

You can run a workflow to add a query to an SNMP device from the plug-in inventory.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an SNMP device from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `snmp` and `query_management` tags in the workflow search box.
2. Locate the **Add a query to an SNMP device** workflow and click **Run**.
3. Select the device to which you want to add the query.
4. From the **Type** drop-down menu, select the query type.
5. In the **OID** text box, enter the object identifier of the variable that you want to query.
   The following are example OID values.
   - `1.3.6.1.2.1.1.5.0`
   - `.1.3.6.1.2.1.1.5.0`
   - `iso.3.6.1.2.1.1.5.0`

   Note:
   The plug-in supports only OID values that are numerical or that begin with `iso` and continue with numbers.
6. Optional: In the **Name** text box, enter a name for the query.
   If you leave the text box blank, the type and OID parameters are used to generate a name automatically.
7. Click **Run**.

You can run workflows on the query from the **Inventory** view.

Managing the SNMP Trap Host

VCF Operations orchestrator can act as an SNMP listener. You can start and stop the SNMP trap host, and change the port on which VCF Operations orchestrator listens for SNMP traps.

The SNMP plug-in supports SNMPv1 and SNMPv2c traps.

Important:
When using a clustered VCF Operations orchestrator deployment, you must send the SNMP trap to all nodes in the cluster.

Add a SNMP trap port to the VCF Operations orchestrator Appliance

Before running the **Set the SNMP trap port** workflow, you must add the port to the VCF Operations orchestrator Appliance.

1. Log in to the VCF Operations orchestrator appliance command line as **root**.
2. Create a variable for the SNMP trap port.

   ```
   SNMP_PORT=port_value
   ```

   Note:
   The ports open for SNMP traps are 81 to 32767.
3. Run the `vracli network ports expose` command.

   ```
   vracli network ports expose --proto udp --targetPort $SNMP_PORT $SNMP_PORT vco-app
   ```

Set the SNMP Trap Port

You can run a workflow to set the port on which VCF Operations orchestrator listens for SNMP traps.

- Prepare the VCF Operations orchestrator Appliance for the configuration of the SNMP trap port. See Add a SNMP trap port to the VCF Operations orchestrator Appliance.
- In the VCF Operations orchestrator, select **Administration** > **Inventory** and verify that the SNMP device is connected.

The default port for SNMP traps is 162. On Linux systems, you can open ports below 1024 only with superuser privileges. The ports open for SNMP traps are 81 to 32767.

1. Log in to VCF Operations orchestrator as an **administrator**.
2. Navigate to **Library** > **Workflows** and enter the `snmp` and `trap_host_management` tags in the workflow search box.
3. Run the **Set the SNMP trap port** workflow.
4. Enter the port number on which VCF Operations orchestrator listens for SNMP traps.
5. Click **Run**.

The workflow stops the trap host, sets the new port, and starts the trap host again.

Note:
If you upgrade your VCF Operations orchestrator environment, you must run the VCF Operations orchestrator Appliance and the **Set the SNMP trap port** workflow again.

Receiving SNMP Traps

The SNMP plug-in can receive SNMP traps by running a workflow, which waits for a single trap message, or with a policy, which can handle traps continuously. The plug-in supports SNMPv1 and SNMPv2c traps.

Wait for a Trap on an SNMP Device

You can run a workflow that waits to receive an SNMP trap from a specified device.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an SNMP device from the **Inventory** view.

This workflow features a trigger, which stops the run of the workflow and waits for an SNMP trap before continuing. When a trap is received, the workflow run resumes. You can use the workflow as part of more complex workflows, or as a sample that you can customize or extend for a specific need.

1. Navigate to **Library** > **Workflows** and enter the `snmp` tag in the workflow search box.
2. Locate the **Wait for a trap on an SNMP device** workflow and click **Run**.
3. Select the device on which you want to wait for an SNMP trap.
4. Optional: In the **OID** text box, enter the object identifier of a specific trap.

   Note:
   If you leave the text box empty, the workflow run resumes after receiving any trap from the specified SNMP device.
5. Click **Run**.

Set an SNMP Trap Policy

You can set a policy to continuously listen for traps from an SNMP device that is already registered in the plug-in inventory.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an SNMP device from the **Inventory** view.

1. In the VCF Operations orchestrator Client, navigate to **Library** > **Policies**.
2. Open the **SNMP Trap policy** template and click **Run**.
3. Enter a name for the policy that you want to create.
4. Optional: Enter a description for the policy.
5. Select an SNMP device for which to set the policy.
6. Click **Run** to create the policy.
   The VCF Operations orchestrator Client switches to **Run** perspective.
7. On the **Policies** view, right-click the policy that you created and select **Start policy**.

The trap policy starts to listen for SNMP traps.
You can edit the SNMP Trap policy.

Configure an SNMP Trap Host Policy

With the SNMP Trap Host policy listens for SNMP traps from hosts that might not be added as registered SNMP devices.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an SNMP device from the **Inventory** view.

1. In the VCF Operations orchestrator Client, navigate to **Library** > **Policies**.
2. Open the **SNMP Trap Host** policy template and click **Run**.
3. Enter a name for the policy that you want to create.
4. Optional: Enter a description for the policy.
5. Select `Trap Host (Online)` from the inventory tree.
6. Click **Run** to create the policy.
   The Orchestrator client switches to **Run** perspective.

7. Right-click the policy and select **Edit**.
8. In the `Scripting` tab, expand **host** > **OnTrapAll**.
   a) Select a workflow or a script to associate with the policy.
9. Click **Save and close** to apply the edited settings.
10. On the **Policies** view, right-click the policy that you edited and select **Start policy**.

You can edit the SNMP Trap Host policy.

Edit a Trap Policy

You can edit a trap policy to customize it for a specific use case. When you edit a trap policy, you can change its priority and startup settings, and customize the scripting and permissions associated with the policy.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to an SNMP device from the **Inventory** view.

1. In the VCF Operations orchestrator Client, navigate to **Library** > **Policies**.
2. If the policy that you want to edit is running, right-click the policy and select **Stop policy**.
3. Right-click the policy and select **Edit**.
4. From the **General** tab, edit the startup settings, priority, and description of the policy.
5. Optional: From the **Scripting** tab, you can associate a specific workflow or scripting code with the policy, for integration in a more complex scenario.
   You can set the policy to trigger a custom workflow when a trap is received.
6. Optional: From the **Permissions** tab, you can modify the access rights.
   You can give permissions to a user or to a group to start the policy, without giving permissions to edit the policy.
7. Click **Save and close** to apply the edited settings.
8. On the **Policies** view, right-click the policy that you edited and select **Start policy**.

Using the SOAP plug-in

The SOAP plug-in allows you to manage SOAP Web services by providing interaction between VCF Operations orchestrator and SOAP hosts. You can define SOAP services as inventory objects by running configuration workflows, and perform SOAP operations on the defined objects.

The plug-in contains a set of standard workflows related to managing SOAP hosts and invoking SOAP operations. You can also generate custom workflows to automate tasks in a SOAP environment.

Configuring the SOAP plug-in

You must use the VCF Operations orchestrator Client to configure the SOAP plug-in.

The Configuration workflow category contains workflows that allow you to manage SOAP hosts.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `soap` and `configuration` tags in the workflow search box.

Workflow Name	Description
Add a SOAP host	Adds a SOAP host to the plug-in inventory.
Reload plug-in configuration	Refreshes the list of SOAP hosts in the plug-in inventory.
Remove a SOAP host	Removes a SOAP host from the plug-in inventory. **CAUTION:** When you remove a host from the inventory, all workflows generated from it stop working.
Update a SOAP host	Updates a SOAP host in the plug-in inventory.

Workflow Name	Description
Update a SOAP host with an endpoint URL	Updates a SOAP host with a preferred endpoint address. The new endpoint address is used for sending and receiving SOAP messages, instead of the endpoint address defined within the WSDL.

Add a SOAP Host

You can run a workflow to add a SOAP host and configure the host connection parameters.

1. Log in to the VCF Operations orchestrator Client as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `soap` and `configuration` tags in the workflow search box.
3. Locate the **Add a SOAP host** workflow and click **Run**.
4. On the **SOAP host** tab, provide the properties to create a host.
   a) In the **Name** text box, enter the name of the host.
   b) Select whether to provide the WSDL content as text.

Option	Action
Yes	Copy the text in the **WSDL content** text box.
No	Enter the correct path in the **WSDL URI** text box.

   c) In the **Connection timeout** text box, enter the number of seconds within which VCF Operations orchestrator must connect to the SOAP host, otherwise the connection times out.
   d) In the **Request timeout** text box, enter the number of seconds within which a SOAP request must succeed before it times out.
5. On the **Proxy** tab, select whether to use a proxy.

Option	Action
Yes	Provide the proxy address and proxy port.
No	Continue to the next step.

6. On the **Host authentication** tab, select the authentication type.

Option	Description
None	No authentication is required.
Basic	Provides basic access authentication. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator Client retrieves credentials from the user who is logged in.
Digest	Provides digest access authentication that uses encryption. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator Client retrieves credentials from the user who is logged in.
NTLM	Provides NT LAN Manager (NTLM) access authentication within the Window Security Support Provider (SSPI) framework. On the **User credentials** tab, select the session mode.

Option	Description
	• If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator Client retrieves credentials from the user who is logged in. On the **NTLM settings** tab, enter the NTLM domain and NTLM workstation.
Negotiate	Provides Kerberos access authentication. On the **User credentials** tab, select the session mode. • If you select **Shared Session**, provide credentials for the shared session. • If you select **Per User Session**, the VCF Operations orchestrator Client retrieves credentials from the user who is logged in. On the **Negotiate** tab, provide the Kerberos service SPN.

7. Click **Run**.

After the workflow runs successfully, the SOAP host appears in the **Inventory** view.

You can explore the SOAP host objects and run workflows on them from the **Inventory** view.

Generate a new workflow from a SOAP operation

You can create a custom workflow from a SOAP operation.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a SOAP host from the **Inventory** view.

You can integrate custom-generated workflows into high-level workflows. For more information about workflow development, see the *Developing Workflows with VCF Operations orchestrator* guide.

1. Navigate to **Library** > **Workflows** and enter the `soap` tag in the workflow search box.
2. Locate the **Generate a new workflow from a SOAP operation** workflow and click **Run**.
3. Select the SOAP operation from the list of available operations.
4. In the **Name** text box, enter the name of the workflow to generate.
5. Select the workflow folder in which to generate the new workflow.
   You can select any existing folder from the workflow library.
6. Click **Run**.

You can test the generated workflow.

Test a Custom-Generated Workflow

You can run a custom workflow generated from a SOAP operation to get the output parameters of the operation.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a SOAP host from the **Inventory** view.

1. Navigate to **Library** > **Workflows**.
2. Locate the custom workflow and click **Run**.
3. Provide the input parameters that the SOAP operation requires.
4. Click **Run**.
5. Optional: In the **Logs** tab, review the list of available output parameters.

Invoke a SOAP Operation

You can call a SOAP operation directly, without generating a new workflow.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a SOAP host from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `soap` tag in the workflow search box.
2. Locate the **Invoke a SOAP operation** workflow and click **Run**.
3. Select the SOAP operation from the list of available operations.
4. Provide the input parameters that the SOAP operation requires.
5. Click **Run**.
6. Optional: In the **Logs** tab, review the list of available output parameters.

Using the SQL Plug-In

You can use the API that the SQL plug-in provides to implement connectivity to SQL databases and other tabular data sources, such as spreadsheets or flat files.

The SQL plug-in API, which is based on JDBC, provides a call-level API for SQL-based database access. The SQL plug-in also provides sample workflows that demonstrate how to use the API in workflows.

Configuring the SQL plug-in

You can use the workflows included in the SQL plug-in and run them from the VCF Operations orchestrator Client to configure the SQL plug-in and to add, update, or remove a database.

The Configuration workflow category of the SQL plug-in contains workflows that allow you to manage databases and database tables.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `sql` and `configuration` tags in the workflow search box.

When configuring a JDBC connector, you must provide the Java version.

- VCF Operations orchestrator 9.1 uses Java 17.
- VCF Operations orchestrator 9.1.1 uses Java 25.

Workflow Name	Description
Add a database	Adds a database object to the SQL plug-in inventory.
Add tables to a database	Adds database tables to a database in the SQL plug-in inventory.
Remove a database	Removes a database object from the SQL plug-in inventory.
Remove a table from a database	Removes a database table from a database in the SQL plug-in inventory.
Update a database	Updates the configuration of a database object in the SQL plug-in inventory.
Validate a database	Validates a database in the SQL plug-in inventory.

Add a Database

You can run a workflow to add a database to the VCF Operations orchestrator server and configure the host connection parameters.

When you add a database that requires a secure connection, you must import the database TLS certificate. You can import the certificate by running one of the **Import a certificate from URL** workflows.

> **Note:**
> You might need to install a third party connector to use the database if it is not already installed. For more information, go to Adding a JDBC connector for the VCF Operations orchestrator SQL plug-in.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `sql` and `configuration` tags in the workflow search box.
3. Locate the **Add a database** workflow and click **Run**.
4. In the **Name** text box, enter the name of the database.
5. Select the type of the database.
6. In the **Connection URL** text box, enter the address of the database.

Database Type	Syntax
Oracle	`jdbc:oracle:thin:@database_url:port_number:SID`
Microsoft SQL with JTDS driver (with SQL authentication)	`jdbc:jtds:sqlserver://database_url:port_number/database_name`
Microsoft SQL with JTDS driver (with Windows account authentication)	`jdbc:jtds:sqlserver://database_url:port_number/database_name;useNTLMv2=true;domain=domain_name`
PostgreSQL	`jdbc:postgresql://database_url:port_number/database_name`
MySQL	`jdbc:mysql://database_url:port_number/database_name`
Microsoft SQL with Microsoft JDBC	`jdbc:sqlserver://database_url:port_number;databaseName=database_name;user=user_name;password=password` **Note:** To use this database, you must first install the JDBC connector. For more information, go to Adding a JDBC connector for the VCF Operations orchestrator SQL plug-in.

7. On the **User credentials** tab, select the session mode that the plug-in uses to connect to the database.

Option	Description
Shared Session	The plug-in uses shared credentials to connect to the database. You must provide the database credentials for the shared session.
Session Per User	The VCF Operations orchestrator Client retrieves credentials from the user who is logged in. **Note:** To use session per user mode, you must authenticate by using a user name only. Do not use *`domain\user`* or *`user@domain`* for authentication.

8. Click **Run**.

After the workflow runs successfully, the database and all tables that belong to it appear in the **Inventory** view.

Note:

If you install third party drivers, verify that you have uploaded only one driver version per vendor. Uploading multiple driver versions can lead to errors and unpredictable behavior. If you must install a newer driver version, delete the old driver version first.

Add Tables to a Database

You can run a workflow to add tables to a database that is in the SQL plug-in inventory.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a database from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `sql` and `configuration` tags in the workflow search box.
2. Locate the **Add tables to a database** workflow and click **Run**.
3. Select a database to which to add tables.
4. Select the tables that you want to add.
5. Click **Run**.

After the workflow runs successfully, the added database tables appear in the **Inventory** view of the VCF Operations orchestrator Client.

Update a Database

You can run a workflow to update the configuration of a database that is in the SQL plug-in inventory.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `sql` and `configuration` tags in the workflow search box.
3. Locate the **Update a database** workflow and click **Run**.
4. Select the database that you want to update.
5. In the **Name** text box, enter the new name of the database.
   The database appears in the **Inventory** view with the name that you entered.
6. Select the type of the database.
7. In the **Connection URL** text box, enter the new address of the database.
8. On the **User credentials** tab, select the session mode that the plug-in uses to connect to the database.

Option	Description
Shared Session	The plug-in uses shared credentials to connect to the database. You must provide the database credentials for the shared session.
Session Per User	The VCF Operations orchestrator Client retrieves credentials from the user who is logged in. **Note:** To use session per user mode, you must authenticate by using a user name only. Do not use *`domain\user`* or *`user@domain`* for authentication.

9. Click **Run**.

Running the SQL Sample Workflows

You can run the SQL plug-in workflows to perform JDBC operations such as generating a JDBC URL, testing a JDBC connection, and managing rows in JDBC tables. You can also run the SQL plug-in workflows to manage databases and database tables, and to run SQL operations.

Generate a JDBC URL

You can run a workflow from the VCF Operations orchestrator Client to generate a JDBC connection URL.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `jdbc` tag in the workflow search box.
3. Locate the **JDBC URL generator** workflow and click **Run**.
4. On the **General** tab, select the type of database for which to generate a URL.

   Note:
   If you use a Microsoft database, select the **Microsoft** tab and provide the database instance name and database user domain name.
5. Provide the required information to generate a database URL.
   a) Enter a database server name or IP address.
   b) Enter a database name.
   c) Optional: Enter a database port number.
      If you do not specify a port number, the workflow uses a default port number.
   d) Enter a user name to access the database.
   e) Enter a password to access the database.
6. Click **Run**.

Test a JDBC connection

You can run a workflow from the VCF Operations orchestrator Client to test the connection to a database.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
3. Locate the **JDBC connection example** workflow and click **Run**.
4. Provide the required information to test a database connection.
   a) Enter a user name to access the database.
   b) Enter the URL to test.
   c) Enter a password to access the database.
5. Click **Run**.

Create a table by using JDBC

You can run a workflow from the VCF Operations orchestrator Client to create a database.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
3. Locate the **JDBC create table example** workflow and click **Run**.
4. Provide the required information, and click **Next**.
   a) Type a password to access the database.
   b) Type a database connection URL.
   c) Type a user name to access the database.
5. Enter an SQL create statement.
   Example syntax is:
   ```
   CREATE TABLE "table_name"
   ("column1" "data_type_for_column1",
   "column2" "data_type_for_column2")
   ```
6. Click **Run**.

Insert a Row into a JDBC Table

You can run a workflow from the VCF Operations orchestrator Client to test the insertion of a row into a JDBC table.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
3. Locate the **JDBC insert into table example** workflow and click **Run**.
4. On the **Database connection** tab, provide the required information.
   a) Type a database connection URL.
   b) Type a user name to access the database.
   c) Type a password to access the database.
5. On the **SQL statement** tab, enter an SQL insert statement similar to the following example.
   ```
   INSERT INTO "table_name" ("column1", "column2")
   VALUES ("value1", "value2")
   ```
6. On the **Values to insert** tab, enter the values to insert into the row.
7. Click **Run**.

Select Rows from a JDBC Table

You can run a workflow from the VCF Operations orchestrator Client to select rows from a JDBC table.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
3. Locate the **JDBC select from table example** workflow and click **Run**.
4. On the **Database connection tab**, provide the required information.
   a) Type a database connection URL.
   b) Type a user name to access the database.
   c) Type a password to access the database.
5. On the **SQL statement** tab, type an SQL select statement similar to the following example.
   Example syntax is:
   ```
   SELECT * FROM "table_name"
   ```

6. Click **Run**.

Delete an Entry from a JDBC Table

You can run a workflow from the VCF Operations orchestrator Client to test the deletion of an entry from a JDBC table.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
2. Locate the **JDBC delete entry from table example** workflow and click **Run**.
3. Provide the required information.
   a) Enter the first name of the user entry to be deleted.
   b) Type a user name to access the database.
   c) Enter a JDBC connection URL.
   d) Enter the last name of the user entry to be deleted.
   e) Type a password to access the database.
4. Enter an SQL delete statement similar to the following example syntax.
   ```
   DELETE FROM "table_name" where ("column1" = ?, "column2" = ?)
   ```
5. Click **Run**.

Delete All Entries from a JDBC Table

You can run a workflow from the VCF Operations orchestrator Client to delete all entries from a JDBC table.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
2. Locate the **JDBC delete all from table example** workflow and click **Run**.
3. Provide the required information.
   a) Type a database connection URL.
   b) Type a user name to access the database.
   c) Type a password to access the database.
4. Type an SQL delete statement similar to the following example syntax.
   ```
   DELETE FROM "table_name"
   ```
5. Click **Run**.

Drop a JDBC Table

You can run a workflow from the VCF Operations orchestrator Client to test the dropping of a JDBC table.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
2. Locate the **JDBC drop table example** workflow and click **Run**.
3. Provide the required information.
   a) Type a password to access the database.
   b) Type a database connection URL.
   c) Type a user name to access the database.
4. Enter an SQL drop statement similar to the following example syntax.
   ```
   DROP TABLE "table_name"
   ```

5. Click **Run**.

Run a Complete JDBC Cycle

You can run a workflow from the VCF Operations orchestrator Client to test all JDBC example workflows in one full cycle.

Verify that the user account you are logged in with has the necessary permissions to run JDBC workflows.

1. Navigate to **Library** > **Workflows** and enter the `jdbc_examples` tag in the workflow search box.
2. Locate the **Full JDBC cycle example** workflow and click **Run**.
3. Provide the required information.
   a) Type a database connection URL.
   b) Type a user name to access the database.
   c) Type a password to access the database.
4. Enter the values to be used as entries in the database.
5. Click **Run**.

Running SQL Operations

You can use the SQL workflows to run SQL operations.

To access the SQL operations workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows** and enter the `sql` tag in the workflow search box.

Workflow Name	Description
Execute a custom query on a database	Runs a custom query on a specified database and returns the number of affected rows. You can run the workflow to update, delete, insert, and write queries.
Generate CRUD workflows for a table	Generates Create, Read, Update, and Delete workflows for a particular table.
Read a custom query from a database	Runs a custom query on a specified database and returns the result in an array of properties. You can run the workflow to select and read queries.

Generate CRUD Workflows for a Table

You can run a workflow to generate Create, Read, Update, and Delete workflows for a particular table.

- Verify that you are logged in to the VCF Operations orchestrator Client as an administrator.
- Verify that you have a connection to a database from the **Inventory** view.

1. Navigate to **Library** > **Workflows** and enter the `sql` tag in the workflow search box.
2. Locate the **Generate CRUD workflows for a table** workflow and click **Run**.
3. Select a table for which to generate the workflows.
4. Select the workflow folder in which to generate the workflows.
5. Select whether to overwrite any existing workflows.

Option	Description
Yes	The generated workflows overwrite existing workflows with the same name.
No	New workflows are not generated if workflows with the same name exist in the folder.

6. Optional: On the **Select read-only columns** tab, select columns that should not be populated.
   You cannot edit the selected columns with the generated CRUD workflows.
7. Click **Run**.

After the workflow runs successfully, the CRUD workflows appear in the selected workflow folder.

You can run the generated workflows on the selected database table.

Adding a JDBC connector for the VCF Operations orchestrator SQL plug-in

This example demonstrates how you can add a MySQL connector for the VCF Operations orchestrator SQL plug-in.

The VCF Operations orchestrator SQL plug-in supports only certain database types. Before adding a MySQL connector, verify that you use one of the following database types:

- Oracle
- Microsoft SQL Server
- PostgreSQL
- MySQL

1. Add the MySQL connector.jar file to the VCF Operations orchestrator Appliance.

   Note:
   For clustered VCF Operations orchestrator deployments, perform this operation on the appliances of all the nodes.

   a) Log in to the VCF Operations orchestrator appliance command line over SSH as **root**.
   b) Navigate to the `/data/vco/var/run/vco` directory.

      ```
      cd /data/vco/var/run/vco
      ```

   c) Create a `plugins/SQL/lib/` directory.

      ```
      mkdir -p plugins/SQL/lib/
      ```

   d) Copy your MySQL connector.jar file from your local machine to the `/data/vco/var/run/vco/plugins/SQL/lib/` directory by running a secure copy (SCP) command.

      ```
      scp ~/local_machine_dir/your_mysql_connector.jar root@orchestrator_FQDN_or_IP:/data/vco/var/run/vco/plugins/SQL/lib/
      ```

      Note:
      You can also use alternative methods for copying your connector.jar file to the VCF Operations orchestrator Appliance, such as PSCP.

2. Restart the VCF Operations orchestrator server.

Using the SSH Plug-In for VCF Operations Orchestrator

You can use the SSH plug-in workflows to run SSH commands on a remote host that supports SSH and transfer files between a VCF Operations orchestrator server and a remote host through a secure connection.

Specify Ciphers Used by the SSH Plug-In

Ciphers and key exchange algorithms that are provided by the SSH plug-in depend on the release version of VCF Operations orchestrator. You must configure the remote SSH host with FIPS-compatible algorithms. You specify ciphers for the SSH plug-in by adding a system property in VCF Operations orchestrator.

1. Log in to VCF Operations orchestrator and navigate to **System Settings**.
2. Click **Configuration Properties**.
3. Click **Add Property**.
4. Under **Name**, enter `com.vmware.o11n.ssh.cipher`.
5. Under **Value**, enter a comma-separated list of ciphers, for example, `aes256-ctr,aes256-gcm@openssh.com`.

The default list of available ciphers is `aes256-ctr,aes192-ctr,aes128-ctr,aes256-cbc,aes192-cbc,aes128-cbc,3des-ctr,3des-cbc,arcfour256,arcfour128,arcfour,blowfish-cbc`.

6. Enter a description for the new system property.
7. Click **Add**.
8. To finish setting the configuration property, click **Save**.

Configuring the SSH Plug-In for VCF Operations Orchestrator

You can run the SSH plug-in configuration workflows to manage the connections between VCF Operations orchestrator and SSH hosts.

To access these workflows in the VCF Operations orchestrator, navigate to **Library** > **Workflows** and enter the `ssh` and `configuration` tags in the workflow search box.

Workflow Name	Description
Add a Root Folder to SSH Host	Adds a root folder to an existing connection to an SSH host.
Add SSH Host	Adds a connection to an SSH host to the existing configuration.
Remove a Root Folder from SSH Host	Removes a root folder from an existing connection to an SSH host.
Remove SSH Host	Removes an existing connection to an SSH host from the existing configuration.
Update SSH Host	Updates an existing connection to an SSH host.

Add an SSH Host to VCF Operations Orchestrator

You can set up the SSH plug-in to ensure encrypted connections.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `ssh` and `configuration` tags in the workflow search box.
3. Locate the **Add SSH host** workflow and click **Run**.
4. On the **General Information** tab, in the **Host name** text box, enter the name of the host that you want to access with SSH through VCF Operations orchestrator.
5. Enter the target port. The default SSH port is 22.

   The host is added to the list of SSH connections.
6. Optional: Configure an entry path on the server.
   a) Click **New root folder**.
   b) Enter the new path and click **Insert value**.
7. On the **Authentication** tab, enter the user name for a user who has the necessary permissions to run SSH commands.
8. Select the authentication type.

Option	Action
Yes	To use password authentication, enter a password.
No	To use key authentication, enter the path to the private key and the private key passphrase.

9. Click **Run**.

The SSH host is available in the **Inventory** view of the VCF Operations orchestrator Client.

Running the SSH Plug-In Sample Workflows in VCF Operations Orchestrator

You can run the SSH plug-in sample workflows from the VCF Operations orchestrator Client to test the connection between the VCF Operations orchestrator server and the SSH host.

Generate a Key Pair in VCF Operations Orchestrator

You can generate a key pair by running a workflow in the VCF Operations orchestrator Client. Alternatively, you can use a key pair that you generate in third-party tools. You can use the key pair to connect to an SSH host without a password.

A key pair consists of a public key and a private key. VCF Operations orchestrator can use the private key to connect to the public key on an SSH host.

Generate a Key Pair in VCF Operations Orchestrator

You can run a workflow from the VCF Operations orchestrator Client to generate a key pair.

Verify that the user account you are logged in with has the necessary permissions to run SSH workflows.

> **CAUTION:**
> All VCF Operations orchestrator users with the right set of privileges can read, use, and overwrite your private key.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `ssh` tag in the workflow search box.
3. Locate the **Generate key pair** workflow and click **Run**.
4. Provide the required information.
   a) Select the key type and the key size.
      Depending on which key type you select during the key-pair generation, you might have to change the SSH service endpoint configuration to enable the type of certificate generated.

Key Type	Key Sizes
RSA	• 1024 • 2048 • 3072 • 4096 • 8192
DSA	• 1024 • 2048 • 3072
ECDSA	• 256 • 384 • 521

   b) Optional: Enter a passphrase.
      You can use a passphrase to improve security. You can change the passphrase later.
   c) Optional: Enter a comment.
5. Click **Run**.

> **Warning:**
> If a key pair exists, the new key pair overwrites it.

Use an Externally Generated Key Pair

You can generate public or private key pairs using external tools such as OpenSSH or OpenSSL.

When generating RSA, DSA, and ECDSA private keys, the key must be in the PEM format.

1. To generate key pairs with an external tool, you must pass specific parameters to the commands of each tool.

Tool	Parameters
OpenSSH	Use the `-m pem` parameter. For example, you can run the following command: `ssh-keygen -m pem -t ecdsa -b {256, 384, 521} -f vco_key`

OpenSSL	The default format is PEM, so no additional parameters are needed.
	For example, you can run the following command:
	`openssl ecparam -name {prime256v1, secp384r1, secp521r1} -genkey -noout -out vco_key`
	Using OpenSSH, extract the corresponding public key from the private key.
	`ssh-keygen -f vco_key -y > vco_key.pub`

2. Transfer the externally generated key pairs to `/data/vco/usr/lib/vco/app-server/conf` in VCF Operations orchestrator.

Change the Key Pair Passphrase

You can run a workflow from the VCF Operations orchestrator Client to change the passphrase for the key pair that you generated most recently.

Verify that the user account you are logged in with has the necessary permissions to run SSH workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `ssh` tag in the workflow search box.
3. Locate the **Change key pair passphrase** workflow and click **Run**.
4. On the **Change passphrase** tab, reset the key pair passphrase.
   a) Enter the current passphrase.
   b) Enter the new passphrase.
5. Click **Run**.

Register a VCF Operations Orchestrator Public Key on an SSH Host

You can use a public key instead of a password. To register a VCF Operations orchestrator public key on an SSH host, you can run a workflow from the VCF Operations orchestrator client.

Verify that the user account you are logged in with has the necessary permissions to run SSH workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `ssh` tag in the workflow search box.
3. Locate the **Register vCO public key on host** workflow and click **Run**.
4. On the **Register VS-O on Host** tab, provide the name of the SSH host, and the user name and password to log in to this host.

   Note:
   You must provide credentials that are registered on the SSH host.
5. Click **Run**.

You can use public key authentication instead of password authentication when you connect to the SSH host as the registered user.

Run an SSH Command from VCF Operations Orchestrator

You can run a workflow from VCF Operations orchestrator to run SSH commands on a remote SSH server.

Verify that the user account you are logged in with has the necessary permissions to run SSH workflows.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `ssh` tag in the workflow search box.
3. Locate the **Run SSH command** workflow and click **Run**.
4. On the **Host selection** tab, enter an SSH host name or IP address.
5. On the **Command** tab, enter an SSH command to run.

   Note:
   The default SSH command is `uptime` which shows how long the server has been active and the user load for that period.
6. On the **Encoding** tab, specify the encoding method.

   Leave this field empty to use the default system encoding.
7. On the **Authentication** tab, enter a user name.
8. Optional: Select the check box to use password authentication.

   Note:
   The default option is to use key file authentication.
9. Enter a password if the authentication method requires a password. Otherwise, enter the path to the private key and enter the passphrase for the private key.
10. Click **Run**.

Copy a File from an SSH Host to VCF Operations Orchestrator

You can run a workflow in VCF Operations orchestrator to copy files from an SSH host to the VCF Operations orchestrator server.

Verify that the user account you are logged in with has the necessary permissions to run SSH workflows.

Note:
VCF Operations orchestrator must have explicit write permissions to write in folders.

The SSH plug-in uses the Java JCraft library, which implements SFTP. The SCP get command workflow transfers files by using SFTP.

1. Log in to VCF Operations orchestrator.
2. Navigate to **Library** > **Workflows** and enter the `ssh` tag in the workflow search box.
3. Locate the **SCP get command** workflow and click **Run**.
4. On the **Host** tab, enter the source host information.
   a) Enter an SSH host name or IP address.
   b) Enter the SSH user name and password.
5. On the **File** tab, enter the file information.
   a) Enter the path to the file to get from the remote SSH host.
   b) Enter the path to the directory on the VCF Operations orchestrator server into which to copy the file.
6. Click **Run**.

Copy a File from VCF Operations Orchestrator to an SSH Host

You can run a workflow in VCF Operations orchestrator to copy files from the VCF Operations orchestrator server to an SSH host.

Verify that the user account you are logged in with has the necessary permissions to run SSH workflows.

The SSH plug-in uses the Java JCraft library, which implements SFTP. The SCP put command workflow transfers files by using SFTP.

1. Navigate to **Library** > **Workflows** and enter the `ssh` tag in the workflow search box.
2. Locate the **SCP put command** workflow and click **Run**.
3. On the **Host** tab, enter the source host information.
   a) Enter an SSH host name or IP address.
   b) Enter the SSH user name and password.
4. On the **File** tab, enter the file information.
   a) Enter the path to the file that you want to copy from the local Orchestrator server to the remote SSH host.
   b) Enter the path to the directory on the remote SSH host into which to copy the file.
5. Click **Run**.

Using the vCenter plug-in

You can use the vCenter plug-in to manage multiple vCenter instances. You can create workflows that use the vCenter plug-in API to automate tasks in your vCenter environment.

vCenter Plug-In Scripting API

The vCenter plug-in maps the vCenter API to the JavaScript that you can use in workflows. The vCenter scripting API contains classes, with their respective attributes, methods, and constructors that allow interaction between VCF Operations orchestrator and vCenter.

The vCenter plug-in supports registering vCenter 6.5, 6.7, 7.0 instances as endpoints. Based on the vCenter version that you register, the plug-in exposes vSphere 6.5 and 6.7 API as SDK, and vSphere 7.0 API as scripting SDK. For information about vCenter scripting objects, navigate to the API Explorer in VCF Operations orchestrator.

The vCenter plug-in provides a library of standard workflows that automate vCenter operations. For example, you can run workflows to create, clone, migrate, or delete virtual machines. The plug-in also provides actions that perform individual vCenter tasks that you can include in workflows. You can use the API to develop custom workflows.

Most vCenter plug-in workflows communicate only with the vCenter. However, some guest operations workflows require communication with the ESXi host managed by vCenter. Before you run these workflows, you must import the ESXi host certificate by running one of the **Import a certificate from URL** workflows.

The vCenter plug-in includes the Policy-Based Management (PBM) and the Storage Monitoring Service (SMS) APIs as scripting objects in the VCF Operations orchestrator scripting API. The Storage Policy-Based Management policies and components appear in the **Inventory** page of the VCF Operations orchestrator.

Using the vCenter Plug-In Inventory

The vCenter plug-in exposes all objects of the connected vCenter instances in the Inventory view.

To display the workflows that are available for a vCenter inventory object, navigate to **Administration** > **Inventory** > **vSphere vCenter Plug-in** in VCF Operations orchestrator.

Configuring the vCenter Server Plug-In in VCF Operations Orchestrator

Before you can manage the objects in your vSphere inventory and run workflows on those objects in VCF Operations orchestrator, you must first connect VCF Operations orchestrator and the vCenter instances you want to orchestrate.

You configure the vCenter plug-in by running the vCenter configuration workflows from the VCF Operations orchestrator.

Configure the Connection to a vCenter Instance

To manage the inventory of a vSphere instance, you configure the connections to vCenter instances by running the vCenter configuration workflows in VCF Operations orchestrator.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `vcenter` and `configuration` tags in the workflow search box.
3. Locate the **Add a vCenter Server instance** workflow and click **Run**.
4. On the **Set the vCenter Server instance properties** tab, enter the IP address or the host name of the machine on which the vCenter instance you want to add is installed.

   Note:
   The host name that you enter is case-sensitive.
5. Retain the default port value, `443`.
6. Retain the default location of the SDK to use to connect to your vCenter instance.
7. Select whether you want to ignore certificate warnings for the vCenter instances that you want to add.

   If you decide to ignore certificate warnings, the vCenter instance certificate is accepted silently and the certificate is added to the trusted store.
8. On the **Set the connection properties** tab, select whether you want to use a session per user method to manage user access on the vCenter system.

Option	Description
Share a unique session	Select in the following cases: • VCF Operations orchestrator uses VCF Automation as an authentication provider. • VCF Operations orchestrator uses vCenter an authentication provider but you want to connect to a different vCenter instance. Allows VCF Operations orchestrator to create only one connection to vCenter. In the **User name** and **Password** text boxes, enter the credentials for VCF Operations orchestrator to use to establish the connection to the vCenter host. The user that you select must be a valid user with privileges to manage vCenter extensions and a set of custom defined privileges. VCF Operations orchestrator uses these credentials

Option	Description
	to monitor the VirtualCenter Web service, typically to run VCF Operations orchestrator system workflows.
Session per user	Select if VCF Operations orchestrator uses the same vCenter as an authentication provider. Creates a session to vCenter. **Note:** This action might rapidly use CPU, memory, and bandwidth.

The user account that you select is also used by the policy engine to collect statistical and other data. If the user that you select does not have enough privileges, the policy engine cannot access the necessary parts of the vCenter inventory and cannot collect the necessary data.

9. Optional: Enter the user domain.

   You must specify the user domain name only when you select to use a shared session.

   Note:
   Fill this text box if Session per user is selected.

10. Optional: On the **Additional Endpoints** tab, enter the URLs for the vSphere storage management endpoints.

    You can configure the Policy-Based Management (PBM) endpoint, the Storage Monitoring Service (SMS) endpoint, or both.

    a) Enter the URL of PBM endpoint. The default vSphere 6.5 PBM endpoint URL is `https://vSphereHostName:443/pbm`.
    b) Enter the URL of SMS endpoint. The default vSphere 6.5 SMS endpoint URL is `https://vSphereHostName:443/sms/sdk`.

11. Click **Run**.

After the workflow runs successfully, the vCenter instance and all vSphere objects that belong to it appear in the **Inventory** view.

Performance Considerations for Querying

With the vCenter plug-in for VCF Operations orchestrator, you can query the vCenter inventory for specific objects.

Querying Methods

For querying, you can either use the `vCSearchIndex` managed object, or the object finder methods that are included in the plug-in inventory, such as `getAllDatastores()`, `getAllVirtualMachines()`, `findAllForType()`, and others.

Performance

By default, both methods return the queried objects without including any of their properties, unless you specify a set of properties as an argument for the method parameters in the search query.

Note:
You must always use query expressions with the `getAll...()` and `findAll...()` finder objects to prevent the VCF Operations orchestrator Client from filtering large sets of returned objects, which might affect the overall performance of the VCF Operations orchestrator server.

You can use two types of expressions for querying the vCenter inventory.

Type of Expression	Description
Name expressions	You can specify a name as an argument for a query parameter. **Note:** The objects are filtered by the specified name argument according to the name of the plug-in object as it is appears in the vCenter plug-in inventory.
XPath expressions	You can use expressions based on the XPath query language. For more information, see Using XPath Expressions with the Plug-In.

When you invoke a vCenter inventory object with custom properties, each reference to this object, in a workflow or an action, sends a query to vCenter, which generates a noticeable performance overhead. To optimize performance and avoid serializing and deserializing the object multiple times within a workflow run, it is best to use a shared resource to store the object, instead of storing it as a workflow attribute, an input, or an output parameter. Such shared resource can be a configuration element or a resource element.

Using XPath Expressions with the vCenter Plug-In

You can use the finder methods in the vCenter plug-in to query for vCenter inventory objects. You can use XPath expressions to define search parameters.

The vCenter plug-in includes a set of object finder methods such as `getAllDatastores()`, `getAllResourcePools()`, `findAllForType()`. You can use these methods to access the inventories of the vCenter instances that are connected to your VCF Operations orchestrator server and search for objects by ID, name, or other properties.

For performance reasons, the finder methods do not return any properties for the queried objects, unless you specify a set of properties in the search query.

For example XPath queries, see XPath Syntax.

Important:

The queries based on XPath expressions might impact the VCF Operations orchestrator performance because the finder method returns all objects of a given type on the vCenter side and the query filters are applied on the vCenter plug-in side.

Using the VCF Operations orchestrator Plug-In for vSphere Web Client

You can use the VCF Operations orchestrator plug-in for vSphere Web Client to perform operations on certain VCF Operations orchestrator content from your vSphere Client.

The content that the plug-in exposes in the vSphere Client includes workflows, workflow runs, scheduled workflows, and workflows waiting for input. In the vSphere Client, users can see workflow details, run or schedule workflows, and manage workflow runs.

Context Actions

You can add VCF Operations orchestrator workflows as context actions in the vSphere Web Client. With context actions, you can map a particular workflow to a specific vSphere item and run or schedule the workflow directly from the vSphere inventory.

When you use the VCF Operations orchestrator plug-in for vSphere Web Client for the first time, a configuration element is created in VCF Operations orchestrator. You must not change or edit the location and any configuration items in that location because they are managed by the VCF Operations orchestrator plug-in for vSphere Web Client service.

Group Assignments

Administrators can use group assignments to assign context actions to specific VCF Operations orchestrator groups. Users in these groups can then use context actions in vSphere. Users can run only the workflows that are assigned to their group.

When a user logs into the vSphere Client, they have the same views and/or permissions in the VCF Operations orchestrator plug-in for vSphere Web Client as in VCF Operations orchestrator. For example, if a user has the Viewer role in VCF Operations orchestrator, they can't run workflows in the vSphere Client.

To use group assignments, you must have at least one group defined in VCF Operations orchestrator.

When you assign context actions to a group, all of the mapped workflows are added as group items in the assigned group, along with the configuration item.

Functional Requirements for the Plug-In

The VCF Operations orchestrator plug-in for vSphere Web Client supports the following:

- Standalone VCF Operations orchestrator environments only. The plug-in is not supported for instances of VCF Operations orchestrator that are embedded in VCF Automation.
- The VCF Operations orchestrator plug-in for vSphere Web Client plug-in is available only with VCF Operations orchestrator. To upgrade the plug-in, you must upgrade VCF Operations orchestrator.
- vSphere 8.0 U3 and later.
- Maximum one VCF Operations orchestrator plug-in for vSphere Web Client can be registered per vCenter Server.
- In vSphere deployments with multiple vCenter instances (in Enhanced Linked Mode), you can register a single plug-in with multiple vCenter instances (maximum one plug-in installation per vCenter), or you can register multiple plug-ins with multiple vCenter servers.
- Before you can use the plug-in, you must configure the plug-in by enabling the service on the VCF Operations orchestrator appliance. See Configuring the.

Configure the VMware Cloud Foundation Operations orchestrator Plug-in for vSphere Web Client

As an administrator, you configure the VCF Operations orchestrator plug-in for vSphere Web Client.

- Verify that your VCF Operations orchestrator instance authenticates with vSphere. The authentication provider must use the vCenter FQDN as the target vCenter instance FQDN. For more information on configuring the authentication provider, see Configure VCF Operations Orchestrator for vSphere in the *Deployment, Convergence, and Upgrade* documentation.
- Add the target vCenter Server as an endpoint in VCF Operations orchestrator by running the **Add a vCenter Server Instance** workflow. For instructions on how to run the workflow, see Configure the Connection to a vCenter Instance.

1. Activate the VCF Operations orchestrator plug-in for vSphere Web Client.
   a) Log in to the VCF Operations orchestrator appliance over SSH.
   b) Run the following command.

```
vracli cap vcoin --enable
```

   You can also add the following optional parameters after the `capabilities` command.

Parameter	Description
`-h, --help`	Get information about optional commands that you can use.
`--enable`	Activate this capability.
`--disable`	Deactivate this capability.
`--replay`	Ignore the current state and potentially replay any state validations.
`-f, --force`	Force set a value for this capability.

c) Redeploy the VCF Operations orchestrator appliance by running the `/opt/script/deploy.sh` command.
Running this command restarts VCF Operations orchestrator.

2. Register the VCF Operations orchestrator plug-in for vSphere Web Client extension in the vSphere Web Client.

Note:

Only a single plug-in can be registered with each vCenter instance. The same plug-in can be registered with multiple vCenter instances if they are in the same SSO domain.

If the certificate of the VCF Operations orchestrator appliance on which the plug-in is running is changed, the extension registration must be updated by running the registration workflow again.

a) Navigate to **Library** > **Workflows** and enter the `vc` tag in the workflow search box.
b) Locate the **Register VCF Operations Orchestrator Plug-in for vSphere Web Client Extension** workflow and click **Run**.
c) In the **SDK Connection** text box, enter the URL of the vCenter endpoint.
d) In the **Address** text box, enter the URL of the plug-in.
e) Click **Run**.

After the VCF Operations orchestrator plug-in for vSphere Web Client plug-in extension is registered, verify that the plug-in is visible in the vSphere Client. You might have to refresh your browser. For troubleshooting information, see the VMware vSphere documentation.

To manage the VCF Operations orchestrator plug-in for vSphere Web Client, use the following workflows.

Workflow	Description
Advanced Options to Register VCF Operations Orchestrator Plug-in for vSphere Web Client Extension	This workflow registers an extension by manually providing the extension version and endpoint certificate.
Unregister VCF Operations Orchestrator Plug-in for vSphere Web Client Extension	This workflow unregisters any extensions from the selected vCenter endpoint.
List the VCF Operations Orchestrator Plug-in for vSphere Web Client Extensions	This workflow lists details on any extensions that are registered with the selected vCenter server.

Create a context action

The vSphere Client provides an inventory of vSphere items that are available for management. You can define context actions for specific vSphere inventory objects and run VCF Operations orchestrator workflows on those objects from the vSphere inventory.

1. Log in to the vSphere Web Client as a user with enough privileges to manage VMs and having an **Administrator** role in VCF Operations orchestrator.
2. On the VCF Operations orchestrator home page, click the **Context Actions** tab.
3. Click **Add**.
4. In the search box, enter the name of the workflow that you want to add as a context action.
5. Select the workflow in the results table.
6. In the **Type of object** dropdown menu, select which vSphere object you want to associate with the context action, such as a vCenter server, for example.
7. Click **Assign**.
You added a VCF Operations orchestrator workflow as a context action.

8. Run the workflow on the vCenter server that you associated with the action.
   a) In the vSphere Client, right click the vCenter server.
   b) Select **Orchestrator** > **Run Workflow**.
   c) Select the context action workflow to run on the vCenter server.

      The workflow input form appears. If the workflow has as an input parameter of the particular inventory type over which you initiated the context action, the form is prepopulated.
   d) Click **Run**.

Using the vCloud Suite API (vAPI) Plug-In

The vCloud Suite API plug-in provides the ability to consume API exposed by any vCloud Suite API provider. The vCloud Suite API provides a service-oriented architecture for accessing resources in the virtual environment by issuing requests to vCenter, through the vCloud Suite Endpoint.

The plug-in contains a set of standard workflows and example workflows. You can also create custom workflows that implement the plug-in to automate tasks in your virtual environment. For information about vCloud Suite API, see *VMware vCloud Suite SDKs Programming Guide*.

Access the vCloud Suite API Plug-In API

VCF Operations orchestrator provides an API Explorer to allow you to search the vCloud Suite API plug-in API and see the documentation for JavaScript objects that you can use in scripted elements.

To access the API Explorer from the VCF Operations orchestrator Client, click **API Explorer** in the VCF Operations orchestrator Client navigation pane.

To access the API Explorer from the **Scripting** tabs of the workflow, policy, and action editors, click **Search API** on the left.

You can copy code from API elements and paste it into scripting boxes. For more information about API scripting, see *Developing with VCF Operations orchestrator*.

Configuring the vCloud Suite API Plug-In

You can configure vCloud Suite API by running the configuration workflows included in the plug-in.

Import a vCloud Suite API metamodel

The vCloud Suite API plug-in discovers vCloud Suite API services dynamically by querying a vCloud Suite API provider metadata service. vCloud Suite API providers which are not exposing metadata service are not supported.

You must import a vCloud Suite API metamodel and add endpoints afterwards.

You can import up to 20 metamodels. The metamodels you import must have the same version as your vCenter or later. You don't need to import separate metamodels for each endpoint. One metamodel per vCenter is enough for all endpoints with that vCenter version.

1. Navigate to **Library** > **Workflows** and enter the `vapi` tag in the workflow search box.
2. Locate the **Import vAPI metamodel** workflow and click **Run**.
3. In the **vAPI endpoint URL** text box, enter the URL of your vCloud Suite API endpoint.

   `https://vCloud_Suite_API_provider_FQDN/api`
4. Select whether to use a secure protocol connection.

Option	Description
No	Import the vCloud Suite API metamodel, without using a secure protocol connection.
Yes	To import the vCloud Suite API metamodel with secure protocol connection: 1. Select whether to ignore certificate warnings and accept the vCloud Suite endpoint automatically.

Option	Description
	2. Provide the user credentials to authenticate with the vCloud Suite endpoint.

5. Select whether to add a vAPI endpoint using the same credentials.
6. Click **Run**.

Add a vCloud Suite API endpoint

Add a vCloud Suite API endpoint.

Import at least one vCloud Suite API metamodel with the same vCenter version or later.

1. Navigate to **Library** > **Workflows** and enter the `vapi` tag in the workflow search box.
2. Locate the **Add vAPI endpoint** workflow and click **Run**.
3. In the **vAPI endpoint URL** text box, enter the URL of your vCloud Suite API endpoint.

   `https://vCloud_Suite_API_provider_FQDN/api`
4. Choose whether to use a secure protocol connection:

Option	Description
No	Import the vCloud Suite API metamodel, without using a secure protocol connection.
Yes	To import the vCloud Suite API metamodel with secure protocol connection: 1. Choose whether to ignore certificate warnings and accept the vCloud Suite endpoint automatically. 2. Provide the user credentials to authenticate with the vCloud Suite endpoint.

5. Click **Run**.

Using the VCF Operations orchestrator Plug-In for VCF Automation for VM Apps

The VMware Cloud Foundation Operations orchestrator™ Plug-in for VMware Cloud Foundation Automation™ for VM Apps integrates VCF Operations orchestrator with VCF Automation. With the plug-in, you can run VCF Operations orchestrator workflows from your VCF Automation instance. You can use the workflows that are provided with the plug-in to deploy and manage VCF Automation resources in VM Apps organizations.

> **Important:**
>
> The VMware Cloud Foundation Operations orchestrator™ Plug-in for VMware Cloud Foundation Automation™ for VM Apps is not supported in VCF Automation for All Apps organizations. In All Apps organizations, use the VMware Cloud Foundation Operations orchestrator™ Plug-in for VMware Cloud Foundation Automation™ for All Apps instead. For more information, see Using the VCF Operations orchestrator Plug-In for VCF Automation for All Apps.

Role of VCF Operations orchestrator with the Automation plug-in for VM Apps

VCF Operations orchestrator powers the Automation plug-in for VM Apps. You use VCF Operations orchestrator to run and create workflows and access the Automation plug-in API.

You can use either the embedded VCF Operations orchestrator instance in your VCF Automation deployment or an external VCF Operations orchestrator server.

Authentication for the Automation plug-in for VM Apps

The VCF Operations orchestrator Plug-In for VCF Automation for VM Apps supports multiple authentication methods for connecting to VCF Automation. The authentication method that you choose depends on your deployment type, your identity provider configuration, and your security requirements.

Method	Description	Requires vIDM?	When to use
vIDM user credentials	Uses a user name and password for a VMware Identity Manager (vIDM) user to obtain an access token.	Requires vIDM as the identity provider for the VM Apps organization.	Use this method for deployments that upgrade from VMware Aria Automation 8.x, where vIDM is already configured. To use the plug-in in shared session in new VCF Automation deployments, you must configure vIDM as the identity provider. See Broadcom knowledge base article 401216.
API token	Uses an API token that a service account generates to obtain an access token.	Does not require vIDM as the identity provider for the VM Apps organization. Supported in VCF Automation 9.1.1 and later.	Use API token authentication for new VCF Automation deployments where vIDM is not configured or where you do not want to use vIDM. For information about creating a service account and generating an API token, see Create a Service Account and Get a Refresh Token in VCF Automation for VM Apps.
Per-user session	Uses the credentials of the user who is currently logged in to VCF Operations orchestrator.	Does not require vIDM or a service account.	Use per-user session authentication when each user runs workflows by using their own credentials. This method is not suitable for shared or automated workflows that run without user interaction.

Using the default Automation plug-in workflows and actions

The Automation plug-in provides out-of-the-box workflows for common tasks, such as host configuration workflows and infrastructure workflows. For a full list of available workflows, navigate to **Library** > **WorkflowsvRealize Automation 8.x and Cloud Services**.

The plug-in library also contains predefined actions that you can use to build you own custom workflows. To access these actions, navigate to **Library** > **Actions**, and enter `com.vmware.library.vra` in the action search box.

Using the Automation plug-in inventory

The VCF Operations orchestrator inventory supports objects for hosts, cloud accounts, cloud zones, disks, machines, machine disks and snapshots, networks, projects, and other entities that are required as lookups for create/update workflows, such as tags, data collectors, regions, NSX-T, and NSX-V cloud accounts.

To display all available inventory objects, navigate to **Administration** > **Inventory** > **VCF Automation for VM Apps**.

Accessing the Automation plug-in API

In the VCF Operations orchestrator API Explorer, you can search the Automation plug-in API and see the documentation for JavaScript objects that you can use in scripted elements. You can copy code from API elements and paste it into scripting boxes.

In the VCF Operations orchestrator API Explorer, click the **VRA** module in the left pane to expand the hierarchical list of Automation plug-in API scripting objects.

To access the API reference for your VCF Automation version, go to `https://VMware-Aria-Automation-hostname/automation-ui/api-docs`.

For up-to-date VCF Automation API documentation, see the VMware Aria Automation 8.18 API Programming Guide.

Configuring VCF Automation hosts

You can manage VCF Automation hosts by running the default workflows provided with the plug-in.

To access these workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows**, and enter the `vra` or `vra-cloud` tag in the workflow search box.

Workflow	Description
Add vRA Host	Adds a VCF Automation on-prem host to the plug-in inventory. See Add a vRealize Automation host. The workflow accepts the `organization ID`, `organization name`, `refresh token` or `API token`, and `API version` as input parameters. The workflow output includes the token expiry time. **Note:** If you use the plug-in in an embedded VCF Operations orchestrator instance, a default host is created in the plug-in inventory. The logged-in user session is used to execute operations on the default host. When you perform operations on the default host, the current user's token is used to execute the operation.
Update vRA Host	Updates a VCF Automation host in the plug-in inventory. The workflow accepts the o `organization ID`, `organization name`, `refresh token` or `API token`, and `API version` as input parameters. The workflow output includes the token expiry time.
Remove Host	Removes a VCF Automation SaaS or on-prem host from the plug-in inventory.
Validate Host	Validates the configuration of the VCF Automation SaaS or on-prem host.

Note:

In VCF Automation 9.1.1 and later, the Add vRA Host and Update vRA Host workflows accept an API token as an alternative to the refresh token. The API token does not require VMware Identity Manager as the identity provider. For information about generating an API token, see Create a Service Account and Get a Refresh Token in VCF Automation for VM Apps.

Multi-Tenancy Support

You can configure the Automation plug-in to work with your multi-tenant environment. You must add a dedicated VCF Automation host for each tenant (organization), using the tenant FQDN as the host name. For each VM Apps organization that is not the default organization, configure CNAME records so that the organization identifier is included in the hostname as a subdomain. For example, for a organization named *org1*, the hostname is *org1.vcfa-fqdn*. For the default organization, the hostname is *vcfa-fqdn*.

If you use the plug-in with a VCF Operations orchestrator instance that is embedded in VCF Automation, the default connection does not use an organization-specific URL. Instead, it uses the default VCF Automation host name.

Token Expiry Behavior

When a host is created with an API token, note the token expiry time in the workflow output. When the API token expires, you must update the host.

Invoking REST Operations on Hosts

The VCF Operations orchestrator plug-in for VCF Automation for VM Apps supports generic REST operations on dynamically created hosts. You can run the default plug-in workflows to invoke any public VCF Automation APIs.

The plug-in supports the following REST operations.

Workflow	Description
Get operation	Generic REST client support for HTTP GET operation.
Put operation	Generic REST client support for HTTP PUT operation.
Post operation	Generic REST client support for HTTP POST operation.
Patch operation	Generic REST client support for HTTP PATCH operation.
Delete operation	Generic REST client support for HTTP DELETE operation.

Add a VCF Automation host

You add a VCF Automation for VM Apps host and configure the connection parameters by running a VCF Operations orchestrator workflow.

1. Navigate to **Library** > **Workflows** and enter the `vra` and `configuration` tags in the workflow search box.
2. Locate the **Add vRA host** workflow and click **Run**.
3. Enter a unique name for the host.
4. Enter the URL address of the host.

   The format for the URL address is https://*`vcfa.fqdn`*.
5. Select whether to install the SSL certificates automatically without user confirmation.
6. On the **User credentials** tab, select the type of connection to the host.

Option	Actions
Shared Session	Connect using the credentials for a VCF Automation user that you provide in the **Authentication User Name** and **Authentication Password** text boxes. This option is supported for vIDM users. Alternatively, you can use an API token that you provide in the **API Token** text box. The API token option is supported in VCF Automation 9.1.1 and later and does not require vIDM.
Per User Session	Connect using the credentials of the user that is currently logged in.

7. Click **Run**.

You have added a VCF Automation on-prem host.

Using the VCF Automation plug-in infrastructure administration workflows

You can use the infrastructure administration workflows to manage cloud accounts, cloud zones, machines, and projects.

Cloud Accounts Workflows

The Cloud Accounts category contains workflows that you can use for managing vSphere cloud accounts.

To access these workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `vsphere_cloud_accounts` tag in the workflow search box.

Workflow Name	Description
Create vSphere Cloud Account	Adds a vCenter cloud account in VCF Automation and to the plug-in inventory in VCF Operations orchestrator. See Add a vSphere cloud account.
Create vSphere Cloud Account Async	Creates a cloud account asynchronously with the selected configuration. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested cloud account by using the `RequestService` and `RequestTracker` objects and the getRequestTrackerById action.
Delete vSphere Cloud Account	Removes a vCenter cloud account in VCF Automation and from the plug-in inventory in VCF Operations orchestrator.
Update vSphere Cloud Account	Updates a vCenter cloud account in VCF Automation and in the plug-in inventory in VCF Operations orchestrator.
Update vSphere Cloud Account Async	Requests to update a cloud account asynchronously with the selected configuration. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested cloud account by using the `RequestService` and `RequestTracker` objects and the getRequestTrackerById action.

Cloud Zones Workflows

To access the Cloud Zones workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `cloud_zones` tag in the workflow search box.

Workflow Name	Description
Create Cloud Zone	Adds a cloud zone in VCF Automation and to the plug-in inventory in VCF Operations orchestrator. See Add a cloud zone.
Delete Cloud Zone	Removes a cloud zone in VCF Automation and from the plug-in inventory in VCF Operations orchestrator.
Update Cloud Zone	Updates a cloud zone in VCF Automation and in the plug-in inventory in VCF Operations orchestrator.

Disks Workflows

To access the Disks workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `disks` tag in the workflow search box.

Workflow Name	Description
Create Disk	Creates a disk (block device) synchronously and adds it to the plug-in inventory in VCF Operations orchestrator. See Add a disk.
Delete Disk	Requests to delete a disk (block device) with the selected configuration asynchronously. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested machine disk by using the `RequestService` and `RequestTracker` objects.

In addition to the default workflows, the Automation plug-in supports the Promote disk and Resize block device operations.

Machines Workflows

To access the Machines workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `machines` tag in the workflow search box.

Workflow Name	Description
Create Machine	Creates a virtual machine in VCF Automation and adds it to the plug-in inventory in VCF Operations orchestrator. See Add a machine.
Create Machine Async	Asynchronously creates a virtual machine with the selected configuration. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested machine by using the `RequestService` and `RequestTracker` objects and the `getRequestTrackerById` action.
Delete Machine	Removes a virtual machine from VCF Automation and from the plug-in inventory in VCF Operations orchestrator. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested machine by using the `RequestService` and `RequestTracker` objects and the `getRequestTrackerById` action.
Resize Machine	Resizes a VCF Automation machine asynchronously. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested machine by using the `RequestService` and `RequestTracker` objects. Note that the Resize Machine workflow does not validate the maximum value for the CPU Count, Core Count, and Memory inputs. The VCF Operations orchestrator API does not support fetching the maximum configured values for these machine attributes.
Update Machine Custom Properties	Updates virtual machine custom properties.
Update Machine Tags	Updates virtual machine tags.

In addition to the default workflows, the Automation plug-in supports various machine power operations, including Power On/Off, Reset, Reboot, Resize, Shutdown.

To access these actions, navigate to **Library** > **Actions**, and search for the `com.vmware.library.vra.infrastructure.machine.power` tag in the action search box.

Note that the power operation action fails if you run an unsupported operation or if the target criteria is not met.

Machine Disks Workflows

To access the Machine Disks workflows in the VCF Operations orchestrator Client, navigate to **Library** > **Workflows**, and enter the `machines` and `disks` tags in the workflow search box.

Workflow Name	Description
Attach Machine Disk	Attaches a VCF Automation machine disk (block device) asynchronously.
Detach Machine Disk	Dettaches a VCF Automation machine disk (block device) asynchronously.

On request submission, the Machine Disks workflows return a `RequestTracker` object. You can get the status of the requested machine disks by using the `RequestService` and `RequestTracker` objects.

Machine Snapshots Workflows

To access the Machine Snapshots workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `machines` and `snapshots` tags in the workflow search box.

Workflow Name	Description
Create Machine Snapshot	Requests to create a machine snapshot with the selected configuration asynchronously.
Delete Machine Snapshot	Requests to delete a machine snapshot with the selected configuration asynchronously.
Revert Machine Snapshot	Reverts a VCF Automation machine snapshot asynchronously.

On request submission, the Machine Snapshots workflows return a `RequestTracker` object. You can get the status of the requested machine snapshots by using the `RequestService` and `RequestTracker` objects.

Networks Workflows

To access the Networks workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `networks` tag in the workflow search box.

Workflow Name	Description
Create Network	Creates a network asynchronously. See Add a network.
Delete Network	Requests to delete a network with the selected configuration asynchronously. On request submission, the workflow returns a `RequestTracker` object. You can get the status of the requested network operation by using the `RequestService` and `RequestTracker` objects.

Projects Workflows

To access the Projects workflows in VCF Operations orchestrator, navigate to **Library** > **Workflows**, and enter the `projects` tag in the workflow search box.

Workflow Name	Description
Create Project	Adds a project in VCF Automation and to the plug-in inventory in VCF Operations orchestrator. See Add a project.
Delete Project	Removes a project in VCF Automation and from the plug-in inventory in VCF Operations orchestrator.
Update Project	Updates a project in VCF Automation and in the plug-in inventory in VCF Operations orchestrator.
Update Project Resource Metadata	Updates the resource metadata, such as tags, that are associated with a project.

Add a vSphere cloud account

You add a vSphere cloud account and configure its parameters by running a VCF Operations orchestrator workflow.

- To configure and work with cloud accounts in VCF Automation, verify that you have the necessary credentials. See Credentials required for working with cloud accounts in VCF Automation.

- For information about creating vSphere cloud accounts, see Create a vCenter cloud account in VCF Automation.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `cloud_accounts` tag in the workflow search box.
3. Locate **Create vSphere Cloud Account** workflow and click **Run**.
4. Select a VCF Automation host for which you want to configure a vSphere cloud account.
5. Enter a name for the cloud account.
6. Configure the **vCenter** credentials.
   a) Enter the IP address or fully-qualified domain name of your **vCenter** server.
   b) Select whether to accept self-signed certificated automatically without user confirmation.
   c) Provide the **vCenter** server user name and password.
7. If you want to add tags to support a tagging strategy, add capability tags.

   You can add tags now, or later when you edit the cloud account. For information about tagging, see How do I use tags to manage Automation Assembler resources and deployments.
8. Click **Run**.

You have added a vSphere cloud account.
Configure a cloud zone within the cloud account you just created. See Add a cloud zone.

Add a cloud zone

You add a cloud zone and configure its parameters by running a VCF Operations orchestrator workflow.

- Verify that you configured at least one cloud account. See Add a vSphere cloud account.
- For information about cloud zones, see Learn more about Automation Assembler cloud zones.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `cloud_zones` tag in the workflow search box.
3. Locate **Create Cloud Zone** workflow and click **Run**.
4. On the **Summary** tab, configure the cloud zone properties.

Option	Description
Host	Select the VCF Automation host for which you want to configure a cloud zone.
Region	Select a cloud region.
Name	Enter a name for the cloud zone.
Description	Add a description.
Placement policy	Placement policy drives host selection for deployments within the specified cloud zone. Apply one of the following placement strategies: • DEFAULT. Places compute resources on random hosts. • BINPACK. Places compute resources on the most loaded host that has enough resources to run the given compute. • SPREAD. Provisions compute resources, at a deployment level, to the cluster or host with the least number of virtual machines. For vSphere, Distributed Resource Scheduler (DRS) distributes the virtual machines across the hosts.

5. On the **Capabilities** tab, add capability tags if you want to support a tagging strategy.

   You can add tags now, or later when you edit the cloud account. For information about tagging, see How do I use tags to manage Automation Assembler resources and deployments.

6. Click **Run**.

You have added a cloud zone.
Configure a project and add your cloud zones to it. See Add a project.

Add a disk

You create a disk and configure its parameters by running a VCF Operations orchestrator workflow.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `disks` tag in the workflow search box.
3. Locate **Create Disk** workflow and click **Run**.
4. Select a VCF Automation host for which you want to configure a disk.
5. Enter a name for the disk.
6. Optional: Add a description for the disk.
7. Select a project to which to add the disk.
8. On the **Specification** tab, define the capacity of the disk in gigabytes.
9. Configure any additional settings for the disk, such as tags, constraints, and custom properties.
10. Click **Run**.

You have added a disk in VCF Automation and to the VCF Operations orchestrator plug-in inventory.

Add a machine

You add a virtual machine and configure its parameters by running a VCF Operations orchestrator workflow.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `machines` tag in the workflow search box.
3. Locate **Create Machine** workflow and click **Run**.
4. Select a VCF Automation host for which you want to configure a machine.
5. Enter a name for the machine.
6. Define how many machines you want to provision.
7. Select a project to which to add the machine.
8. Go to the **Flavors and Images** tab.
   a) Select the type of image used for the machine.
   b) Select the direct image reference used for the machine.
      This setting is required if you have multiple zones of the same cloud connection configured under image mapping.
   c) Select the flavor of the machine.
      If multi-zone cloud accounts are attached to a project, machines are created for AWS, GCP, and Azure only if the flavor reference is not attached to the machine specification object.
   d) Optional: Select the provider-specific flavor reference for the machine.
      This setting is required if you have multiple zones of the same cloud connection configured under flavor mapping.
9. Configure any additional settings for the machine, such as tags, custom properties, remote access, and others.
   Keep in mind the following considerations:
   - When you configure disk specifications, you must select a block device. The block device must be a data disk and must be available to attach to the machine.
     You can create a block device using the `createBlockDevice` method. For more information about this method, go to **Plugins** > **VRA** > **Objects** > **VraDiskService** in the VCF Operations orchestrator API Explorer.

- When you configure network specifications, you must select a fabric network. Do not specify a Network ID. Network ID and fabric network can't be passed at the same time.

For more information about these settings, consult the signpost help.

10. Click **Run**.

You have added a machine in VCF Automation and to the VCF Operations orchestrator plug-in inventory.

Add a network

You create a network and configure its parameters by running a VCF Operations orchestrator workflow.

Before you add a network in VCF Operations orchestrator, make sure that you have an on-demand network isolation policy set up under Network Profiles in VCF Automation. See Learn more about network profiles in VMware Aria Automation.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `networks` tag in the workflow search box.
3. Locate the **Create Network** workflow and click **Run**.
4. Select a VCF Automation host for which you want to configure a network.
5. Enter a name for the network.
6. Optional: Add a description for the network.
7. Select a project to which to add the network.
8. On the **Specification** tab, configure the following settings.
   a) Select whether you want to activate or deactivate outbound access.

      By default, outbound access is activated, which creates an outbound network.
   b) Select whether you want to create a gateway in the network.
   c) Enter the deployment ID of your VCF Automation instance.
9. Configure any additional settings for the network, such as tags, constraints, and custom properties. For example:
   a) Use constraints to select a network policy defined in VCF Automation.
   b) Use custom properties to select the Network type in the workflow request, for example, `networkType:private`.

      The Automation plug-in supports the `private`, `routed`, and `outbound` network types.
10. Click **Run**.

You have added a network in VCF Automation and to the VCF Operations orchestrator plug-in inventory.

Add a project

You add a project and configure its parameters by running a VCF Operations orchestrator workflow.

- Verify that you configured at least one cloud account. See Add a vSphere cloud account.
- Verify that you configured at least one cloud zone. See Add a cloud zone.
- For information about projects, see Adding and managing Automation Assembler projects.

1. Log in to VCF Operations orchestrator as an administrator.
2. Navigate to **Library** > **Workflows** and enter the `projects` tag in the workflow search box.
3. Locate **Create Project** workflow and click **Run**.
4. On the **Summary** tab, select the VCF Automation host for which you want to configure a project, and enter a name for the project.
5. On the **Provisioning: Zones** tab, add one or more cloud zones.
   a) Configure the cloud zone properties.
   b) Select a placement policy.
6. Go to the **Provisioning: Resource Tags & Constraints** tab, add tags and constraints for your project.
7. Go to the **Provisioning: Custom Properties, Custom Naming & Request Timeout** tab.
   a) Add custom properties that you want added to all requests in the project.
   b) Specify the naming template for machines, networks, security groups, and disks in the project.
   c) If the workloads requested for this project take more than two hours to deploy, enter a longer value for the **Timeout**. The default value is 2 hours.
8. Click **Run**.

You have added a project.

Using the VCF Operations orchestrator Plug-In for VCF Automation for All Apps

The VMware Cloud Foundation Operations orchestrator™ Plug-in for VMware Cloud Foundation Automation™ for All Apps integrates VCF Operations orchestrator with VCF Automation. With the plug-in, you can run VCF Operations orchestrator workflows from your VCF Automation instance. You can use the workflows that are provided with the plug-in to deploy and manage VCF Automation resources in VCF Automation organizations.

Using the Default Automation plug-in for All Apps Workflows and Actions

The Automation plug-in provides out-of-the-box workflows for common tasks, such as VCF Automation host configuration. For a full list of available workflows, navigate to **Library** > **Workflows** > **VCFA**. For example, you can use the **Add a VCF Automation All Apps host** workflow to add a new host.

The plug-in library also contains predefined actions that you can use to build you own custom workflows. To access these actions, navigate to **Library** > **Actions**, and enter `com.vmware.library.vcfa` in the action search box.

Using the Automation plug-in for All Apps Inventory

The VCF Operations orchestrator inventory supports objects for VCF Automation. hosts, supervisor namespaces, and Supervisor resources.

To display all available inventory objects, navigate to **Administration** > **Inventory** > **VCF Automation for All Apps**.

Add a VCF Automation for All Apps host

You add a VCF Automation for All Apps host and configure the connection parameters by running a VCF Operations orchestrator workflow.

1. Navigate to **Library** > **Workflows** and enter the `vcfa` tag in the workflow search box.
2. Locate the **Add a VCF Automation host** workflow and click **Run**.
3. Enter a name for the connection.
4. Enter a unique name for the host.
5. Enter the organization name.
6. Enter the Kubernetes API version to use with the supervisor API.
   To use the default API version, leave this textbox empty.
7. Select whether to install the SSL certificates automatically without user confirmation.

8. Select the type of connection to the host.

Option	Actions
Shared Session	Connect using a user API token.
Per User Session	Connect using the credentials of the user that is currently logged in.

9. Click **Run**.

You have added a VCF Automation for All Apps host.

Using the XML Plug-In

You can use the XML plug-in to run workflows that create and modify XML documents.

The XML plug-in adds an implementation of a Document Object Model (DOM) XML parser to the VCF Operations orchestrator JavaScript API. The XML plug-in also provides some sample workflows to demonstrate how you can create and modify XML documents from workflows.

Alternatively, you can use the ECMAScript for XML (E4X) implementation in the VCF Operations orchestrator JavaScript API to process XML documents directly in JavaScript. For an E4X scripting example, see *Developing Workflows with VCF Operations orchestrator*.

For information about E4X, go to the website of the organization that maintains the ECMA-357 standard.

Running the XML Plug-In Sample Workflows

You can run the XML plug-in sample workflows from the VCF Operations orchestrator Client to create and modify XML documents for testing purposes.

The workflows can create, read, or modify files, so you must have sufficient access rights to the working directory.

Orchestrator has read, write, and execute rights to a folder named `orchestrator`, at the root of the server system. Although workflows have permission to read, write, and execute in this folder, you must create the folder on the server system. If you use the Orchestrator Appliance, the folder is named `vco` and is located at `/var/run/vco`.

You can allow access to other folders by changing the settings for server file system access from workflows and JavaScript. See *Installing and Configuring VCF Operations orchestrator, Setting Server File System Access from Workflows and Actions*.

Create a Simple XML Document

You can run a workflow from the VCF Operations orchestrator Client to create a simple XML document for testing purposes.

- Verify that the user account you are logged in with has the necessary permissions to run XML workflows.
- Verify that you created the `c:/orchestrator` folder at the root of the Orchestrator server system or set access rights to another folder.

1. Navigate to **Library** > **Workflows** and enter the `xml` and `samples_xml_(simple)` tags in the workflow search box.
2. Locate the **Create a simple XML document** workflow and click **Run**.
3. Enter the filepath to the XML document to create.
   For example, `c:/orchestrator/`*`filename`*`.xml`.
4. Click **Run**.

The workflow creates an XML document that contains a list of users. The attributes for each entry are `user ID` and `name`.

Find an Element in an XML Document

You can run a workflow from the VCF Operations orchestrator Client to find an element in the XML created by the Create a simple XML document workflow.

- Verify that the user account you are logged in with has the necessary permissions to run XML workflows.
- Verify that you created the `c:/orchestrator` folder at the root of the Orchestrator server system or set access rights to another folder.

1. Navigate to **Library** > **Workflows** and enter the `xml` and `samples_xml_(simple)` tags in the workflow search box.
2. Locate the **Find element in document** workflow and click **Run**.
3. Type the filepath to the XML document.
   For example, `c:/orchestrator/`*`filename`*`.xml`.
4. Click **Run**.
   The workflow searches for an element and displays the result in the system log.

To view the result, select the completed workflow run in the VCF Operations orchestrator Client and click **Logs** on the **Schema** tab.

Modify an XML Document

You can run a workflow from the VCF Operations orchestrator Client to modify the XML that the Create a simple XML document workflow creates.

- Verify that the user account you are logged in with has the necessary permissions to run XML workflows.
- Verify that you created the `c:/orchestrator` folder at the root of the Orchestrator server system or set access rights to another folder.

1. Navigate to **Library** > **Workflows** and enter the `xml` and `samples_xml_(simple)` tags in the workflow search box.
2. Locate the **Modify XML document** workflow and click **Run**.
3. Provide the input and output filepaths.
   a) Type the filepath to the XML document to modify.
      For example, `c:/orchestrator/`*`filename`*`.xml`.
   b) Type the filepath to the modified XML document.
      For example, `c:/orchestrator/`*`filename`*`.xml`.

      Note:
      If you type the same filepath in both fields, the workflow overwrites the original file with the modified file. If you type an output filepath to a file that does not exist, the workflow creates a modified file.
4. Click **Run**.

The workflow searches for an element and modifies the entry where the element is found.

Create an Example Address Book from XML

You can run a workflow from the VCF Operations orchestrator Client to create an address book for testing purposes.

- Verify that the user account you are logged in with has the necessary permissions to run XML workflows.
- Verify that you created the `c:/orchestrator` folder at the root of the Orchestrator server system or set access rights to another folder.

1. Navigate to **Library** > **Workflows** and enter the `xml` and `samples_xml_(address_book)` tags in the workflow search box.
2. Locate the **Full address book test** workflow and click **Run**.
3. Type the path to the address book folder.
   For example, `c:/orchestrator/`*`foldername`*.

The workflow automatically creates the folder if it does not exist.

4. Click **Run**.

The workflow creates a DTD, an XML, and a CSS file, appends the stylesheet, and stores the files in the specified folder.